

Welcome

Welcome to **DS-API** documentation! Kick-off things with the [quickstart](#) guide to look at a few recipes that will help you with common tasks. For further details, refer to the [API Reference](#).

User Guides

The user guide contains mostly information on what the DS-API is as well as what it is not! Some recipes/patterns for common tasks are also provided, with additional context in what's happening under the hood.

1. [Quickstart](#)
2. [String Indexing](#)

API Reference

Check the [Python reference](#) section for details on all the Python functions, classes, and all that jazz.

Java Reference

At the moment we are migrating the DS-API Java reference. Please check back later for updates.

ds_api

A pythonic interface to the Java DS-API. It provides tools to interact with Pulse in a programmatic way.

DS_API_CLASSPATH module-attribute

```
DS_API_CLASSPATH: str = 'com.feedzai.common.dsapi.'
```

MANAGED_LISTS_CLASSPATH module-attribute

```
MANAGED_LISTS_CLASSPATH: str = MODEL\_CLASSPATH + 'managedlists.'
```

MODEL_CLASSPATH module-attribute

```
MODEL_CLASSPATH: str = PULSE\_API\_CLASSPATH + 'apps.model.'
```

PULSE_API_CLASSPATH module-attribute

```
PULSE_API_CLASSPATH: str = 'com.feedzai.pulse.api.'
```

PULSE_HOST_ENV module-attribute

```
PULSE_HOST_ENV = 'DS_API_PULSE_HOST'
```

PULSE_RMI_PORT_ENV module-attribute

```
PULSE_RMI_PORT_ENV = 'DS_API_PULSE_RMI_PORT'
```

USER_ID_ENV module-attribute

```
USER_ID_ENV = 'DS_API_USER'
```

USER_NAME_ENV module-attribute

```
USER_NAME_ENV = 'DS_API_USER_NAME'
```

String Indexing

What is String Indexing?

String Indexing comes in many forms but in all cases it boils down to pre-processing data to create new Machine Learning model feature(s) from raw string/categorical/text field(s). The generated features are always numerical – either scalars or vectors.

Pulse and the `string_indexing` library provide support for [Frequency Indexing](#). In some circles this is also known as **Count Indexing**.

There are two steps to use (frequency) indexers in Pulse:

1. **Fit** - Produce an indexer from train data using the `string_indexing` DS-API library:

```
indexer = string_indexing.fit(train_data, feature_name)
```

2. **Transform** - Transform input data using that indexer to add the new feature: Performed inside Pulse plans through the new `FrequencyIndexerUDF`.

About Frequency Indexing

Frequency Indexing is one of the simplest yet surprisingly effective forms of Indexing.

The pros of Frequency Indexing are its simplicity in terms of tweaking and robustness to over-fit when compared with other forms of indexing. Internal Feedzai benchmarks show recall gains of 8-15pp or 20-40%.

Say you have a string field with the country. Instead of defining the possible values of a categorical field, you can use Frequency Indexing.

The first step is to `fit` a new indexer with a train dataset that includes that field. This will:

1. Compute the number of occurrences of all the categories in that dataset.
2. Rank them in descending number of occurrences. The most frequent value will become 0, the second most frequent 1, and so on and so forth.
3. Store that mapping (string -> rank) and default output value for unknown input values.

What if you have values in the test dataset that you haven't seen before?

That's expected. Let's call those the "other" values. The Frequency Indexer will map those to the integer N, where N is the total number of categories stored in the indexer mapping.

This ensures the output value of the indexer is well defined even for unknown values.

Indexer clipping and good Data Science modeling practices

Is there a possibility your indexer will eventually receive new or unforeseen values?

If so, you should train your **Machine Learning model** by clipping your indexers first so that the indexed train dataset has "other" values that the indexer will convert to N. These train examples will make the model generalize better for new "other" values observed while predicting.

How to perform clipping^{AI}

Two forms of clipping are supported:

- Max categories (**M**): During the indexer fit step, only the **M** most frequent categories from the train data are kept in the indexer.
- Min samples per category (**S**): Any category/string seen less than **S** times during the indexer fit is dropped from the indexer.

These parameters can be tuned, but sensible values are already used by default, which work well for most scenarios.

Note

Although it can be tweaked, Pulse's `FrequencyIndexerUDF` will by default clip the indexer to **90%** of its input categories. This means that even if you forget to tweak **M** and **S**, the pre-processing pipeline already prevents over-fit and ensures that your Machine Learning model's train and evaluation data will contain "other" values.

Workflow^{AI}

The whole user journey when relying on indexing is:

1. Create model with Indexed features in the Data Science environment:
2. Create your Pulse indexer(s) using the `string_indexing` DS-API module.
3. Go to your Pulse Plan and using that Pulse Indexer and the new Pulse `FrequencyIndexerUDF` convert a raw field to an indexed feature.
4. Train a Machine Learning model with those features.
5. Put the model (and indexers) in production

To put the model in another environment, carry the model, features plan and the lists associated to the indexers. Indexers are currently stored as Pulse Lists whose name starts with prefix `"{StringIndexer}"`. Hence, follow the usual procedure to copy lists across environments.

Quickstart

The modules provided by the Python DS-API package provide convenient utilities that allow interacting with Pulse and performing common data science tasks. This is a list of the available libraries:

- `ds_api` . A pythonic interface for Pulse using `pyjnius`. This library is the entry point to interacting programmatically with Pulse. It provides tools that connect to Pulse and perform common data science operations, such as accessing data sources, running plans, training and evaluating models, etc.
- `ds_api.exploration` . Utilities for efficient exploratory data analysis via PySpark. You can efficiently profile your data, easily generate detailed reports, and quickly access personalized results of the exploration in the form of statistics, validations and visualizations.
- `ds_api.sampling` . Efficient sampling utilities through PySpark. It provides a stratified and a random sampling strategy, but can be easily extended to user defined strategies. It supports smart optimized sampling that preserves the minimal history of entities, so historical profiles can be correctly computed while maximizing the sampling reduction power.

Recipes

The best learning resource to get familiar with the DS-API and its available libraries are the notebooks provided in the JupyterLab environment in Pulse.

DS-API

The entry point to the DS-API and its functionalities is to connect to the Pulse server and instantiate a `Feedzai` object:

```
from ds_api import Feedzai, Server

PULSE_SERVER_ADDRESS = "localhost"
PULSE_SERVER_PORT = 1099
# Usual pulse.manager.rmi.localregistryport port. Confirm this in Admin UI, inside Pulse configuration.
pulse = Server.connect(address=PULSE_SERVER_ADDRESS, port=PULSE_SERVER_PORT)
feedzai = Feedzai(pulse=pulse)
```

Now you can connect to a Pulse application, and for instance list its available data sources:

```
app = feedzai.app(name="app-name")
app.pullDataSources()
# ["data source1", "data source2"]
```

To analyze a datasource:

```
DATASOURCE = "data source1"
datasource = app.pullDataSourceByDesc(DATASOURCE)
datasource.analyze()
```

To convert a Pulse datasource to a PySpark DataFrame:

```
spark_session = feedzai.get_spark_session()
data = feedzai.pulse_to_pyspark(spark_session, app, DATASOURCE)
```

More detailed demonstrations on how to use the DS-API can be found in the following notebooks that are available in the JupyterLab integration with Pulse:

- **01_getting_started.ipynb** - shows the first steps of using the DS-API: loading data from/to Pulse and converting data to a format that you can use in JupyterLab.

- Demonstration of how to interact with Pulse's data sources, plans and models using the DS-API:
 - **01_datasources.ipynb**
 - **02_plans.ipynb**
 - **03_models.ipynb**
 - **04_data_loading.ipynb** - shows how to use the DS-API to programmatically load data into Pulse.
-

Exploration

Continuing from the example in the previous section. To profile a PySpark DataFrame for exploration:

```
from ds_api.exploration import compute_stats

TIMESTAMP_FIELD = "timestamp"
COST_FIELD = "amount"
TARGET_FIELD = "fraud"
TARGET_VALUE = "true"
TARGET = (TARGET_FIELD, TARGET_VALUE)

results = compute_stats(
    data=data,
    timestamp_field=TIMESTAMP_FIELD,
    target=TARGET,
    cost_field=COST_FIELD
)
```

Now you can display the exploration report (in JupyterLab) and save it:

```
from IPython.core.display import HTML

HTML(results.report)
results.save_report_html("report.html")
```

You can also quickly access statistics and visualizations using a convenient API. Some examples:

```
results.time_dependencies
results[COST_FIELD].quantile()
results[TARGET_FIELD].frequencies()
results[TIMESTAMP_FIELD].plot_with_fraud_rate()
```

The notebook **05_data_exploration.ipynb**, available in Pulse's JupyterLab environment, provides a more exhaustive guide to the exploration library.

Sampling

Continuing from the examples in the previous sections. First initialize the desired sampler:

```
sampler = StratifiedRandomUndersampler(
    target_field=TARGET_FIELD,
    target_value=TARGET_VALUE,
    balance=0.9,
)
```

Create a filter if you want filter the events that will be marked by the sampling strategy:

```
from pyspark.sql import functions as F

EVENT_TYPE_FIELD = "event_type_field"
SCORABLE_EVENTS = ["scorable_event_type1", "scorable_event_type2"]
scorable_events_filter = F.col(EVENT_TYPE_FIELD).isin(SCORABLE_EVENTS)
```

Now you can sample the PySpark DataFrame. In this example the minimal history of the given entity field is preserved in the output dataset, so historical profiles can correctly be computed:

```
ENTITY_FIELD = "card_id"
MAX_WINDOW_SIZE = 30 # days

sampled_data = sampler.sample(
    train_data,
    filter_expr=scorable_events_filter,
    entity_field=ENTITY_FIELD,
    timestamp_field=TIMESTAMP_FIELD,
    recovery_days=MAX_WINDOW_SIZE
)
```

A more detailed illustration on how to use the sampling tools for model training and testing purposes is available in Pulse's JupyterLab environment via the notebook **06_data_sampling.ipynb**.

action_param [↗](#)

ActionParamTypeEnum [↗](#)

Bases: [JavaEnumWrapper](#)

The type of parameters of a rules project action parameter.

This allows the user that defines an action to characterize the parameters with a type. With this type, the screens that allow users to trigger an action will also be able to have an optimized UI to provide a better UX.

EXPRESSION [↗](#) class-attribute instance-attribute

EXPRESSION = auto()

MANAGED_LIST [↗](#) class-attribute instance-attribute

MANAGED_LIST = auto()

MANAGED_LIST_ITEMS [↗](#) class-attribute instance-attribute

MANAGED_LIST_ITEMS = auto()

NUMERIC [↗](#) class-attribute instance-attribute

NUMERIC = auto()

STRING [↗](#) class-attribute instance-attribute

STRING = auto()

WorkflowActionParam [↗](#)

```
WorkflowActionParam(desc: str, action_type: ActionParamTypeEnum, is_mandatory: bool, resource_id: Optional[str] = None)
```

Bases: [JavaWrapper](#)

Define a parameter of a rules project action.

Create a parameter for a workflow action.

Parameters:

Name	Type	Description	Default
<code>desc</code> ↗	<code>str</code>	The description / user visible name of the action parameter.	<i>required</i>
<code>action_type</code> ↗	ActionParamTypeEnum	The type of the action parameter.	<i>required</i>
	<code>bool</code>	A flag defining if the parameter is mandatory or not.	<i>required</i>

Name	Type	Description	Default
is_mandatory ¶			
resource_id ¶	str	The Id of the resource used if the type requires that information (e.g.: ActionParamTypeEnum.MANAGED_LIST). Default is None	None

action_type instance-attribute [¶](#)

```
action_type = require_type(action_type, ActionParamTypeEnum)
```

desc instance-attribute [¶](#)

```
desc = require_type(desc, str)
```

is_mandatory instance-attribute [¶](#)

```
is_mandatory = require_type(is_mandatory, bool)
```

resource_id instance-attribute [¶](#)

```
resource_id = require_type_or_none(resource_id, str)
```

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast staticmethod [¶](#)

```
auto_cast(java_object) -> WorkflowActionParam
```

Helper method called when a return obj comes from java to be auto converted to python.

adaptive_partitioning_config

AdaptivePartitioningConfig

```
AdaptivePartitioningConfig(predominant_count: int = DEFAULT_PREDOMINANT_COUNT,
partition_capacity: Optional[int] = None)
```

Bases: PartitioningConfig

Represents an Adaptive Partitioning configuration.

Create a new Adaptive Partitioning configuration.

Parameters:

Name	Type	Description	Default
<code>predominant_count</code>	int	The maximum number of values to compute the cardinality for when collecting statistics about the partitioning fields. By default, AdaptivePartitioningConfig.DEFAULT_PREDOMINANT_COUNT .	<code>DEFAULT_PREDOMINANT_COUNT</code>
<code>partition_capacity</code>	int	The maximum number of instances that should be allocated to each partition. By default, None. If this is not set, then a capacity shall be inferred automatically.	None

`DEFAULT_PREDOMINANT_COUNT` class-attribute instance-attribute

```
DEFAULT_PREDOMINANT_COUNT = DEFAULT_PREDOMINANT_COUNT
```

`partition_capacity` property

```
partition_capacity: Optional[int]
```

Get the maximum number of instances that should be allocated to each partition.

`predominant_count` property

```
predominant_count: int
```

Get the maximum number of values to compute the cardinality for.

`subclasses` class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

app

logger module-attribute

```
logger = getLogger(__name__)
```

App

```
App(name=None, id=None, pulse=None, **kwargs)
```

Bases: [JavaWrapper](#)

Represents a Pulse Application.

pulse instance-attribute

```
pulse = connect() if len(kwargs) == 0 else connect(**kwargs)
```

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

auto_cast staticmethod

```
auto_cast(java_object) -> App
```

Helper method called when a return obj comes from java to be auto converted to python.

pullById classmethod

```
pullById(pulse, id)
```

pullByName classmethod

```
pullByName(pulse, name)
```

connectors

Connectors

```
Connectors(pyjnius_java)
```

Bases: [JavaWrapper](#)

Connectors class that allows users to define if the connectors should use encryption or not.

Note

By connectors we mean the PulseConnector and AppEngineConnector that are used by the Java DS API to connect to Pulse or the AppEngine respectively.

subclasses **class-attribute** **instance-attribute** 

```
subclasses: List[JavaWrapper] = []
```

with_encryption **staticmethod** 

```
with_encryption()
```

Sets the Connectors to use encryption.

without_encryption **staticmethod** 

```
without_encryption()
```

Resets the connectors to use no encryption.

Start by fetching a data source by running

Then you can launch analyze jobs, run feature selection, call AutoML to generate features and more.

The schema from the data source.

slug property [¶](#)

```
slug: str
```

Sanitized name. Equivalent to Pulse 'internal name'.

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

analyze [¶](#)

```
analyze(default_breakdowns: bool = True, wait: bool = False, job_profile: Optional[JobProfile] = None, job_properties: Optional[Dict[str, str]] = None) -> bool
```

Analyze this datasource with Pulse.

By analyzing a data source, Pulse will launch a job to compute approximate histograms for each numerical feature.

The default analysis configuration will also include histogram breakdowns with **all categorical variables**. For each categorical variable in the data source, there will be a new histogram in each numerical feature.

data sources with many categorical fields

It is recommended to set `default_breakdowns=False` when there are many categorical fields. Otherwise, the amount of metadata that is generated might compromise the performance of Pulse.

The datasource analysis will be run with Pulse's default config for datasource analysis.

Parameters:

Name	Type	Description	Default
<code>default_breakdowns</code> ¶	bool	Whether the analysis should include breakdowns for all categorical fields or not. Default is <code>True</code> .	<code>True</code>
<code>wait</code> ¶	bool	Whether to wait for the datasource analysis to finish. The data analysis outcome will be logged. Default is <code>False</code> .	<code>False</code>
<code>job_profile</code> ¶	JobProfile	Pulse UI custom job profile.	<code>None</code>
<code>job_properties</code> ¶	dict	Additional spark properties, e.g. <code>{"spark.executor.memory": "1g"}</code> . By default, no additional spark properties are added.	<code>None</code>

Returns:

Type	Description
bool	Whether the analysis started or not.

create classmethod [¶](#)

```
create(app: App, name: str, path: str, file_format: str = 'csv', schema: Schema | None = None, **kwargs)
```

Creates a new data source in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	The App where the datasource will be created.	<i>required</i>
<code>name</code> ¶	<code>str</code>	The name of the data source as it appears in the UI.	<i>required</i>
<code>path</code> ¶	<code>str</code>	The path of the file containing the dataset.	<i>required</i>
<code>file_format</code> ¶	<code>str</code>	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	<code>'csv'</code>
<code>schema</code> ¶	<code>Schema</code>	The Pulse Schema to use. Optional. If not specified, the Schema will be inferred by Pulse. Only used for csv files.	<code>None</code>
<code>kwargs</code> ¶	<code>Dict</code>	Keyword arguments used for the configuration of the DataSource and for the SchemaInference job.	<code>{}</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Only used for csv files. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Only used for csv files. Defaults to <code>','</code> .
<code>missing_values</code>	<code>set of str</code>	Set of strings with values for empty value. Only used for csv files. Defaults to <code>{',', 'NA', 'NULL'}</code> .
<code>quote</code>	<code>str</code>	The character used to denote the start and end of a quoted item. Only used for csv files. Defaults to <code>"'</code> .
<code>line_separator</code>	<code>str</code>	The string used mark the end of line. Only used for csv files. Defaults to <code>'\n'</code> .
<code>encoding</code>	<code>str</code>	Encoding to use for UTF when reading/writing. Only used for csv files. Defaults to <code>'UTF-8'</code> .
<code>cat_threshold</code>	<code>int</code>	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15.
<code>job_properties</code>	<code>Dict</code>	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse PropertiesProfile.
<code>n_records</code>	<code>int</code>	The number of records that will be read per partition when performing datasource schema inference. Will only be used if no Schema is specified. Defaults to 15000.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Defaults to None.

Name	Type	Description
<code>merge_schema</code>	bool	Specify if schema merging should be allowed. Only used for parquet files. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
DataSource	DataSource object corresponding to the Pulse datasource created.

create_automl_fe_plan [↗](#)

```
create_automl_fe_plan(
    semantics: Sequence[SemanticField],
    timestamp_field: str,
    realtime_windows: Optional[Sequence[TimeModel]] = None,
    scheduled_windows: Optional[Sequence[TimeModel]] = None,
    plan_name: Optional[str] = None,
    profile_filter: Optional[str] = None,
    compute_unfiltered_metrics: bool = False
) -> JavaWrapper
```

Creates automl's feature engineering plan.

Parameters:

Name	Type	Description	Default
<code>semantics</code> ↗	sequence of SemanticField	Sequence of semantic objects to define the transformations.	<i>required</i>
<code>timestamp_field</code> ↗	str	The name of the timestamp field.	<i>required</i>
<code>realtime_windows</code> ↗	sequence of TimeModel	Realtime windows to use in the metrics.	None
<code>scheduled_windows</code> ↗	sequence of TimeModel	Scheduled windows to use in the metrics.	None
<code>plan_name</code> ↗	str	Name of the resulting feature engineering plan. In case it is not provided, a name is automatically generated.	None
<code>profile_filter</code> ↗	str	Filter to be used in metrics. Fields must be enclosed with the pattern <code>\${field}</code> .	None
<code>compute_unfiltered_metrics</code> ↗	bool	Boolean that controls if unfiltered metrics are also computed, in case a profile filter is defined. Default is False.	False

Returns:

Type	Description
JavaWrapper	The created plan.

Examples:


```
>>> app = Feedzai().app(name="sample")
>>> dataset: DataSource = app.pullDataSourceByDesc("data")
>>> real_time_windows = (TimeModel(1, TimeUnit.DAY),)
>>> semantics = (
...     TaggedField("card", tags=["card", "grouping-entity"]),
...     TaggedField("ip", tags=["ip", "entity", "grouping-entity"]),
...     TaggedField("amount", tags=["amount"]),
...     CompositeField(
...         "card, customer", tags=["grouping-entity"], fields=["card", "customer"]
...     ),
... )
>>> plan = dataset.create_automl_fe_plan(
...     semantics=semantics,
...     timestamp_field="timestamp1",
...     realtime_windows=real_time_windows,
...     plan_name="test fe plan",
...     profile_filter="${event_type} == 'TRANSACTION' and ${scorable} == true"
... )
```

get_dtypes [¶](#)

```
get_dtypes(pyspark_context=False)
```

Returns a dictionary mapping field names to DataScienceFields.

Parameters:

Name	Type	Description	Default
pyspark_context ¶		Indicates if this is a pyspark context or a Pulse context. In a pyspark context the type of time fields will be given as timestamp in seconds (this is the default unit after the pulse_to_pyspark transformation), otherwise the actual type from Pulse will be returned.	False

Returns:

Type	Description
Dict	Dictionary with field description as keys and the corresponding DataScienceField type as values.

run_automl_feature_engineering [¶](#)

```
run_automl_feature_engineering(semantics: Sequence, timestamp_field: str = 'event_occurred_at', realtime_windows: Optional[Sequence[TimeModel]] = None, scheduled_windows: Optional[Sequence[TimeModel]] = None, plan_name: Optional[str] = None, output_datasource_name: Optional[str] = None, profile_filter: Optional[str] = None, compute_unfiltered_metrics: bool = False, output_path: Optional[str] = None) -> str
```

Runs automl's feature engineering on this datasource.

Parameters:

Name	Type	Description	Default
semantics ¶	sequence of SemanticField	Sequence of semantic objects to define the transformations.	required
timestamp_field ¶	str	The name of the timestamp field.	'event_occurred_at'

Name	Type	Description	Default
<code>realtime_windows</code> ↗	sequence of TimeModel	Realtime windows to use in the metrics.	None
<code>scheduled_windows</code> ↗	sequence of TimeModel	Scheduled windows to use in the metrics.	None
<code>plan_name</code> ↗	str	Name of the resulting feature engineering plan. In case it is not provided, a name is automatically generated.	None
<code>output_datasource_name</code> ↗	str	Name of the feature engineered datasource. In case it is not provided, the process created a derived name from the input source.	None
<code>profile_filter</code> ↗	str	Filter to be used in metrics. Fields must be enclosed with the pattern <code>\${field}</code> .	None
<code>compute_unfiltered_metrics</code> ↗	bool	Boolean that controls if unfiltered metrics are also computed, in case a profile filter is defined. Default is False.	False
<code>output_path</code> ↗	str	The path where the new datasource should be stored. In case it is not provided, the new datasource is saved in the source's path.	None

Returns:

Type	Description
str	The plan execution id.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> dataset: DataSource = app.pullDataSourceByDesc("data")
>>> real_time_windows = (TimeModel(1, TimeUnit.DAY),)
>>> semantics = (
...     TaggedField("card", tags=["card", "grouping-entity"]),
...     TaggedField("ip", tags=["ip", "entity", "grouping-entity"]),
...     TaggedField("amount", tags=["amount"]),
...     CompositeField(
...         "card, customer",
...         tags=["grouping-entity"], fields=["card", "customer"]
...     ),
... )
>>> plan_execution_id = dataset.run_automl_feature_engineering(
...     semantics=semantics,
...     timestamp_field="timestamp1",
...     realtime_windows=real_time_windows,
...     plan_name="test fe plan",
...     output_datasource_name="output data",
...     profile_filter="${event_type} == 'TRANSACTION' and ${scorable} == true"
... )
```

run_feature_selection [¶](#)

```
run_feature_selection(target_field: str, threshold: float = 0.9, sample_size: int = 200000, max_features: int | None = None, job_properties: dict[str, str] | None = None, job_profile: JobProfile | None = None)
```

Run feature selection on the Datasource.

Features are selected so that the cumulative sum of the feature importance on the selected features is below the given threshold. A maximum of max_features will be returned. The feature importance is computed from an H2O GBT model run on a sample of the data.

Parameters:

Name	Type	Description	Default
<code>target_field</code> ¶	str	The field to be used as target.	<i>required</i>
<code>threshold</code> ¶	float	Threshold on the cumulative feature importance, defaults to 0.9.	0.9
<code>sample_size</code> ¶	int	Sample size, defaults to 200,000.	200000
<code>max_features</code> ¶	int	Maximum number of features to return, defaults to Inf.	None
<code>job_properties</code> ¶	Mapping of str	Map of spark configurations for the job. If not provided defaults to the config of the current spark session.	None
<code>job_profile</code> ¶	JobProfile	Pulse UI custom job profile.	None

Notes

If both job_properties and job_profile are specified and there is intersection between their properties, the job_properties property will overwrite the job_profile property.

Returns:

Type	Description
set of str	The selected features.

Examples:

```
>>> selected_features = datasource.run_feature_selection(  
...     target_field="target"  
... )  
>>> job_profile = app.pullJobProfileByDesc("job_profile_name")  
>>> selected_features = datasource.run_feature_selection(  
...     target_field="target",  
...     threshold=0.7,  
...     sample_size=1_000_000,  
...     max_features=20,  
...     job_properties={  
...         "spark.master": "yarn",
```

```

...     "spark.app.name": "Feature Selection",
...     "spark.executor.memory": "1g",
...     "spark.dynamicAllocation.enabled": "True",
...     "spark.dynamicAllocation.maxExecutors": "4",
...     }
... )
>>> selected_features = datasource.run_feature_selection(
...     target_field="target",
...     threshold=0.7,
...     sample_size=1_000_000,
...     max_features=20,
...     job_profile=job_profile
... )

```

split [¶](#)

```

split(plan_name: str, splits: Set[Split], timestamp_field: str = 'event_occurred_at', scorable_filter: Optional[str] = None, wait: bool = False) -> Set[str]

```

Splits the datasource according to the splits passed.

Parameters:

Name	Type	Description	Default
<code>plan_name</code> ¶	str	Name that the resulting datasource split plan will have.	<i>required</i>
<code>splits</code> ¶	set of Split	A set of <code>Split</code> that will determine how the data source will be split. Must have at least one element.	<i>required</i>
<code>timestamp_field</code> ¶	str	The name of the timestamp field.	'event_occurred_at'
<code>scorable_filter</code> ¶	str	Optional string to filter scorable events. Fields must be enclosed with the pattern <code>\${field}</code> .	None
<code>wait</code> ¶	bool	Optional flag to indicate whether the method should wait for the datasource split to end before returning the designated method output.	False

Returns:

Type	Description
set of str	The names of the new datasources that the splitting process will create.

Examples:

```

>>> scorable = "(${amount} > 0)"
>>> splits = {
...     Split("train", 1350776834512, 1351173363000),
...     Split("test", 1351173363000, 1351473463000)
... }
>>> datasources = datasource.split(
...     plan_name="plan",
...     splits=splits,
...     timestamp_field="timestamp",

```

```
...     scorable_filter=scorable
... )
```

to_pyspark [↗](#)

```
to_pyspark(spark\_session: _TSparkSession) -> _TDataFrame
```

Convert datasource to pyspark.

Parameters:

Name	Type	Description	Default
spark_session ↗	<code>pyspark.sql.SparkSession</code> .	Spark session to use. Defaults to Pulse's session.	<i>required</i>

Returns:

Type	Description
<code>DataFrame</code>	The converted pyspark dataframe.

datasource_config

DataSourceConfig

```
DataSourceConfig(pyjnius_java)
```

Bases: [JavaWrapper](#)

A Data Source Configuration representation.

subclasses **class-attribute** **instance-attribute** 

```
subclasses: List[JavaWrapper] = []
```

builder **classmethod** 

```
builder()
```

fromPulseDataSource **classmethod** 

```
fromPulseDataSource(datasource)
```

datasource_config_with_inference

DataSourceConfigWithInference

```
DataSourceConfigWithInference(pyjnius_java)
```

Bases: [JavaWrapper](#)

A configuration builder used for DataSource imports.

subclasses **class-attribute** **instance-attribute** 

```
subclasses: List[JavaWrapper] = []
```

builder **classmethod** 

```
builder()
```

datasources

deprecate

deprecated_argument

```
deprecated_argument(removal_version: str, kw_name: str, replacement: Optional[str] = None) -> Callable[[_TFunc], _TFunc]
```

Decorator factory that marks an argument of a function as deprecated.

Returns a decorator that, when applied to a function, checks at call time whether the deprecated keyword argument (`kw_name`) was supplied. If so, a `DeprecationWarning` is issued indicating the argument's pending removal and, optionally, pointing to a replacement.

Parameters:

Name	Type	Description	Default
<code>removal_version</code>	<code>str</code>	The version of the library in which the argument is marked as deprecated (e.g., "2.0").	<i>required</i>
<code>kw_name</code>	<code>str</code>	The name of the argument to be deprecated.	<i>required</i>
<code>replacement</code>	<code>str</code> or <code>None</code>	Optional name of the replacement argument to suggest to the user. If <code>None</code> , it means the argument is removed entirely.	<code>None</code>

Returns:

Type	Description
<code>Callable</code>	A decorator that wraps a function and warns when the deprecated keyword argument is used.

evaluation

Useful utilities for model evaluations.

The Java ds-api counterpart is `com.feedzai.commons.dsapi.Evaluation` .

ABSOLUTE_IMPORTANCE module-attribute

```
ABSOLUTE_IMPORTANCE = 'Absolute Importance'
```

FEATURE module-attribute

```
FEATURE = 'Feature'
```

IMPORTANCE module-attribute

```
IMPORTANCE = ['#Recall', '[$Recall]']
```

RELATIVE_IMPORTANCE module-attribute

```
RELATIVE_IMPORTANCE = 'Relative Importance'
```

ConfusionMatrix

```
ConfusionMatrix(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Confusion Matrix returned by Evaluation objects.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

accuracy

```
accuracy() -> float
```

Accuracy metric.

Returns:

Type	Description
<code>float</code>	Accuracy value.

alert_rate

```
alert_rate() -> float
```

Alert Rate metric (a.k.a., positive predictive rate, or positive rate).

Returns:

Type	Description
float	Alert Rate value if denominator is > 0 else 0.

call_metric [¶](#)

```
call_metric(metric: Metric) -> float
```

Hides the metric dictionary from the caller to preserve its mutability when calling a metric.

Parameters:

Name	Type	Description	Default
metric ¶	Metric	Desired metric to use.	<i>required</i>

Returns:

Type	Description
float	Metric value.

false_discovery_rate [¶](#)

```
false_discovery_rate() -> float
```

False Discovery Rate metric.

Returns:

Type	Description
float	False Discovery Rate value if denominator is > 0 else 0.

false_negative_rate [¶](#)

```
false_negative_rate() -> float
```

False Negative Rate metric.

Returns:

Type	Description
float	False Negative Rate value if denominator is > 0 else 0.

negative_predictive_value [¶](#)

```
negative_predictive_value() -> float
```

Negative Predictive Value metric.

Returns:

Type	Description
float	Negative Predictive Value value if denominator is > 0 else 0.

positive_predictive_value [¶](#)

```
positive_predictive_value() -> float
```

Positive Predictive Value metric.

Returns:

Type	Description
float	Positive Predictive Value value if denominator is > 0 else 0.

specificity [¶](#)

```
specificity() -> float
```

Specificity metric.

Returns:

Type	Description
float	Specificity value if denominator is > 0 else 0.

Evaluation [¶](#)

```
Evaluation(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents an Evaluation in Pulse.

subclasses **class-attribute** **instance-attribute** [¶](#)

```
subclasses: List[JavaWrapper] = []
```

get_feature_importance_results [¶](#)

```
get_feature_importance_results(target_value: str | bool | int) -> DataFrame
```

Extract the results of the absolute and relative feature importance evaluated in terms of #Recall and \$Recall. Feature importance based on \$Recall is only available if a cost field has been defined in the model evaluation config.

Parameters:

Name	Type	Description	Default
target_value ¶	(str, bool, int or float)	The target value for which the feature importance results will be extracted.	<i>required</i>

Returns:

Type	Description
DataFrame	Pandas df with feature importance results, ordered by descending values of the #Recall-based feature importance.

Examples:

```
>>> model_eval.get_feature_importance_results('true')
```

get_result [↗](#)

```
get_result(target_value: bool | int | str = True, cost_field: str | None = None) -> list
```

Return the evaluation result.

Parameters:

Name	Type	Description	Default
target_value ↗	(bool, int, str)	Value for the target positive class.	True
cost_field ↗	str	Name (desc) of the cost field. By using this parameter, the result will be a confusion matrix with cost-related metrics.	None

Returns:

Type	Description
list	Sequence of confusion matrices, optionally with cost metrics.

get_score_field [↗](#)

```
get_score_field(target_value: bool | int | str = True) -> str
```

Get the score field for a given target value.

Raises:

Type	Description
ValueError	When the target value is not found in the list of possible fields.

get_score_field_names [↗](#)

```
get_score_field_names() -> dict[str, str]
```

Return the possible name of scored fields in this evaluation.

Examples:

```
>>> evaluation.get_score_field_names()
```

Returns:

Name	Type	Description
fields	tuple of str	The names of the fields associated with scores in this evaluation.

Name	Type	Description
values	tuple of str	The possible values from the target label (sanitized).

run_feature_importance [¶](#)

```
run_feature_importance(desired_fpr: float = 0.01, sample_rate: float = 0.3, random_seed: int | None = None, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, wait: bool = False) -> None
```

Run feature importance on the model evaluation object.

The values of the features are shuffled, and the loss in (transaction or cost) Recall with respect to the true Recall is computed, for each feature individually.

Parameters:

Name	Type	Description	Default
desired_fpr ¶	float	The desired FPR for feature importance. Defaults to 0.01.	0.01
sample_rate ¶	float	Sampling rate to be applied prior to feature importance. Defaults to 0.3.	0.3
random_seed ¶	int	Random seed, for reproducibility purposes. Defaults to a random integer between 1 and 1000000.	None
job_profile ¶	JobProfile	Pulse UI custom job profile.	None
job_properties ¶	Mapping of str	Map of spark configurations for the job. Defaults to Pulse's Properties Profile.	None
wait ¶	bool	Waits or not for feature importance to end. Defaults to False.	False

Examples:

```
>>> model_eval.run_feature_importance()
>>> # or
>>> model_eval.run_feature_importance(
...     desired_fpr=0.01,
...     sample_rate=0.3,
...     random_seed=123456
... )
>>> # or
>>> model_eval.run_feature_importance(
...     desired_fpr=0.01,
...     sample_rate=0.3,
...     random_seed=123456,
...     job_properties={"spark.executor.memory": "5g"},
... )
```

Metric [¶](#)

Bases: Enum

Supported metrics in model evaluations.

ACCURACY class-attribute instance-attribute [↗](#)

```
ACCURACY = auto()
```

ALERT_RATE class-attribute instance-attribute [↗](#)

```
ALERT_RATE = auto()
```

FALSE_DISCOVERY_RATE class-attribute instance-attribute [↗](#)

```
FALSE_DISCOVERY_RATE = auto()
```

FALSE_NEGATIVES class-attribute instance-attribute [↗](#)

```
FALSE_NEGATIVES = auto()
```

FALSE_NEGATIVE_RATE class-attribute instance-attribute [↗](#)

```
FALSE_NEGATIVE_RATE = auto()
```

FALSE_POSITIVES class-attribute instance-attribute [↗](#)

```
FALSE_POSITIVES = auto()
```

FALSE_POSITIVE_RATE class-attribute instance-attribute [↗](#)

```
FALSE_POSITIVE_RATE = auto()
```

F_MEASURE class-attribute instance-attribute [↗](#)

```
F_MEASURE = auto()
```

INFORMEDNESS class-attribute instance-attribute [↗](#)

```
INFORMEDNESS = auto()
```

MARKEDNESS class-attribute instance-attribute [↗](#)

```
MARKEDNESS = auto()
```

NEGATIVE_PREDICTIVE_VALUE class-attribute instance-attribute [↗](#)

```
NEGATIVE_PREDICTIVE_VALUE = auto()
```

POSITIVE_PREDICTIVE_VALUE class-attribute instance-attribute [↗](#)

```
POSITIVE_PREDICTIVE_VALUE = auto()
```

PRECISION class-attribute instance-attribute [↗](#)

```
PRECISION = auto()
```

PREDICTED_NEGATIVES class-attribute instance-attribute [↗](#)

```
PREDICTED_NEGATIVES = auto()
```

PREDICTED_POSITIVES class-attribute instance-attribute [↗](#)

```
PREDICTED_POSITIVES = auto()
```

RECALL class-attribute instance-attribute [↗](#)

```
RECALL = auto()
```

SPECIFICITY class-attribute instance-attribute [↗](#)

```
SPECIFICITY = auto()
```

TRUE_NEGATIVES class-attribute instance-attribute [↗](#)

```
TRUE_NEGATIVES = auto()
```

TRUE_POSITIVES class-attribute instance-attribute [↗](#)

```
TRUE_POSITIVES = auto()
```


exceptions

Helper exception handling functions.

logger module-attribute

```
logger = getLogger(__name__)
```

JavaAttributeNotFound

```
JavaAttributeNotFound(attribute: str, java_attributes: Sequence[str])
```

Bases: `AttributeError`

Exception to throw when a user requests method/field not present in the java object.

PulseIndexerExistsError

Bases: `RuntimeError`

Raised when creating an Indexer resource that already exists in Pulse.

unwrap_java_exception

```
unwrap_java_exception(func)
```

Executes the function passed by argument and if that function raises a `JavaException`, unwraps that exception, logs its name, message and stacktrace and then propagates the thrown exception

Parameters:

Name	Type	Description	Default
<code>func</code> 		A lambda with no arguments.	<i>required</i>

Returns:

Type	Description
<code>Any</code>	The result returned by func.

exploration

Utilities for efficient exploratory data analysis via PySpark. If you want to perform EDA in an efficient way, then you've come to the right place.

This package provides tools to:

- efficiently profile your data in a distributed way,
- easily generate a detailed report of your data,
- quickly explore the results of the profiled data via convenient visualizations and access to key statistics and validations,
- all this via a flexible and user-friendly interface, and providing a convenient plotting library.

tooltips

```
tooltips()
```

expression_template [↗](#)

Module to handle the parsing and conversion of a Pulse ExpressionTemplate containing Pulse schema fields and a string expression.

ExpressionTemplateJava module-attribute [↗](#)

```
ExpressionTemplateJava = autoclass(MODEL_CLASSPATH + 'datascience.ExpressionTemplate')
```

ExpressionTemplate [↗](#)

```
ExpressionTemplate(fields: List, expression: str, raw_expression: str)
```

Util class to handle conversion from expression templates to indexed strings and vice versa.

FIELD_PATTERN class-attribute instance-attribute [↗](#)

```
FIELD_PATTERN = '\\$\\{(.+?)\\}'
```

INDEX_PATTERN class-attribute instance-attribute [↗](#)

```
INDEX_PATTERN = '\\$\\{((\\d+)\\}'
```

expression instance-attribute [↗](#)

```
expression = expression
```

fields instance-attribute [↗](#)

```
fields = fields
```

raw_expression instance-attribute [↗](#)

```
raw_expression = raw_expression
```

from_java_template classmethod [↗](#)

```
from_java_template(exp_template: JavaClass) -> ExpressionTemplate
```

Converts a Pulse API ExpressionTemplate to a string with indexed fields.

If a field is used multiple times the field is duplicated and the index incremented.

Parameters:

Name	Type	Description	Default
<code>exp_template</code> ↗	<code>ExpressionTemplate</code>	Pulse API ExpressionTemplate	<i>required</i>

Returns:

Type	Description
<code>ExpressionTemplate</code>	ExpressionTemplate

`from_string` classmethod [¶](#)

```
from_string(indexed_str: str)
```

Creates a new ExpressionTemplate from a string with indexed fields.

Fields must be enclosed with the pattern `${field}`.

Parameters:

Name	Type	Description	Default
<code>indexed_str</code> ¶	<code>str</code>	String with indexed fields eg. <code>"\${TransactionAmount} > 10"</code>	<i>required</i>

Returns:

Type	Description
<code>ExpressionTemplate</code>	ExpressionTemplate

`rename_fields` [¶](#)

```
rename_fields(old_name: str, new_name: str)
```

Renames all the fields matching the `old_name`.

Parameters:

Name	Type	Description	Default
<code>old_name</code> ¶	<code>str</code>	Old name to change from.	<i>required</i>
<code>new_name</code> ¶	<code>str</code>	New name to change to.	<i>required</i>

`to_java` [¶](#)

```
to_java() -> JavaClass
```

Converts to DS-API object.

Returns:

Type	Description
<code>JavaClass</code>	ExpressionTemplate Java class

feedzai

logger module-attribute

```
logger = getLogger(__name__)
```

Feedzai

```
Feedzai(pulse: Server | None = None)
```

Bases: object

Entry point to ds_api functionalities.

Parameters:

Name	Type	Description	Default
pulse	Server	Server used to communicate with Pulse. If not specified default ds_api.Server will be used.	None
_remote_session		Most recent SparkSession created (used as local cache of the SparkSession object)	required
_remote_session_params		Parameters of the most recent SparkSession created	required

pulse instance-attribute

```
pulse = connect()
```

app

```
app(**kwargs)
```

Wrapper around ds_api.App constructor to use Pulse Server configured in this instance.

Other Parameters:

Name	Type	Description
address	str	The hostname.
port	int	The RMI port.
timeout	int	Maximum time to wait until timeout.
user_id	str	The user id used during RMI connections.

Returns:

Type	Description
<code>App</code>	New App instance.

`get_spark_local_session` staticmethod [¶](#)

```
get_spark_local_session(spark_params: Optional[Mapping[str, str]] = None, spark_app_name: Optional[str] = None, log_level: SparkLogLevel = ERROR)
```

Get SparkSession configured in local mode. It will use all available CPUs and each executor has 1G RAM and 1 vcore.

Note that Spark maintains a single activate session, thus unless it was crashed, you would always get the same one, but configured in local mode. Even if you stop SparkSession or SparkContext, in pySpark, it will remember previous configurations.

This means the following:

1. Previously spark had properties A, B, and C defined
2. You call this function and you will get a SparkSession with A, B, and C defined as well as the local mode

Parameters:

Name	Type	Description	Default
<code>spark_params</code> ¶	<code>dict</code>	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be found at https://spark.apache.org/docs/2.3.0/configuration.html .	<code>None</code>
<code>spark_app_name</code> ¶	<code>str</code>	The name for the spark application. If 'None' then one is automatically assigned.	<code>None</code>
<code>log_level</code> ¶	<code>SparkLogLevel</code>	Log Level of the spark session, defaults to <code>SparkLogLevel.ERROR</code> .	<code>ERROR</code>

Returns:

Type	Description
<code>SparkSession</code>	A Spark Session in local mode, None if spark_params is not a dictionary.

`get_spark_session` [¶](#)

```
get_spark_session(spark_params: Dict[str, str] | None = None, exclude_spark_params: List[str] | None = None, app_name: str | None = None, spark_app_name: str | None = None, job_profile_name: str | None = None, group_name: str | None = None, clean_spark_properties: bool = True, log_level: SparkLogLevel = ERROR, session_type: SparkSessionType = CLASSIC, spark_remote: str | None = None)
```

Get SparkSession with default Pulse's configuration.

Note that Spark maintains a single activate session, thus unless it was crashed, you would always get the same one, but with last Pulse's configuration (and possible old configurations). Even if you stop SparkSession or SparkContext, in pySpark, it will remember previous configurations.

This means the following:

1. Pulse has properties A, B, and C defined

2. You call this function and you will get a SparkSession with A, B, and C properties
3. You remove property A and add property D
4. You call this function and you will get a SparkSession with **A**, B, C, and D

Parameters:

Name	Type	Description	Default
<code>spark_params</code> ↗	dict	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be find in https://spark.apache.org/docs/latest/configuration.html	None
<code>exclude_spark_params</code> ↗	list	List with Spark properties to be removed. Full list of Spark properties can be find in https://spark.apache.org/docs/latest/configuration.html	None
<code>app_name</code> ↗	str	An optional name of a pulse application. This parameter should be used when we want to have access to application specific properties. This allows to use the same properties being used by Spark jobs executed in Pulse.	None
<code>spark_app_name</code> ↗	str	The name for the spark application. If 'None' then one is automatically assigned.	None
<code>job_profile_name</code> ↗	str	The name of the job profile to reuse the Spark properties defined there. If 'None' then it will retrieve the properties associated to default job profile for jobs of type 'EXTERNAL_DS_JOB'.	None
<code>group_name</code> ↗	str	The name of the group model used to retrieve the configurations defined in its properties.	None
<code>clean_spark_properties</code> ↗	bool	True if it should clean the Spark properties specific to Pulse use case, false otherwise.	True
<code>log_level</code> ↗	SparkLogLevel	Log Level of the spark session, defaults to SparkLogLevel.ERROR.	ERROR
<code>session_type</code> ↗	SparkSessionType	The type of Spark Session to be created, defaults to <code>SparkSessionType.CLASSIC</code>	CLASSIC
<code>spark_remote</code> ↗	str	The URL of the remote Spark Connect server	None

Returns:

Type	Description
<code>_TSparkSession</code>	A SparkSession with Pulse's configuration, None if spark_params is not a dictionary

pandas_to_pulse staticmethod [↗](#)

```
pandas_to_pulse(dataframe: DataFrame, app: App, name: str, path: str, file_format: str = 'csv', schema: Optional[Schema] = None, **kwargs: Union[bool, str, int, Dict[str, Any], Set[str]])
```

Converts a given pandas DataFrame to Pulse datasource using SparkSession#createDataFrame method of pySpark dataframe.

Parameters:

Name	Type	Description	Default
<code>dataframe</code> ↗	DataFrame	DataFrame which will be used to create a Pulse datasource.	<i>required</i>
<code>app</code> ↗	App	The app in which the datasource will be created.	<i>required</i>
<code>name</code> ↗	str	The name of the new datasource.	<i>required</i>
<code>path</code> ↗	str	The path of the new datasource.	<i>required</i>
<code>file_format</code> ↗	str	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	'csv'
<code>schema</code> ↗	Schema	The pulse schema used to create the datasource. If not specified, the schema will be inferred.	None

Other Parameters:

Name	Type	Description
<code>has_header</code>	bool	Flags whether the dataset is to be read with header. Defaults to True.
<code>sep</code>	str	Delimiter to use. Should contain exactly one character. Defaults to ','.
<code>missing_values</code>	set of str	Set of strings with values for empty value. Defaults to { "", "NA", "NULL" }.
<code>quote</code>	str	The character used to denote the start and end of a quoted item. Defaults to "".
<code>line_separator</code>	str	The string used mark the end of line. Defaults to "\n".
<code>encoding</code>	str	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'.
<code>cat_threshold</code>	int	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
<code>job_properties</code>	Dict	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse Properties Profile.
<code>n_records</code>	int	

Name	Type	Description
		The number of records that will be read per partition when performing datasource schema inference. Defaults to 15000. Will only be used if no Schema is specified.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Default to None.
<code>merge_schemas</code>	<code>bool</code>	Specify if schema merging should be allowed. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
<code>DataSource</code>	The new <code>ds_api.DataSource</code> instance.

`pulse_to_pandas` staticmethod [¶](#)

```
pulse_to_pandas(app: App, name: str, spark_properties: dict | None = None)
```

Converts a given datasource (by app and name) to a pandas DataFrame using toPandas method of pySpark dataframe.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Application name, where the datasource is persisted.	<i>required</i>
<code>name</code> ¶	<code>str</code>	The DataSource name.	<i>required</i>
<code>spark_properties</code> ¶	<code>Dict</code>	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be found in https://spark.apache.org/docs/2.3.0/configuration.html .	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	A pandas DataFrame correspondent to given pulse-based DataSource.

`pulse_to_pyspark` staticmethod [¶](#)

```
pulse_to_pyspark(spark_session, app, name)
```

Uses `ds_api.SparkDataSourceAdapter` to convert a given data source (by app and name), to a pyspark DataFrame.

Parameters:

Name	Type	Description	Default
<code>spark_session</code> ¶	<code>pyspark.sql.SparkSession</code>		<i>required</i>

Name	Type	Description	Default
<code>app</code> ⌕	<code>App</code>	Application name, where the Data Source is persisted.	<i>required</i>
<code>name</code> ⌕	<code>str</code>	The Data Source name as it appears in the UI.	<i>required</i>

Returns:

Type	Description
<code>DataFrame</code>	A pyspark DataFrame correspondent to given pulse-based Data Source.

`pyspark_to_pulse` [staticmethod](#) [⌕](#)

```
pyspark_to_pulse(spark_session: _TSparkSession, dataframe: _TDataFrame, app: App, name: str,
path: str, file_format: str = 'csv', schema: Optional[Schema] = None, **kwargs: Union[bool, str,
int, Dict[str, Any], Set[str]])
```

Uses `ds_api.SparkDataSourceAdapter` to convert a given pySpark dataframe to a Pulse datasource.

The dataframe will be written in the path specified before being added to Pulse. `DateType` and `TimestampType` fields are written as unix time (in seconds). The schema will be automatically inferred using `AbstractDataSource#inferSchema` approach if the Schema is not specified.

All pySpark types but `DecimalType` and `Mllib` types are supported.

Parameters:

Name	Type	Description	Default
<code>spark_session</code> ⌕	<code>pyspark.sql.SparkSession</code>	The Spark session.	<i>required</i>
<code>dataframe</code> ⌕	<code>DataFrame</code>	DataFrame which will be used to create a Pulse datasource.	<i>required</i>
<code>app</code> ⌕	<code>App</code>	The app in which the datasource will be created.	<i>required</i>
<code>name</code> ⌕	<code>str</code>	The name of the new datasource.	<i>required</i>
<code>path</code> ⌕	<code>str</code>	The path of the new datasource.	<i>required</i>
<code>file_format</code> ⌕	<code>str</code>	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	<code>'csv'</code>
<code>schema</code> ⌕	<code>Schema</code>	The pulse schema used to create the datasource. If not specified, the schema will be inferred.	<code>None</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Defaults to ','.
<code>missing_values</code>	<code>set of str</code>	Set of strings with values for empty value. Defaults to { "", "NA", "NULL" }.
<code>quote</code>	<code>str</code>	The character used to denote the start and end of a quoted item. Defaults to "".
<code>line_separator</code>	<code>str</code>	The string used mark the end of line. Defaults to "\n".
<code>encoding</code>	<code>str</code>	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'.
<code>cat_threshold</code>	<code>int</code>	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
<code>job_properties</code>	<code>Dict</code>	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse Properties Profile. Deprecated in version 25.0.41 and will be removed.
<code>n_records</code>	<code>int</code>	The number of records that will be read per partition when performing datasource schema inference. Defaults to 15000. Will only be used if no Schema is specified.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Default to None.
<code>merge_schemas</code>	<code>bool</code>	Specify if schema merging should be allowed. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
<code>DataSource</code>	New DataSource instance.

SparkLogLevel

Bases: `Enum`

ALL `class-attribute` `instance-attribute` 

ALL = 'ALL'

DEBUG `class-attribute` `instance-attribute` 

DEBUG = 'DEBUG'

ERROR `class-attribute` `instance-attribute` 

ERROR = 'ERROR'

FATAL class-attribute instance-attribute [Â¶](#)

```
FATAL = 'FATAL'
```

INFO class-attribute instance-attribute [Â¶](#)

```
INFO = 'INFO'
```

OFF class-attribute instance-attribute [Â¶](#)

```
OFF = 'OFF'
```

TRACE class-attribute instance-attribute [Â¶](#)

```
TRACE = 'TRACE'
```

WARN class-attribute instance-attribute [Â¶](#)

```
WARN = 'WARN'
```

SparkSessionType [Â¶](#)

Bases: Enum

CLASSIC class-attribute instance-attribute [Â¶](#)

```
CLASSIC = 'CLASSIC'
```

CONNECT class-attribute instance-attribute [Â¶](#)

```
CONNECT = 'CONNECT'
```

field

logger module-attribute

```
logger = getLogger(__name__)
```

FieldTemplate dataclass

```
FieldTemplate(name: str, native_type: NativeType, datascience_field: DataScienceField)
```

Defines a field template used to add fields to a schema.

Attributes:

Name	Type	Description
<code>self.name</code>	<code>str</code>	Name of the field.
<code>self.native_type</code>	NativeType	The native type of the field.
<code>self.datascience_field</code>	DataScienceField	The DataScienceField class corresponding to this field.

datascience_field instance-attribute

```
datascience_field: DataScienceField
```

name instance-attribute

```
name: str
```

native_type instance-attribute

```
native_type: NativeType
```

group

get_group

```
get_group(pulse: Server, group_name: Optional[str] = None)
```

Pulls Pulse job profiles that match a given name.

Parameters:

Name	Type	Description	Default
<code>pulse</code>	<code>Server</code>	The ds_api.Server used to communicate with Pulse.	<i>required</i>
<code>group_name</code>	<code>str</code>	Desc name of the Pulse group model, by default None	<code>None</code>

Returns:

Type	Description
<code>(JavaWrapper, optional)</code>	A wrapper of 'com.feedzai.pulse.api.security.model.GroupModel', or None if it was not able to find a group model with a given name.

hash_based_partitioning_config

HashBasedPartitioningConfig

```
HashBasedPartitioningConfig()
```

Bases: [PartitioningConfig](#)

Represents a Hash-Based Partitioning configuration.

subclasses **class-attribute** **instance-attribute** 

```
subclasses: List[JavaWrapper] = []
```

hyperparameter

Implementation of the hyperparameter tuning using Optuna framework.

MODEL_PROJECT_TEMPLATE module-attribute

```
MODEL_PROJECT_TEMPLATE = 'ds_api_hyperparameter'
```

STUDY_NAME_TEMPLATE module-attribute

```
STUDY_NAME_TEMPLATE = 'ds_api_hyperparameter_study'
```

TRAINING_CONFIG_SCHEMA module-attribute

```
TRAINING_CONFIG_SCHEMA = Schema({'model_type': And(str, lambda value: any(model_type for model_t
ype in (__subclasses__()) if value in __name__)), 'model_parameters': And({Optional(str): Or(Or(s
tr, Number), And(list, lambda x: len(x) > 1), And({'type': Or('int', 'float', 'uniform', 'discre
te_uniform', 'loguniform'), 'range': And(list, lambda values: len(values) == 2, lambda values: r
equire_all_of_type(values, Number)), Optional('log', default=False): Or(True, False), Optional('
step'): int, Optional('q'): float}})}, lambda x: len(x) > 1))
```

logger module-attribute

```
logger = getLogger(__name__)
```

DatabaseConnection dataclass

```
DatabaseConnection(database: str, user: str, password: str, host: str = 'localhost', port: int =
5432)
```

Validates a database connection to safely return a valid URI.

Currently only works with PostgreSQL DB.

database instance-attribute

```
database: str
```

host class-attribute instance-attribute

```
host: str = 'localhost'
```

password instance-attribute

```
password: str
```

port class-attribute instance-attribute

```
port: int = 5432
```

user instance-attribute

```
user: str
```


get_connection_uri [¶](#)

```
get_connection_uri() -> str
```

Builds the DB connection URI from a set of configs.

HyperparameterStudy [¶](#)

```
HyperparameterStudy(study_name: str, train_datasource: DataSource, validation_datasource: DataSource, target_field: str, target_value: str | int | bool = 'true', cost_field: str | None = None, model_project: ModelProject | None = None, optimization_metric: Metric = RECALL, target_metric: Metric = FALSE_POSITIVE_RATE, target_metric_value: float = 0.01, direction: StudyDirection = MAXIMIZE, database: DatabaseConnection | None = None, sampler: BaseSampler | None = None, pruner: BasePruner | None = None, load_if_exists: bool = True)
```

Bases: `Study`

A study corresponds to an optimization task, i.e., a set of trials.

Parameters:

Name	Type	Description	Default
<code>study_name</code> ¶	<code>str</code>	Name given to the study.	<i>required</i>
<code>train_datasource</code> ¶	DataSource	Datasource used for training models.	<i>required</i>
<code>validation_datasource</code> ¶	DataSource	Datasource used for tuning parameters.	<i>required</i>
<code>target_field</code> ¶	<code>str</code>	The name of the target field in the datasource.	<i>required</i>
<code>target_value</code> ¶	<code>str</code> or <code>int</code> or <code>bool</code>	The target value in the datasource, defaults to "true".	<code>'true'</code>
<code>cost_field</code> ¶	<code>str</code>	Cost field of the datasource. Defining it will make the optimization cost based (optimizing for cost recall vs recall, for example).	<code>None</code>
<code>model_project</code> ¶	ModelProject	The project to run the train and evaluation on. If not provided, one is created.	<code>None</code>
<code>optimization_metric</code> ¶	Metric	Metric used to maximize/minimize. Default is recall.	RECALL
<code>target_metric</code> ¶	Metric	Metric to pin while optuna maximizes/minimizes the optimization metric. Default is False Positive Rate.	FALSE_POSITIVE_RATE
	<code>float</code>	Value to aim for in the target metric. Default is 0.01.	<code>0.01</code>

Name	Type	Description	Default
<code>target_metric_value</code> ¶			
<code>direction</code> ¶	<code>StudyDirection</code>	Direction of optimization. Default is <code>StudyDirection.MAXIMIZE</code>	<code>MAXIMIZE</code>
<code>database</code> ¶	<code>DatabaseConnection</code>	Connection object to store the studies data in. If not provided, in-memory storage is used, and the Study will not be persistent.	<code>None</code>
<code>sampler</code> ¶	<code>BaseSampler</code>	A sampler object that implements background algorithm for value suggestion.	<code>None</code>
<code>pruner</code> ¶	<code>BasePruner</code>	A pruner object that decides early stopping of unpromising trials.	<code>None</code>
<code>load_if_exists</code> ¶	<code>bool</code>	Flag to control the behavior to handle a conflict of study names. Defaults to <code>True</code> , loading a previous study.	<code>True</code>

Examples:

```
>>> app : App
>>> train_datasource = app.pullByName("train")
>>> validation_datasource = app.pullByName("validation")
>>> model_project = ModelProject.create(app, "model_project")
>>> database_connection = DatabaseConnection(
...     database="test",
...     user="my_user",
...     password="my_password"
... )
>>>
>>> study = HyperparameterStudy(
...     study_name="test_study",
...     train_datasource=train_datasource,
...     validation_datasource=validation_datasource,
...     target_field="label",
...     target_value="true",
...     cost_field="amount",
...     model_project=model_project,
...     optimization_metric=Metric.RECALL,
...     target_metric=Metric.FALSE_POSITIVE_RATE,
...     target_metric_value=0.01,
...     database=database_connection,
... )
```

`run` [¶](#)

```
run(config: Path, ignored_fields: set | None = None, n_trials: int = 100, timeout: float | None = None, n_jobs: int = 1, catch: tuple[Type[Exception], ...] = (), show_progress_bar: bool = False, keep_trained_models: bool = False) -> None
```

Optimizes the study. Use this method instead of `Study#optimize`.

Parameters:

Name	Type	Description	Default
	<code>Path</code>		<i>required</i>

Name	Type	Description	Default
<code>config</code> ↗		The model configuration file.	
<code>ignored_fields</code> ↗	Set	Set of fields to be ignored in the model training.	None
<code>n_trials</code> ↗	int	The number of trials. If not provided, it runs 100 trials. If timeout is provided the study will finish when one of the conditions is met. It can also be stopped if it receives a termination signal such as Ctrl+C or SIGTERM.	100
<code>timeout</code> ↗	float	Stops study after the given number of seconds.	None
<code>n_jobs</code> ↗	int	The number of parallel jobs. If this argument is set to -1, the number is set to CPU count.	1
<code>catch</code> ↗	tuple of type Exception	A study continues to run even when a trial raises one of the exceptions specified in this argument. The default is to stop at any exception raised.	()
<code>show_progress_bar</code> ↗	bool	Flag to show progress bars or not. Currently, progress bar is experimental feature and disabled when <code>n_jobs > 1</code> .	False
<code>keep_trained_models</code> ↗	bool	Flag to control if trained models should be kept in Pulse or removed. By default, they are deleted after the evaluation and metrics calculation.	False

Raises:

Type	Description
<code>RuntimeError:</code>	If nested invocation of this method occurs.
Example of yaml Random Forest Config to tune only the number of trees:	
<code>```yaml</code>	
<code>model_type: RandomForestConfig</code>	
<code>model_parameters:</code>	<code>num_trees: type: int range: [50, 500] log: False</code>
<code>```</code>	

Examples:

```
>>> study : HyperparameterStudy
>>> study.run(
...     config=Path("random_forest_config.yaml"),
...     ignored_fields={"card", "mcc"},
...     n_trials=100,
...     n_jobs=4,
... )
```

java_enum_wrapper

JavaEnumWrapper

Bases: `str`, `Enum`

Class to represent Java Enums as native Python Enums.

This enum is a proxy for Java Enums. It's preferred to using the pyjnius wrapped object so that we keep the advantages of having the Enum, such as identifying invalid types at compile time, thereby catching bugs early on. Additionally, this decouples the user's code from our Java or Pulse's implementation.

When using this enum, the use of the `auto` function from the enum package is recommended, which will set the value of an attribute be the same as the name. The values of attributes are given to the `JavaEnumWrapper.__new__` method upon attribute creation, which override the attribute's value with Java Enum attribute corresponding to the given value. Therefore, the use of `auto` is recommended, which will assign the value to be the same as the name of the python enum attribute. However, if you want to give any of the attributes a name different from the java counterpart, you can go without using `auto` and assigning the value from the java counterpart to the new name in python.

java_wrapper [↗](#)

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

JavaWrapper [↗](#)

```
JavaWrapper(pyjnius_java)
```

A wrapper for jnius generated objects.

The jnius framework already does a substantial work when converting java objects into python object. Unfortunately, this work is more focused on java native types. This class complements this functionally by providing additional transformation functions that can automatically convert objects from both the Java native library as well as some types from the Feedzai common library. As such, this class should always be used to wrap pyjnius generated objects.

An example of the usage for this class would be: Given a java class Car, which contains a Car#getWheels which returns a java.util.List. A JavaWrapper for an instance of this class would return a python array of Wheel objects (the conversion from java.util.List to a python array would occur automatically). Note that pyjnius only converts java.util.List objects into a pyjnius generated class for java.util.List, thus making the code that interacts with it more verbose.

Note also that the conversions are transitive. A java method that returns a List< is automatically converted into a python array of arrays of strings.

Another useful feature of this class is that it is able to log details for the JVM exceptions thrown from the pyjnius JVM. The details include the java stacktrace, which should be useful to debug java code that pyjnius interacts with.

Class variables "java_classref" and "java_class" are to be defined by every subclass, which refer to the java class that the python class can represent. Imagining for a DataSource, it basically says, if you give me a "com.feedzai.common.dsapi.DataSource" I can convert it to python. They will be automatically added to a map available to every class, so the return types coming from java can accurately/automatically be converted to their Python friends. In this case, if no good match is found, it defaults to this JavaWrapper class.

Finally, note that if you intend to change the init's signature of your new subclass, be a pal and define a static method named "auto_cast" that must have one parameter: the java object that represents our python counterpart. That way, our auto-cast system will cleverly pick up that method and work smoothly. This avoids extra ".unwrap()" and ".get()" that make the use of the api by the end-user less intuitive.

There are examples of this auto-cast method in the models package and types module.

Parameters:

Name	Type	Description	Default
<code>pyjnius_java</code> ↗		The original pyjnius object.	<i>required</i>

subclasses class-attribute instance-attribute [↗](#)

```
subclasses: List[JavaWrapper] = []
```

job_profile

Java ds-api counterpart in Java: `com.feedzai.commons.dsapi.JobProfile` .

logger

```
logger = getLogger(__name__)
```

JobProfile

```
JobProfile(pyjnius_java)
```

Bases: `JavaWrapper`

Represents a JobProfile in Pulse.

subclasses

```
subclasses: List[JavaWrapper] = []
```

create

```
create(app: App, desc: str, job_properties: Mapping[str, str]) -> JobProfile
```

Create a new JobProfile in `app` with the user facing name `desc` with and the specified spark `job_properties` , complementary to the default properties.

Parameters:

Name	Type	Description	Default
<code>app</code>	<code>App</code>	The App where the job profile will be created.	<i>required</i>
<code>desc</code>	<code>str</code>	The name of the job profile user faces in UI.	<i>required</i>
<code>job_properties</code>	<code>Dict</code>	Dictionary with Spark job configurations.	<i>required</i>

Returns:

Type	Description
<code>JobProfile</code>	JobProfile object corresponding to the Pulse job profile created.

get_job_config_dict

```
get_job_config_dict(job_properties: Optional[Mapping[str, str]] = None) -> Dict
```

Merges this job profile properties with job_properties.

Note

if there is intersection between the properties, the job_properties property will overwrite this job profile property.

Parameters:

Name	Type	Description	Default
<code>job_properties</code> A1	<code>dict</code>	Additional spark properties, eg. {"spark.executor.memory": "1g"}, by default dict()	<code>None</code>

Returns:

Type	Description
<code>dict</code>	Merged Pulse job config and spark properties. eg. {"spark.executor.memory": "1g"}

lists

DS-API module to manage lists and list items in Pulse.

A List in Pulse is either Simple ([SimpleManagedListModel](#)) or Complex ([ComplexManagedListModel](#)). A Pulse List can contain items of the respective type: [SimpleManagedListItemModel](#) or [ComplexManagedListItemModel](#) .

Additionally, lists can have tags ([ManagedListTagModel](#)).

This module also provides a wrapper for [ManagedList](#) , to simplify list operations like `delete()` , and to provide an API that is more consistent with the rest of the DS-API.

ComplexManagedListItemModel module-attribute

```
ComplexManagedListItemModel = autoclass(MANAGED_LISTS_CLASSPATH + 'ComplexManagedListItemModel')
```

ComplexManagedListModel module-attribute

```
ComplexManagedListModel = autoclass(MODEL_CLASSPATH + 'ComplexManagedListModel')
```

FieldType module-attribute

```
FieldType = autoclass('com.feedzai.pulse.esp.FieldType')
```

Include module-attribute

```
Include = autoclass(PULSE_API_CLASSPATH + 'apps.enums.Include')
```

ManagedListTagModel module-attribute

```
ManagedListTagModel = autoclass(MANAGED_LISTS_CLASSPATH + 'ManagedListTagModel')
```

SimpleManagedListItemModel module-attribute

```
SimpleManagedListItemModel = autoclass(MANAGED_LISTS_CLASSPATH + 'SimpleManagedListItemModel')
```

SimpleManagedListModel module-attribute

```
SimpleManagedListModel = autoclass(MODEL_CLASSPATH + 'SimpleManagedListModel')
```

ManagedList

```
ManagedList(pyjnius_java)
```

Bases: [JavaWrapper](#)

Wrapper for Pulse's Lists. Can handle all list types.

Although one can use the Pulse API directly by accessing the methods of the managed list wrapped by this class with `getManagedList()` , this Python wrapper pulls all those methods to the main class.

Any method implemented by `ManagedList` takes precedence over exposed `self.getManagedList()._java_attributes()` members, to allow `ManagedList` to implement custom behaviour that overrides the wrapped class's members.

subclasses `class-attribute` `instance-attribute` [¶](#)

```
subclasses: List[JavaWrapper] = []
```

create_tag [¶](#)

```
create_tag(app: App, tag_name: str)
```

Creates a new tag (`ManagedListTagModel`) to use in a Pulse List.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Pulse Application.	<i>required</i>
<code>tag_name</code> ¶	<code>str</code>	Name of the tag that will be used in the Pulse List.	<i>required</i>

Returns:

Type	Description
<code>ManagedListTagModel</code>	Pulse List Tag.

get_list_items_manager [¶](#)

```
get_list_items_manager(pulse: Server) -> JavaClass
```

Returns Pulse's list item manager.

Parameters:

Name	Type	Description	Default
<code>pulse</code> ¶	<code>Server</code>	The <code>ds_api.Server</code> used to communicate with Pulse.	<i>required</i>

Returns:

Type	Description
<code>AppManagedListItemManager</code>	List item manager to manage Pulse list items.

get_list_manager [¶](#)

```
get_list_manager(pulse: Server) -> JavaClass
```

Returns Pulse's list model manager.

Parameters:

Name	Type	Description	Default
<code>pulse</code> 	<code>Server</code>	The ds_api.Server used to communicate with Pulse.	<i>required</i>

Returns:

Type	Description
<code>ModelManager<ManagedListModel></code>	List manager to manage Pulse lists.

manual_partitions_hash_based_partitioning_config

ManualPartitionsHashBasedPartitioningConfig

```
ManualPartitionsHashBasedPartitioningConfig(num_partitions: int = DEFAULT_PARTITIONS)
```

Bases: [PartitioningConfig](#)

Represents Manual Partitions Hash-Based Partitioning configuration.

Parameters:

Name	Type	Description	Default
<code>num_partitions</code>	int	The number of partitions to use. By default, <code>ManualPartitionsHashBasedPartitioningConfig.DEFAULT_PARTITIONS</code> .	<code>DEFAULT_PARTITIONS</code>

`DEFAULT_PARTITIONS` [class-attribute](#) [instance-attribute](#)

```
DEFAULT_PARTITIONS = DEFAULT_PARTITIONS
```

`num_partitions` [property](#)

```
num_partitions: int
```

Get The number of partitions to use.

`subclasses` [class-attribute](#) [instance-attribute](#)

```
subclasses: List[JavaWrapper] = []
```

model

Java ds-api counterpart in Java: `com.feedzai.commons.dsapi.Model` .

logger

```
logger = getLogger(__name__)
```

Model

```
Model(pyjnius_java)
```

Bases: [JavaWrapper](#)

Representing a Data Science Model in Pulse.

slug

```
slug: str
```

Sanitized name. Equivalent to Pulse's 'internal name'.

subclasses

```
subclasses: List[JavaWrapper] = []
```

evaluate

```
evaluate(name: str, input_datasource: DataSource, model_eval_config: ModelEvalConfig, wait: bool = False, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, file_format: str | None = None) -> 'Evaluation'
```

Evaluate the model against input_datasource using model_eval_config, with the option of using the default job profiles, a specific job profile, or custom spark_job_properties.

Note

if both job_properties and job_profile are specified and there is intersection between their properties, the job_properties property will prevail over the job_profile property.

Parameters:

Name	Type	Description	Default
name	str	Evaluation name.	required
input_datasource	DataSource	DataSource to be evaluated.	required
model_eval_config	ModelEvalConfig	ModelEvalConfig configuration for the model evaluation.	required
	bool		False

Name	Type	Description	Default
<code>wait</code> ¶		Whether to wait for the model evaluation to finish. The evaluation outcome will be logged. By default, False.	
<code>job_profile</code> ¶	JobProfile	Pulse UI custom job profile.	None
<code>job_properties</code> ¶	Mapping	Map containing spark job configurations. It will apply default configurations (Pulse Properties Profile) and job_properties.	None
<code>file_format</code> ¶	str	Output file format. When not defined, it will reuse the format of the input data source. Possible options are: "parquet" (preferred) or "csv".	None

Returns:

Type	Description
Evaluation	The model evaluation object.

Examples:

```
>>> app : App
>>> data = app.pullDataSourceByDesc("data")
>>> model_project = app.pullModelProjectByDesc("model_project")
>>> model = model_project.pullModelByDesc("model")
>>> job_profile = app.pullJobProfileByDesc("job_profile_name")
>>> eval_config = ModelEvalConfig()
...
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_profile=job_profile,
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_properties={"spark.executor.memory": "5g"},
... )
>>> # or
>>> evaluation = model.evaluate(
...     name=str(uuid.uuid4()),
...     input_datasource=data,
...     model_eval_config=eval_config,
...     job_profile=job_profile,
...     job_properties={"spark.executor.memory": "5g"},
... )
```

export_binary [¶](#)

```
export_binary() -> bytes | None
```

Export the model binary bundle in a ZIP.

The result of this export is a directory with the model ID and its contents inside it. E.g. for a LightGBM model, you should expect to have something akin to:

```
<model ID>/  
LightGBM_model.txt
```

To read its contents you can unzip it by doing

```
import io  
from zipfile import ZipFile  
  
with ZipFile(io.BytesIO(model.export_binary())) as f:  
    txt = f.read(f"{model.id}/LightGBM_model.txt").decode("ascii")
```

For arbitrary models, you can search for the files in the bundle by using `ZipFile.namelist` :

```
with ZipFile(io.BytesIO(binary)) as f:  
    print(f.namelist())
```

Note

In case the binary fails to export, this method returns None.

Returns:

Type	Description
<code>bytes</code>	The model binary bundle.

get_config [¶](#)

```
get_config()
```

Get the training configuration from the model.

Returns:

Type	Description
<code>TrainingConfig</code>	The training config.

get_hyperparameters [¶](#)

```
get_hyperparameters(ignore_defaults: bool = False, parameters_to_exclude: list[str] | set[str] |  
None = None) -> dict[str, Any]
```

Returns the mapping between hyperparameters and their values.

Parameters:

Name	Type	Description	Default
<code>ignore_defaults</code> ¶	<code>bool</code>	Whether to ignore or not hyperparameters with default values.	<code>False</code>
			<code>None</code>

Name	Type	Description	Default
<code>parameters_to_exclude</code> ¶	sequence of str	Hyperparameters to exclude from the result.	

Returns:

Type	Description
<code>dict[str, Any]</code>	Hyperparameters and their values for the model.

`get_ignored_fields` [¶](#)

```
get_ignored_fields() -> set[str]
```

Return the field names ignored during model training.

`get_schema` [¶](#)

```
get_schema() -> Schema | None
```

Return the schema associated with the data source used to train this ML model.

`refresh` [¶](#)

```
refresh(datasource: DataSource, target: str | None = None, new_model_name: str | None = None, model_project: ModelProject | None = None, model_config: TrainingConfig | None = None, wait: bool = False) -> 'Model'
```

Refreshes a model with a new DataSource.

Parameters:

Name	Type	Description	Default
<code>datasource</code> ¶	DataSource	The new datasource to train.	<i>required</i>
<code>target</code> ¶	str	The target column, default is the same as the previous model. This is optional for supervised models, but required for unsupervised models.	None
<code>new_model_name</code> ¶	str	The new model name, default is the model's name with suffix "_fresh" and current datetime as unix timestamp.	None
<code>model_project</code> ¶	ModelProject	The model project, default is the same one as the model.	None
<code>model_config</code> ¶	TrainingConfig	The model configuration, default is the same one as the model.	None
<code>wait</code> ¶	bool	Whether to wait for the model refresh to finish. By default, False.	False

Returns:

Type	Description
Model	The new model object.

Examples:

```
>>> data = app.pullDataSourceByDesc("data")
>>> model_project = app.pullModelProjectByDesc("model_project")
>>> model = model_project.pullModelByDesc("model")
...
>>> refreshed_model = model.refresh(data)
```


model_eval_config

ModelEvalConfig

```
ModelEvalConfig(ignored_fields: set[str] | None = None, cost_field: str | None = None, scored_instances_filepath: str | None = None, sink_desc: str | None = None, breakdowns: list[dict[str, set[str]]] | None = None, target_variable: str | None = None, schema: Schema | None = None)
```

Bases: [JavaWrapper](#)

A configuration used for Model evaluation.

Parameters:

Name	Type	Description	Default
ignored_fields	set of str	Set of names corresponding to the fields to be ignored when evaluating a model.	None
cost_field	str	Additional cost field used as weight for evaluation metrics.	None
scored_instances_filepath	str	Path to file where scored instances should be saved.	None
sink_desc	str	The sink name for the model evaluation datasource. If not provided the new datasource will take the evaluation's name.	None
breakdowns	list of mappings of (str->{str...})	Each element of the list represents a breakdown analysis. Each element in the contained mapping represents a breakdown field, with -> .	None
target_variable	str	The name of the target variable to be used in the model evaluation. This is required for unsupervised models and optional for supervised models. When provided for supervised models, it overrides the model's target variable. The target variable must be a binary categorical field (exactly 2 values) from the evaluation datasource and should be included in ignored_fields.	None
schema	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> import ds_api
>>> model: ds_api.Model # Train model with a ds_api.ModelProject
>>> # Define ModelEvalConfig w/ breakdown per value of `caucasian`:
```

```

>>> test_datasource: ds_api.DataSource
>>> eval_config = ds_api.ModelEvalConfig(
...     breakdowns=[
...         {"Caucasian": {"true", "false"}},
...         # {"Age Group": {"-25", "26-46", "46-65", "66-"}},
...         # ...
...     ],
...     schema=test_datasource.getSchema(),
... )
>>> evaluation = model.evaluate("test_eval", test_datasource, eval_config)
>>> evaluation.waitForEvaluationFinished()
>>> (
...     (_, global_results),
...     ((non_caucasian_col_val,), non_caucasian_results),
...     ((caucasian_col_val,), caucasian_results),
... ) = evaluation.getResultForBreakdowns("true")
>>> assert non_caucasian_col_val == "false" and caucasian_col_val == "true"

```

breakdowns property [🔗](#)

breakdowns

cost_field property [🔗](#)

cost_field

ignored_fields property [🔗](#)

ignored_fields

scored_instances_filepath property [🔗](#)

scored_instances_filepath

sink_desc property [🔗](#)

sink_desc

subclasses class-attribute instance-attribute [🔗](#)

subclasses: List[[JavaWrapper](#)] = []

target_variable property [🔗](#)

target_variable

auto_cast staticmethod [🔗](#)

auto_cast(java_object) -> [ModelEvalConfig](#)

Helper method called when a return obj comes from java to be auto converted to python.

Warning

In this case the result will not have a schema attached.

builder classmethod [🔗](#)

builder()

duplicate [↗](#)

```
duplicate(ignored_fields: set[str] | None = None, cost_field: str | None = None, scored_instances_filepath: str | None = None, sink_desc: str | None = None, breakdowns: list[dict[str, set[str]]] | None = None, target_variable: str | None = None, schema: Schema | None = None) -> ModelEvalConfig
```

Duplicate current ModelEvalConfig, with the possibility of updating its properties.

Parameters:

Name	Type	Description	Default
ignored_fields ↗	set[str] None	Set of names corresponding to the fields to be ignored when evaluating a model.	None
cost_field ↗	str None	Additional cost field used as weight for evaluation metrics.	None
scored_instances_filepath ↗	str None	Path to file where scored instances should be saved.	None
sink_desc ↗	str None	The sink name for the model evaluation datasource. If not provided the new datasource will take the evaluation's name.	None
breakdowns ↗	list[dict[str, set[str]]] None	Each element of the list represents a breakdown analysis. Each element in the contained mapping represents a breakdown field, with -> .	None
target_variable ↗	str None	The name of the target variable to be used in the model evaluation. This is required for unsupervised models and optional for supervised models. When provided for supervised models, it overrides the model's target variable. The target variable must be a categorical field present in the evaluation datasource and should be included in ignored_fields.	None
schema ↗	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Returns:

Type	Description
ModelEvalConfig	A model evaluation configuration.

model_project

logger module-attribute

```
logger = getLogger(__name__)
```

ModelProject

```
ModelProject(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Model Project in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

create classmethod

```
create(app, name)
```

train

```
train(name: str, input_datasource: DataSource, target: str, model_config: TrainingConfig, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, wait: bool = False) -> Model
```

Creates and trains a given model with TrainingConfig using DataSource with target label field, with the option of using the default job profiles, a specific job profile, or custom spark_job_properties.

Note: if both job_properties and job_profile are specified and there is intersection between their properties, the job_properties property will prevail over the job_profile property.

Parameters:

Name	Type	Description	Default
<code>name</code> 	<code>str</code>	Model name visible to the user.	<i>required</i>
<code>input_datasource</code> 	DataSource	Datasource used in the model training.	<i>required</i>
<code>target</code> 	<code>str</code>	Name of the label field.	<i>required</i>
<code>model_config</code> 	TrainingConfig	Training configuration used in the model training.	<i>required</i>
<code>job_profile</code> 	JobProfile	Pulse UI custom job profile.	<code>None</code>

Name	Type	Description	Default
<code>job_properties</code> ¶	Mapping	Map containing spark job configurations. It will apply default configurations (Pulse Properties Profile) and job_properties.	None
<code>wait</code> ¶	bool	Whether to wait for the model training to finish. The training outcome will be logged. By default, False.	False

Returns:

Type	Description
<code>Model</code>	The trained model.

models

Module that includes all available model training configurations.

MODELS_CLASSPATH module-attribute

MODELS_CLASSPATH: str = [DS_API_CLASSPATH](#) + 'models.'

TrainingConfig

TrainingConfig(builder, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: [Schema](#) | None = None)

Bases: [JavaWrapper](#)

The base class for a model training configuration. It takes care of the initialization of common parameters across all algorithms.

Parameters:

Name	Type	Description	Default
builder 		Builder that comes from the specific algorithm.	<i>required</i>
undersampling 	float	The value for undersampling. Only this or oversampling can be used.	None
oversampling 	float	The value for oversampling. Only this or undersampling can be used.	None
seed 	int	The seed value.	None
ignored_fields 	set of str	Set of names corresponding to the fields to be ignored when training a model.	None
schema 	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

ignored_fields property writable

ignored_fields

oversampling property writable

oversampling

seed property writable [¶](#)

seed

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

undersampling property writable [¶](#)

undersampling

partitioning_config

PartitioningConfig

```
PartitioningConfig(pyjnius_java)
```

Bases: [JavaWrapper](#)

Representing a Partitioning configuration.

subclasses **class-attribute** **instance-attribute** 

```
subclasses: List[JavaWrapper] = []
```


plan

DS-API module to manage Plans and their contents in Pulse.

Omnichannel Plans

At the moment Omnichannel integration is still in beta. Feature dependencies are unstable for Omnichannel plans.

AVAILABLE_FILE_FORMATS module-attribute

```
AVAILABLE_FILE_FORMATS = ['csv', 'parquet']
```

DIFF_NAME_PLACEHOLDER module-attribute

```
DIFF_NAME_PLACEHOLDER = '{plan_name}_diff_{other_plan_name}'
```

DeployablePlanConfig module-attribute

```
DeployablePlanConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.DeployablePlanConfig')
```

FilterFeature module-attribute

```
FilterFeature = autoclass(DS_API_CLASSPATH + 'plan.features.FilterFeature')
```

FilterOperatorConfig module-attribute

```
FilterOperatorConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.FilterOperatorConfig')
```

JChar module-attribute

```
JChar = autoclass('java.lang.Character')
```

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

Plan

```
Plan(pyjnius_java)
```

Bases: [JavaWrapper](#)

Representing a Plan in Pulse.

CSV_SCHEMA class-attribute instance-attribute

```
CSV_SCHEMA = Enum('CSV_SCHEMA', _headers)
```

main_channel property [¶](#)

```
main_channel: Channel
```

secondary_channels property [¶](#)

```
secondary_channels: list[Channel]
```

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

add_empty_filter_operator [¶](#)

```
add_empty_filter_operator(name: str) -> FilterOperator
```

Adds a new filter operator to the Plan.

Parameters:

Name	Type	Description	Default
name ¶	str	The name of the new operator	<i>required</i>

Returns:

Type	Description
FilterOperator	The new operator

add_empty_list_operator [¶](#)

```
add_empty_list_operator(name: str) -> ListOperator
```

Adds a new list operator to the Plan.

Parameters:

Name	Type	Description	Default
name ¶	str	The name of the new operator.	<i>required</i>

Returns:

Type	Description
ListOperator	The new operator.

add_empty_map_operator [¶](#)

```
add_empty_map_operator(name: str) -> MapOperator
```

Adds a new map operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the new operator.	<i>required</i>

Returns:

Type	Description
MapOperator	The new operator.

`add_empty_metric_operator` [¶](#)

```
add_empty_metric_operator(name: str) -> MetricOperator
```

Adds a new metric operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the new operator.	<i>required</i>

Returns:

Type	Description
MetricOperator	The new operator.

`add_operator` [¶](#)

```
add_operator(operator: BaseOperator) -> BaseOperator
```

Adds a constructed operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>operator</code> ¶	BaseOperator	Operator object.	<i>required</i>

Returns:

Type	Description
BaseOperator	The new operator.

`auto_rename_metrics` [¶](#)

```
auto_rename_metrics()
```

Automatically renames all the metrics based on their configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation fields}_{window size}_{is_real_time})` eg. `accountid_avg_amount_1h_rt`. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_`

`{window_size}_{is_real_time}`) . If a metric name is duplicated anywhere in the plan it appends "_n". Updates the references to the old names across the plan.

connect_operators [¶](#)

```
connect_operators(connection: Connection)
```

Adds a new connection between two Operators.

Parameters:

Name	Type	Description	Default
<code>connection</code> ¶	<code>Connection</code>	Connection between two operators.	<i>required</i>

connect_operators_names [¶](#)

```
connect_operators_names(op_names: list[str]) -> None
```

Connects the operators in the same order as they are indexed.

Only supports deployable linear plans.

Parameters:

Name	Type	Description	Default
<code>op_names</code> ¶	<code>list</code>	List of ordered operator names.	<i>required</i>

Raises:

Type	Description
<code>ValueError</code>	If the plan contains duplicated operator names. If there are not at least two operator names.
<code>TypeError</code>	If the operator name does not exist.

create_empty_deployable_plan `classmethod` [¶](#)

```
create_empty_deployable_plan(app: App, schema: Schema, timestamp_field_desc: str, name: str) -> Plan
```

Creates a new (empty) Deployable Plan in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Application ds_api object.	<i>required</i>
<code>schema</code> ¶	<code>Schema</code>	Pulse Schema	<i>required</i>
<code>timestamp_field_desc</code> ¶	<code>str</code>	Desc name of the schema timestamp field.	<i>required</i>

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the new plan.	<i>required</i>

Returns:

Type	Description
<code>Plan</code>	New Plan

`create_empty_omnichannel_plan` [¶](#) `classmethod` [¶](#)

```
create_empty_omnichannel_plan(app: App, name: str, main_channel: Channel, secondary_channels: list[Channel] | None = None) -> Plan
```

Create a new empty Deployable Omnichannel Plan in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Pulse application ds_api object.	<i>required</i>
<code>name</code> ¶	<code>str</code>	Name of the new plan.	<i>required</i>
<code>main_channel</code> ¶	<code>Channel</code>	Main channel configuration.	<i>required</i>
<code>secondary_channels</code> ¶	<code>list of Channel</code>	Secondary channels. When not configured or if the list is empty, this is equivalent to create an empty deployable plan.	<code>None</code>

Returns:

Type	Description
<code>Plan</code>	New Plan.

`create_snapshot` [¶](#)

```
create_snapshot(schema_name: str | None = None, snapshot_name: str | None = None) -> JavaClass
```

Creates a Plan snapshot.

Parameters:

Name	Type	Description	Default
<code>snapshot_name</code> ¶	<code>str</code>	Name of the snapshot, by default "planname_yyyyMMdd_HHmm_ms".	<code>None</code>
<code>schema_name</code> ¶	<code>str</code>	If specified it exports the schema with this name, by default None.	<code>None</code>

Returns:

Type	Description
PlanSnapshot	PlanSnapshot JavaClass

diff [↗](#)

```
diff(other: Plan, show: bool = True, only_diff: bool = True, save: bool = False, save_as: Optional[str] = None) -> Optional[Plan]
```

Compare 2 plans. Differences can be displayed graphically when running inside a Jupyter Notebook.

Note

When creating a new plan from the diff, the schema and timestamp fields must be the in both plans. Also note that a renamed operator acts like a new operator for the diff, which means an operator that is not present in the 'other' plan will be discarded.

Danger

List and Sample operators are not supported.

Parameters:

Name	Type	Description	Default
other ↗	Plan	Plan to compare against. The 'new' plan. When pushing the diff to Pulse, this is the same as doing <code>other - self</code> .	<i>required</i>
show ↗	bool	Whether to display the differences as an HTML output. Only possible within a Jupyter Notebook. Defaults to True.	True
only_diff ↗	bool	If True only shows features with differences.	True
save ↗	bool	If True it outputs a new valid plan with the differences. This will override 'show'. Defaults to False.	False
save_as ↗	str	The name of the diff plan, if None it will be <code>"{plan_a}_diff_{plan_b}"</code> .	None

Raises:

Type	Description
NotImplementedError	When trying to show a diff outside a Jupyter Notebook.
ValueError	When the plans do not have the same schema or a different timestamp field. This is only raised when 'return_diff_plan' is set to true.

disconnect_operators [↗](#)

```
disconnect_operators(connection: Connection)
```

Deletes the connection between two operators.

Parameters:

Name	Type	Description	Default
<code>connection</code> ¶	Connection	Connection between two operators	<i>required</i>

`duplicate` [¶](#)

```
duplicate(new_name: Optional[str] = None, app: Optional[App] = None) -> Plan
```

Duplicates current plan to Pulse.

Optionally allows to duplicate the plan to a different app. This assumes the necessary dependencies are available, ie. Plan schema and lists referenced by a ListOperator.

Parameters:

Name	Type	Description	Default
<code>new_name</code> ¶	<code>str</code>	Desc name of the duplicated plan, by default will be the current name followed by '_duplicate'.	None
<code>app</code> ¶	App	Application ds_api object where the Plan should be created in, by default will be the current app.	None

Returns:

Type	Description
Plan	Duplicated plan.

Raises:

Type	Description
<code>ValueError</code>	If a plan with the same name as the duplicate already exists.

`from_csv` classmethod [¶](#)

```
from_csv(app: App, schema: Schema | None = None, timestamp_field_desc: str | None = None, name: str | None = None, path: Path | str | None = None, *, main_channel: Channel | None = None, secondary_channels: list[Channel] | None = None) -> Plan
```

Returns a new Deployable Plan with the configuration from the csv file.

The new Plan configuration will be updated in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Application ds_api object.	<i>required</i>
<code>schema</code> ¶	Schema	Pulse schema	None
<code>timestamp_field_desc</code> ¶	<code>str</code>	Desc name of the schema timestamp field.	None

Name	Type	Description	Default
<code>name</code> ↗	<code>str</code>	Name of the new plan	None
<code>path</code> ↗	<code>str</code> or <code>Path</code>	Path to where the csv will be read from.	None
<code>main_channel</code> ↗	Channel	If a main_channel is given, the schema, and timestamp_field_desc parameters are ignored.	None
<code>secondary_channels</code> ↗	list of Channel	Optionally pass the omnichannel configuration for the plan. Note that it is assumed the "schema" and "timestamp_field_desc" are properties of the main channel.	None

Notes

This method is intended to edit an existing Plan configuration previously exported.

General guidelines to edit the csv:

- The row order of the 'operator_name' will determine their position in the Pulse Plan.
- Pulse fields must be enclosed within the pattern `${field}`.
- Boolean values: {True, "TRUE", "True", "true", "T", "t", "1", 1} are accepted.
- Any feature to be removed should be on a single row and the field 'map_remove_fields' should be True.
- Feature updates will occur if a feature name is repeated.

Fields input format and considerations:

- A more extensive documentation on each of the fields can be found on the Feature class docstrings.
- operator_type: possible values are: map, metric, list, filter.
- expression: to be used for Filter, Map and List operators.
- metric_groupby_fields: formatted as a list of strings, eg. `['MerchantName', 'City']`.
- metric_aggregation: Pulse fields must be enclosed within the pattern `'${field_desc}'`. eg. `'avg(${amount})'`, `'countDistinct(${MerchantCity})'`, `'count()'`.
- metric_window_size: usual types "second", "minute", "hour", "day", "week". eg. 1 day.
- metric_is_real_time: boolean.
- metric_filter: Fields must be enclosed with the pattern `'${field}'` eg. `'${TransactionAmount} > 100 and ${Fraud} != false'`.
- metric_delay: int.
- metric_is_tumbling: boolean.
- metric_filter_incomplete_instances: boolean, must be the same for the whole operator.
- metric_timestamp_field: must be the same for the whole operator.
- map_field_type: possible values are: numerical, string, datetime, categorical.
- map_field_config: contains additional field configuration for datetime fields, possible values: nanoseconds, microseconds, milliseconds, seconds, minutes, hours, days. Or categories for categorical fields, eg. `['true', 'false']`.
- map_remove_fields: boolean.
- list_operation: get, or contains.
- list_field: desc of the list field.

Returns:

Type	Description
Plan	New Plan

Raises:

Type	Description
Exception	If the Plan configuration is invalid.

from_pandas classmethod

```
from_pandas(app: App, schema: Schema | None = None, timestamp_field_desc: str | None = None,
name: str | None = None, pd_df: DataFrame | None = None, *, main_channel: Channel | None = None,
secondary_channels: list[Channel] | None = None) -> Plan
```

Returns a new Deployable Plan with the configuration from the pandas dataframe.

The new Plan configuration will be updated in Pulse.

Parameters:

Name	Type	Description	Default
app 	App	Application ds_api object.	required
schema 	Schema	Pulse schema	None
timestamp_field_desc 	str	Desc name of the schema timestamp field.	None
name 	str	Name of the new plan	None
pd_df 	DataFrame	Pandas dataframe with the plan features.	None
main_channel 	Channel	If a main_channel is given, the schema, and timestamp_field_desc parameters are ignored.	None
secondary_channels 	list of Channel	Optionally pass the omnichannel configuration for the plan. Note that it is assumed the "schema" and "timestamp_field_desc" are properties of the main channel.	None

See Also

from_csv : For guidelines to input csv data in the right format.

Returns:

Type	Description
Plan	New Plan

Raises:

Type	Description
(JavaException, ValueError, TypeError)	If the Plan configuration is invalid.

from_plan_snapshot classmethod [¶](#)

```
from_plan_snapshot(app: App, plan_snapshot: JavaWrapper) -> Plan
```

Returns a new Plan based on the snapshot.

Parameters:

Name	Type	Description	Default
app ¶	App		<i>required</i>
plan_snapshot ¶	JavaWrapper		<i>required</i>

Returns:

Type	Description
Plan	New Plan.

get_feature_dependencies [¶](#)

```
get_feature_dependencies(feature: BaseFeature, ascending=True) -> set[FeatureUID]
```

Gets a set of feature dependencies calculated recursively.

Feature dependencies are represented as a unique tuple of feature name and operator name. Considers the filter operator as a feature with dependencies.

Parameters:

Name	Type	Description	Default
feature ¶	BaseFeature	Feature to get the dependencies from.	<i>required</i>
ascending ¶	bool	If True it will look for dependencies in upstream operators, including the direct dependencies, otherwise it will look downstream. (Default = True).	True

Returns:

Type	Description
set	Sequence of FeatureUID with the names of the feature dependencies calculated recursively and the operator name where they are defined.

get_feature_operators [¶](#)

```
get_feature_operators() -> List[BaseFeaturesOperator]
```

Return this plan's 'feature operators', i.e., map, list, and metric operators. Filter operators are excluded.

Returns:

Type	Description
List of BaseFeaturesOperator	

Type	Description
	List of feature operators in the plan.

`get_features` [↗](#)

```
get_features() -> List[BaseFeature]
```

Return all features in the plan excluding "schema features".

`get_features_dependencies_by_type` [↗](#)

```
get_features_dependencies_by_type() -> Dict[str, Dict[str, Set[str]]]
```

Gets a set of feature dependencies calculated recursively.

Feature dependencies are represented as a unique tuple of feature name and operator name.

Considers the filter operator as a feature with dependencies.

If we have the following graph of dependencies for features A, B and C and specific dependency types:

```
[C]--(map)-->[B]--(aggregation)-->[A]
```

Feature C is considered an "aggregation" dependency of Feature A (regardless of its dependency type to B) because it is a dependency of the intermediate Feature B that is used in the aggregation expression of Feature A.

Returns:

Type	Description
<code>Dict[str, Dict[str, Set[str]]]</code>	Dictionary of features' dependencies by type. The keys are the names of the features and the values are the dicts of upstream dependencies by type.

`get_features_dependencies_graph` [↗](#)

```
get_features_dependencies_graph() -> DiGraph
```

Gets a directed graph object from NetworkX built with the dependencies between the Plan features.

Nodes are unique tuples of feature and operator names.

Considers the filter operator as a feature with dependencies.

Returns:

Type	Description
<code>DiGraph</code>	DiGraph

`get_operator_by_name` [↗](#)

```
get_operator_by_name(name: str) -> Optional[BaseOperator]
```

Selects the Pulse Operators with the matching name.

If more than one operator is found then the first one is returned.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the operator.	<i>required</i>

Returns:

Type	Description
<code>Optional[BaseOperator]</code>	The Pulse operator if matching name or None.

`get_operator_connections` [¶](#)

```
get_operator_connections() -> List[Connection]
```

Gets the operator connections.

Returns:

Type	Description
<code>list of Connections</code>	list of Connections

`get_operators` [¶](#)

```
get_operators() -> List[BaseOperator]
```

Gets the Plan operators.

Returns:

Type	Description
<code>list of BaseOperator</code>	List of operators in the plan.

`get_type` [¶](#)

```
get_type() -> str
```

Returns plan type.

Returns:

Type	Description
<code>str</code>	Either 'DEPLOYABLE' or 'NON_DEPLOYABLE'

`remove_feature_and_dependants` [¶](#)

```
remove_feature_and_dependants(feature: BaseFeature)
```

Removes the given feature and all the downstream dependencies.

Parameters:

Name	Type	Description	Default
	<code>BaseFeature</code>	Feature to be deleted and from which to remove downstream dependencies.	<i>required</i>

Name	Type	Description	Default
feature ↗			

remove_operator [↗](#)

```
remove_operator(operator: BaseOperator)
```

Removes an operator from the Plan.

Automatically deletes any connections involving this operator.

Parameters:

Name	Type	Description	Default
operator ↗	BaseOperator	Operator to be removed.	<i>required</i>

rename_feature [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

Renames the given features updating the references to the old names across the plan.

Parameters:

Name	Type	Description	Default
old_name ↗	str	Old name to change from.	<i>required</i>
new_name ↗	str	New name to change to.	<i>required</i>

run [↗](#)

```
run(input_datasources: list[DataSource], output_names: list[str], output_paths: list[str], *, secondary_datasources: dict[str, DataSource] | None = None, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, wait: bool = False, file_format: str = 'parquet', partitioning_config: PartitioningConfig = AdaptivePartitioningConfig(), **kwargs: bool | str | set[str]) -> str
```

Runs this Plan, supports deployable and non-deployable plans.

The plan will be run with the Pulse default config in place unless specified another Pulse job config and/or spark parameters.

Parameters:

Name	Type	Description	Default
input_datasources ↗	list	List of DataSource objects.	<i>required</i>
output_names ↗	list	Desc names of the output data sources.	<i>required</i>
	list	Paths to the output datasets.	<i>required</i>

Name	Type	Description	Default
<code>output_paths</code> ↗			
<code>secondary_data_sources</code> ↗	<code>dict[str, DataSource]</code>	Mapping of channel name to input Data source for secondary channels. For example <code>{"cards": <DataSource: CardsClean>}</code> .	None
<code>job_profile</code> ↗	JobProfile	Pulse UI custom job profile.	None
<code>job_properties</code> ↗	<code>dict</code>	Additional spark properties, eg. <code>{"spark.executor.memory": "1g"}</code> , by default no additional spark properties are added.	None
<code>wait</code> ↗	<code>bool</code>	Whether to wait for the plan execution to finish. The plan execution outcome will be logged. By default, False.	False
<code>file_format</code> ↗	<code>str</code>	Chosen file format for outputs. By default, parquet.	'parquet'
<code>partitioning_config</code> ↗	PartitioningConfig	The configuration for the partitioning approach to use when executing the plan. By default, <code>AdaptivePartitioningConfig()</code> .	AdaptivePartitioningConfig()

Other Parameters:

Name	Type	Description
<code>delimiter</code>	<code>str</code>	Delimiter character for the output datasets, by default ",". Only used if output format is csv.
<code>line_sep</code>	<code>str</code>	Line separator for the output datasets, by default "\n". Only used if output format is csv.
<code>quote_char</code>	<code>str</code>	Quotation character for the output datasets, by default "". Only used if output format is csv.
<code>header</code>	<code>bool</code>	Write out the column names, by default True. Only used if output format is csv.
<code>missing_values</code>	<code>set of str</code>	Missing values for the output datasets, by default "", empty string. Only used if output format is csv.

Returns:

Type	Description
<code>str</code>	Name of the plan execution.

`select_features` [↗](#)

```
select_features(feature_names: List[str])
```

Removes from the Plan any feature that is not needed to output the given list of features.

If a feature is updated, thus having duplicated names, it will conservatively keep all upstream dependencies.

Parameters:

Name	Type	Description	Default
<code>feature_names</code> ¶	<code>list of str</code>	List of feature names to keep.	<i>required</i>

`to_csv` [¶](#)

```
to_csv(path: Union[Path, str])
```

Exports the deployable Plan to a tabular csv format.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	<code>Path or str</code>	Path to where the csv will be saved.	<i>required</i>

`to_pandas` [¶](#)

```
to_pandas() -> DataFrame
```

Exports the deployable Plan to a pandas dataframe.

Returns:

Type	Description
<code>DataFrame</code>	Pandas dataframe containing features and all their editable attributes.

`to_pulse` [¶](#)

```
to_pulse()
```

Sends the current plan configuration of Operators and Features to Pulse.

Raises:

Type	Description
<code>JavaException</code>	If the Plan configuration is not valid.

`update_from_csv` [¶](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Deployable Plan with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	<code>str or path</code>	Path to where the csv will be read from.	<i>required</i>

See Also

from_csv : For guidelines to input csv data in the right format.

Raises:

Type	Description
(<code>JavaException</code> , <code>ValueError</code> , <code>TypeError</code>)	If the Plan configuration is invalid.

update_from_pandas [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Deployable Plan with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ↗	<code>DataFrame</code>	Pandas dataframe with the plan features.	<i>required</i>

See Also

from_csv : For guidelines to input csv data in the right format.

Raises:

Type	Description
(<code>JavaException</code> , <code>ValueError</code> , <code>TypeError</code>)	If the Plan configuration is invalid.

validate [↗](#)

```
validate()
```

Validates the current state of the Plan including any alterations to Operators and Features.

Raises:

Type	Description
<code>JavaException</code>	If the Plan configuration is invalid.

plans

Plans subpackage containing the implementation of features, operators and utilities to enable the programmatic manipulation of Plans.

Omnichannel Plans

At the moment Omnichannel integration is still in beta. Feature dependencies are unstable for Omnichannel plans.

rule

AnyFeature module-attribute

AnyFeature = Union[[ListFeature](#), [MetricFeature](#), [MapFeature](#)]

logger module-attribute

logger = getLogger(__name__)

ControlFlowActionTypeEnum

Bases: [JavaEnumWrapper](#)

Rule Control flow action enum.

CONTINUE class-attribute instance-attribute

CONTINUE = auto()

NEXT_WORKFLOW_BLOCK class-attribute instance-attribute

NEXT_WORKFLOW_BLOCK = auto()

STOP_WORKFLOW class-attribute instance-attribute

STOP_WORKFLOW = auto()

Rule

Rule(pyjnius_java)

Bases: [JavaWrapper](#)

Represents a Rule in Pulse.

action_params property

action_params: Mapping[str, List[[JavaWrapper](#)]]

Get the map of parameters to return within the actions triggered by this rule.

Returns:

Type	Description
Mapping[str, List[Rule ActionParam]]	The map of parameters to return within the actions triggered by this rule. The map indexes lists of parameters by the name of the action of the parameters.

actions property writable

actions: Set[str]

Get or set the list of actions to take when the rule evaluates to true.

app **property** [🔗](#)

app: [App](#)

Get the rule's application

comment **property** **writable** [🔗](#)

comment: str

Get or set the description of the rule.

condition **property** **writable** [🔗](#)

condition: str

Get or set the RDL expression used as a condition for triggering the rule.

control_flow **property** **writable** [🔗](#)

control_flow: [ControlFlowActionTypeEnum](#)

Get or set the rule control flow action to take after evaluating a rule.

desc **property** **writable** [🔗](#)

desc: str

Get or set the user visible name of this rule.

explanation_template **property** **writable** [🔗](#)

explanation_template: str

Get or set the velocity template used to generate the rule's explanation description.

fields **property** [🔗](#)

fields: Set[str]

Get the schema fields used in this rule's condition and variables.

id **property** [🔗](#)

id: str

Get the rule's id.

is_enabled **property** **writable** [🔗](#)

is_enabled: bool

Get or set the rule's active status.

is_score_enabled **property** **writable** [🔗](#)

is_score_enabled: bool

Get or set the flag that indicates if score and weight are enabled.

To set the flag to true, you must ensure that you first set the `Rule.scores` property, otherwise a validation error is thrown.

`is_snapshot` property [🔗](#)

```
is_snapshot: bool
```

Get the rule's snapshotness.

`managed_list_ids` property [🔗](#)

```
managed_list_ids: Set[str]
```

Get the Ids of the lists used by this rule.

`order` property [🔗](#)

```
order: int
```

Get the order in which this rule should be evaluated.

`original_id` property [🔗](#)

```
original_id: str
```

Get the rule's original id, if this is a snapshot rule.

`parent_rules_project` property [🔗](#)

```
parent_rules_project: RulesProject
```

Get the rules project this rule belongs to.

`parent_rules_project_snapshot_id` property [🔗](#)

```
parent_rules_project_snapshot_id: Optional[str]
```

Get the Id of the snapshot that this rule is associated to.

`schema` property [🔗](#)

```
schema: Schema
```

Get the schema associated to this rule and which belongs to the parent RulesProject.

`scores` property `writable` [🔗](#)

```
scores: Optional[List[float]]
```

Get or set an optional List that, if present, contains the scores that this rule should set when triggered.

The order in which the scores are present in the list should correspond to the order in which the possible target values are configured in the {@link AbstractRulesProjectModel#getSchema()}. The scores are expected to all be between Rule.model.MIN_RULE_SCORE and Rule.model.MAX_RULE_SCORE.

`subclasses` `class-attribute` `instance-attribute` [🔗](#)

```
subclasses: List[JavaWrapper] = []
```

`target_var_value` property `writable` [🔗](#)

```
target_var_value: Optional[str]
```

Get or set the value that should be assigned to the target variable.

`variables` `property` `writable` [¶](#)

```
variables: str
```

Get or set the RDL expression defining the variables used in the rule.

`weight` `property` `writable` [¶](#)

```
weight: int
```

Get or set the integer that defines the weight of this rule.

`create` `classmethod` [¶](#)

```
create(app: App, rules_project: RulesProject, name: str, condition: str, variables: Optional[str] = None, comment: Optional[str] = None, actions: Optional[Set[str]] = None, is_enabled: bool = True, control_flow: Optional[ControlFlowActionTypeEnum] = None, managed_lists_id: Optional[Set[str]] = None) -> Rule
```

Creates and returns a new Rule.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Application ds_api object.	<i>required</i>
<code>rules_project</code> ¶	<code>RulesProject</code>	<code>RulesProject</code> where the rule will be created.	<i>required</i>
<code>name</code> ¶	<code>str</code>	Name of the new rule.	<i>required</i>
<code>condition</code> ¶	<code>str</code>	Condition implemented by the rule.	<i>required</i>
<code>variables</code> ¶	<code>str</code>	Variables implemented to use in the rule's condition.	<code>None</code>
<code>comment</code> ¶	<code>str</code>	Comment on the rule.	<code>None</code>
<code>actions</code> ¶	<code>Set[str]</code>	Actions to take when the rule triggers. Default is an empty set.	<code>None</code>
<code>is_enabled</code> ¶	<code>bool</code>	Boolean identifying whether the rule is enabled or not. Default is True.	<code>True</code>
<code>control_flow</code> ¶	<code>ControlFlowActionTypeEnum</code>	Control flow action to take after evaluating a rule. Default is <code>ControlFlowActionTypeEnum.CONTINUE</code>	<code>None</code>
<code>managed_lists_id</code> ¶	<code>Set[str]</code>	Set of list_ids used by the rule. Default is an empty set.	<code>None</code>

Returns:

Type	Description
Rule	New Rule.

duplicate [¶](#)

```
duplicate(new_desc: Optional[str] = None, new_control_flow: Optional[ControlFlowActionTypeEnum] = None, new_comment: Optional[str] = None, new_actions: Optional[Set[str]] = None, new_variables: Optional[str] = None, new_condition: Optional[str] = None, new_rules_project: Optional[RulesProject] = None) -> Rule
```

Duplicate this rule, optionally changing some of its attributes.

Parameters:

Name	Type	Description	Default
<code>new_desc</code> ¶	str	The name of the duplicated rule. Defaults to the original name suffixed with '_duplicate'.	None
<code>new_control_flow</code> ¶	Optional[ControlFlowActionTypeEnum]	The control flow to use in the duplicate. Defaults to the original.	None
<code>new_actions</code> ¶	Optional[Set[str]]	The actions to used in the duplicate. Defaults to the original.	None
<code>new_comment</code> ¶	str	The comment to use in the duplicate. Defaults to the original.	None
<code>new_variables</code> ¶	str	The variables to use in the duplicate. Defaults to the original.	None
<code>new_condition</code> ¶	str	The condition to use in the duplicate. Defaults to the original.	None
<code>new_rules_project</code> ¶	RulesProject	The rules project to create the duplicate in. Defaults to the original.	None

Returns:

Type	Description
Rule	The duplicated rule.

Raises:

Type	Description
ValueError	Raised if a rule with the same name exists within the rules project.

from_dict classmethod [¶](#)

```
from_dict(app: App, rules_project: RulesProject, rule_as_dict: Dict[str, Any]) -> Rule
```

Create a Rule, given its dictionary representation.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	The app in which to create the rule in.	<i>required</i>
<code>rules_project</code> ¶	RulesProject	The rules project in which to create the rule in.	<i>required</i>
<code>rule_as_dict</code> ¶	<code>Dict[str, Any]</code>	The dictionary representation of the rule. It must have the following keys: <pre>name : str Name of the new rule. condition : str Condition implemented by the rule. variables : str Variables implemented to use in the rule's condition. comment : str Comment on the rule. actions : Set[str], optional Actions to take when the rule triggers. Default is an empty set. is_enabled : bool, optional Boolean identifying whether the rule is enabled or not. Default is True. control_flow : str, The name of the control flow action to take after evaluating a rule. It must be the name of a ControlFlowActionTypeEnum member. Default is `CONTINUE` target_var_value : str, optional The value that should be assigned to the target variable. is_score_enabled : bool, optional The flag that indicates if score and weight are enabled. scores : Sequence[float], optional The scores that this rule should set when triggered. weight : str, optional The integer that defines the weight of this rule explanation_template : str, optional The velocity template used to generate the rule's explanation description.</pre>	<i>required</i>

Returns:

Type	Description
Rule	The Rule created from the given dictionary representation.

`to_dict` [¶](#)

`to_dict()` -> `Dict[str, Any]`

Generate a description of this rule in dictionary format.

Returns:

Type	Description
<code>Dict[str, Any]</code>	A description of this rule's main attributes.

`update` [↗](#)

```
update(**kwargs: Mapping[str, Any]) -> None
```

Update several properties of this Rule at once.

Danger

This method doesn't check whether the properties you pass as a kwarg is one of the existing modifiable ones, so it's up to the caller to make sure the keys are correct.

Parameters:

Name	Type	Description	Default
<code>**kwargs</code> ↗	<code>Mapping[str, Any]</code>	Map between rule property names and the values we want to update them to	<code>{}</code>

Raises:

Type	Description
<code>TypeError</code>	If any of the given property values aren't of the required type

rules_project

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

RulesProject

```
RulesProject(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Rules Project in Pulse.

actions property writable

```
actions: dict[str, list[WorkflowActionParam]]
```

Get or set the actions that the rules project can take.

app property

```
app
```

Get the rules project's application

desc property writable

```
desc: str
```

Get or set the user visible name of this rules project.

evaluations property

```
evaluations: dict[str, RulesProjectEvaluation]
```

Get a mapping of this project's snapshots and their evaluations.

The key of the dictionary is the name of the snapshot, and the value is the corresponding snapshot evaluation object.

If a snapshot hasn't been evaluated, no entry for that snapshot is included in the dictionary.

id property

```
id: str
```

Get the rules project's id.

rules property

```
rules: list[Rule]
```

Get the rules belonging to this rules project.

schema property [¶](#)

```
schema: Schema
```

Get the schema associated with this rules project.

snapshots property [¶](#)

```
snapshots: dict[str, RulesProjectSnapshot]
```

Get a mapping of this project's snapshot objects and their names.

The key of the dictionary is the name of the snapshot, and the value is the corresponding snapshot object.

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

target_field property writable [¶](#)

```
target_field: str
```

Get or set the target field of this rules project.

tenancy_type property [¶](#)

```
tenancy_type: RulesProjectTenancyTypeEnum
```

Get the tenancy type of this rules project.

create classmethod [¶](#)

```
create(app: App, name: str, schema: Schema, target_field: str, actions: Mapping[str, Sequence[WorkflowActionParam]] | None = None) -> RulesProject
```

Creates and returns a new Rules Project.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Application ds_api object.	<i>required</i>
<code>name</code> ¶	str	Name of the new rules project.	<i>required</i>
<code>schema</code> ¶	Schema	Pulse schema	<i>required</i>
<code>target_field</code> ¶	str	Target field for the rules decision.	<i>required</i>
<code>actions</code> ¶	Mapping[str, Sequence[WorkflowActionParam]]	Mapping between the possible actions for the rules to trigger, and the respective parameters. Default is None.	None

Returns:

Type	Description
RulesProject	

Type	Description
	New RulesProject.

create_rule [¶](#)

```
create_rule(name: str, condition: str, variables: str | None = None, comment: str | None = None, actions: Set[str] | None = None, is_enabled: bool = True, control_flow: ControlFlowActionTypeEnum | None = None, managed_lists_id: Set[str] | None = None, reload: bool = True) -> Rule
```

Create a new rule within this rules project.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the rule to add.	<i>required</i>
<code>condition</code> ¶	<code>str</code>	The expression of the rule's triggering condition.	<i>required</i>
<code>variables</code> ¶	<code>str</code>	The rule's variables field, by default None.	None
<code>comment</code> ¶	<code>str</code>	The comment describing this rule, by default None.	None
<code>actions</code> ¶	<code>Set[str]</code>	Which actions are triggered by this rule, by default None.	None
<code>is_enabled</code> ¶	<code>bool</code>	Boolean identifying whether the rule is enabled or not, by default True.	True
<code>control_flow</code> ¶	ControlFlowActionTypeEnum	Control flow action to take after evaluating a rule, by default <code>ControlFlowActionTypeEnum.CONTINUE</code> .	None
<code>managed_lists_id</code> ¶	<code>Set[str]</code>	The lists associated with the rule, by default None.	None
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation.	True

Returns:

Type	Description
Rule	The newly created rule.

create_snapshot [¶](#)

```
create_snapshot(snapshot_desc: str | None = None, include_rules: Sequence[str | RulesProjectSnapshot] | None = None, find_by: str | None = None, reload: bool = True) -> RulesProjectSnapshot
```

Create a snapshot of the rules project and some/all of its rules.

Parameters:

Name	Type	Description	Default
snapshot_desc ¶	str	Name given to the snapshot. If None is provided, a snapshot name is given using the format: %m-%d-%Y_%H-%M-%S_project-desc .	None
include_rules ¶	Sequence[str Rule]	The list of rule objects, rule ids or rule names which uniquely identify the rules. If None, then all rules in the project are included in the snapshot, which is the default behavior.	None
find_by ¶	('object', 'id', 'desc')	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>include_rules</code> parameter: • 'object' : Default behaviour. Include the given rule objects. • 'id' : Include the rules in the project, given the list of rule ids. • 'desc' : Include the rules in the project, following the order of the given rule's user visible names.	'object'
reload ¶	bool	If this rules project should be reloaded, by default True.	True

Returns:

Type	Description
RulesProjectSnapshot	The newly created rules project snapshot.

delete_rule [¶](#)

```
delete_rule(rule: str, find_by: str, reload: bool = True) -> None
```

Delete a rule associated with this rules project.

Parameters:

Name	Type	Description	Default
rule ¶	str	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to delete.	<i>required</i>
find_by ¶	('id', 'desc')	Name of the property of a Rule, which uniquely identifies the rule to delete, which describes the given <code>rule</code> : • 'id' : delete rule with the given id. • 'desc' : delete rule with the given user visible name.	'id'
reload ¶	bool	If the rules project should be reloaded to reflect the deleted rule, by default True.	True

delete_snapshot [↗](#)

```
delete_snapshot(snapshot: str, find_by: str, reload: bool = True) -> None
```

Delete a snapshot associated with this rules project.

Parameters:

Name	Type	Description	Default
snapshot ↗	str	The value of the snapshot property, specified by <code>find_by</code> , which identifies the snapshot to delete.	<i>required</i>
find_by ↗	('id', 'desc')	Name of the property of a snapshot, which uniquely identifies the snapshot to delete, which describes the given <code>snapshot</code> : 'id' : delete snapshot with the given id. 'desc' : delete snapshot with the given user visible name.	'id'
reload ↗	bool	If the rules project should be reloaded to reflect the deleted snapshot, by default True.	True

duplicate_rule [↗](#)

```
duplicate_rule(rule: str | Rule, find_by: str = OBJECT, new_desc: str | None = None, new_control_flow: ControlFlowActionTypeEnum | None = None, new_comment: str | None = None, new_actions: Set[str] | None = None, new_variables: str | None = None, new_condition: str | None = None, reload: bool = True) -> Rule
```

Duplicate the given rule within this project.

Optionally, you can change some attributes of the duplicate.

If an object is provided to the `rule` parameter, the rule doesn't have to belong to this project. It will take the properties of the given rule, and create a new one within this project with those properties.

Parameters:

Name	Type	Description	Default
rule ↗	str Rule	The value of the rule property, or the rule object itself, specified by <code>find_by</code> , which identifies the rule to duplicate.	<i>required</i>
find_by ↗	('id', 'desc', 'object')	Name of the property of a Rule, which uniquely identifies the rule to update (or the rule object itself), which describes the given <code>rule</code> parameter: 'id' : duplicate rule with the given id. 'desc' : duplicate rule with the given user visible name. (Default) 'object' : duplicate the given rule.	'id'
new_desc ↗	str	The name of the duplicated rule. Defaults to the original name suffixed with '_duplicate'.	None
new_control_flow ↗	ControlFlowActionTypeEnum None	The control flow to use in the duplicate. Defaults to the original.	None

Name	Type	Description	Default
<code>new_actions</code> ¶	<code>Set[str] None</code>	The actions to used in the duplicate. Defaults to the original.	<code>None</code>
<code>new_comment</code> ¶	<code>str</code>	The comment to use in the duplicate. Defaults to the original.	<code>None</code>
<code>new_variables</code> ¶	<code>str</code>	The variables to use in the duplicate. Defaults to the original.	<code>None</code>
<code>new_condition</code> ¶	<code>str</code>	The condition to use in the duplicate. Defaults to the original.	<code>None</code>
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation. By default True.	<code>True</code>

Returns:

Type	Description
<code>Rule</code> ¶	The duplicated rule.

Raises:

Type	Description
<code>ValueError</code>	Raised if a rule with the same name exists within the rules project.

`evaluate` [¶](#)

```
evaluate(input_datasource: DataSource, evaluation_config: RulesProjectEvaluationConfig, snapshot_desc: str | None = None, include_rules: list[str] | None = None, find_by: str | None = None, profile_file_name: str | None = None, spark_job_properties: Mapping[str, str] | None = None, wait: bool = True, reload: bool = True) -> RulesProjectEvaluation
```

Create a snapshot and evaluate it on the given input data source.

Both spark job profile name and profile properties can be passed at the same time, but the profile name is prioritized.

Parameters:

Name	Type	Description	Default
<code>input_data_source</code> ¶	<code>DataSource</code> ¶	Data to run through the rules project to calculate alerts.	<i>required</i>
<code>evaluation_config</code> ¶	<code>RulesProjectEvaluationConfig</code> ¶	Configuration for the snapshot evaluation.	<i>required</i>
	<code>str</code>	Name given to the snapshot. If none is provided, a snapshot name is given using the format: <code>%m-%d-%Y_%H-%M-%S_project-desc</code> .	<code>None</code>

Name	Type	Description	Default
<code>snapshot_desc</code> ¶			
<code>include_rules</code> ¶	Sequence[str RulesProjectSnapshot]	The list of snapshots objects, snapshots ids or snapshots names which uniquely identify the rules. If None, then all rules in the project are included in the snapshot, which is the default behavior.	None
<code>find_by</code> ¶	('object', 'id', 'desc')	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>include_rules</code> parameter: • 'object' : Default behaviour. Include the given rule objects. • 'id' : Include the rules in the project, given the list of rule ids. • 'desc' : Include the rules in the project, following the order of the given rule's user visible names.	'object'
<code>profile_name</code> ¶	str None	Job profile name, by default None	None
<code>spark_job_properties</code> ¶	Mapping None	Job profile properties, by default None	None
<code>wait</code> ¶	bool	Waits for the snapshot evaluation to finish before returning the evaluation object.	True
<code>reload</code> ¶	bool	If the rules project should be reloaded to reflect the created snapshot and evaluation, by default True.	True

Returns:

Type	Description
<code>RulesProjectEvaluation</code> ¶	The evaluation object containing its status.

from_dict [¶](#) `classmethod` [¶](#)

```
from_dict(app: App, rules_project_as_dict: dict[str, Any]) -> RulesProject
```

Create a Rules Project, given its dictionary representation.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code> ¶	The app in which to create the rules project on.	<i>required</i>
<code>rules_project_as_dict</code> ¶	dict[str, Any]	The dictionary representation of the rules project. It must have the following keys: desc : str User visible name of the rules project to create schema : str Name of the schema to be used for the project, which must already exist in the app target_field : str Target field for the rules decision. actions : dict[str, dict[str, Any], optional Mapping between the action names, and the action parameters. The available parameters are:	<i>required</i>

Name	Type	Description	Default
		<pre> param_desc : str, The user visible name of the action parameter. param_type : str The name of the parameter type, which must be the name of an ActionParamTypeEnum member. param_is_mandatory : bool A flag defining if the parameter is mandatory or not. param_resource_id : str The Id of the resource used if the type requires that information (e.g.: ActionParamTypeEnum.MANAGED_LIST). Default is None rules: Sequence[dict[str, Any]], optional The rules to be created, represented as a dictionary, following the same representation expected from Rule.from_dict </pre>	

Returns:

Type	Description
RulesProject	The Rules Project created from the given dictionary.

get_rule [¶](#)

```
get_rule(rule: str, find_by: str) -> Rule | None
```

Retrieve a rule associated with this Rules Project.

Parameters:

Name	Type	Description	Default
rule ¶	str	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to get.	<i>required</i>
find_by ¶	('id', 'desc')	Name of the property of a Rule, which uniquely identifies the rule to get, which describes the given <code>rule</code> : 'id' : get rule with the given id. 'desc' : get rule with the given user visible name.	'id'

Returns:

Type	Description
Rule None	The Rule with the provided <code>find_by</code> property or None if it doesn't exist.

get_snapshot [¶](#)

```
get_snapshot(snapshot: str, find_by: str) -> RulesProjectSnapshot
```

Retrieve a rule associated with this Rules Project.

Parameters:

Name	Type	Description	Default
	str		<i>required</i>

Name	Type	Description	Default
snapshot ¶		The value of the snapshot property, specified by <code>find_by</code> , which identifies the snapshot to get.	
find_by ¶	<code>('id', 'desc')</code>	Name of the property of a snapshot, which uniquely identifies the snapshot to get, which describes the given <code>snapshot</code> : 'id' : get snapshot with the given id. 'desc' : get snapshot with the given user visible name.	<code>'id'</code>

Returns:

Type	Description
RulesProjectSnapshot <code>None</code>	The Rules Project Snapshot with the provided <code>find_by</code> property or <code>None</code> if it doesn't exist.

reorder_rules [¶](#)

```
reorder_rules(rules_in_order: Sequence[Union[str, Rule]], find_by: str = OBJECT, tenant_id: str | None = None) -> None
```

Update the order of this project's rules in the context of a tenant.

Parameters:

Name	Type	Description	Default
rules_in_order ¶	<code>Sequence[Union[str, Rule]]</code>	The sorted list of rule objects, rule ids or rule names which uniquely identify the rules.	<i>required</i>
find_by ¶	<code>('object', 'id', 'desc')</code>	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>rules_in_order</code> parameter: 'object' : Default behaviour. Re-order the rules in the project, following the order of the given rules instances. 'id' : Re-order the rules in the project, following the order of the given rule ids. 'desc' : Re-order the rules in the project, following the order of the given rule's user visible names.	<code>'object'</code>
tenant_id ¶	<code>str</code>	The id of the tenant where this operation is being performed, default is <code>None</code> .	<code>None</code>

Returns:

Type	Description
<code>bool</code>	True if any of the rules has been updated, false otherwise.

shift_rule [¶](#)

```
shift_rule(rule: str, find_by: str, offset: int) -> None
```

Moves the rule the number of positions specified by the offset.

This will in shift this rule in the rules project's ordered list of rules, copying the rule order value between affected rules. Therefore, even if the order values are not continuous (e.g. 1, 3, 5) they will be preserved as a whole but assigned to different rules (i.e. after any shift the order values will still be 1, 3 and 5).

A rule will only be shifted forward (positive offset) or backwards (negative offset) at most until it reaches the end or beginning of the list, respectively - i.e. the list is not treated like a circular buffer.

Parameters:

Name	Type	Description	Default
<code>rule</code> ¶	<code>str</code>	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to shift.	<i>required</i>
<code>find_by</code> ¶	<code>('id', 'desc')</code>	Name of the property of rules, which uniquely identifies the rule to shift, whose value is provided by the <code>rule</code> : • 'id' : shift rule with the given id. • 'desc' : shift rule with the given user visible name.	<code>'id'</code>
<code>offset</code> ¶	<code>int</code>	The number of positions to move the rule. It can be positive to move the rule <code>offset</code> positions forward or negative to move it backwards.	<i>required</i>

`to_dict` [¶](#)

```
to_dict(include_params: Sequence[str] | None = None) -> dict[str, Any]
```

Get a dictionary representation of this project and its rules.

Parameters:

Name	Type	Description	Default
<code>include_params</code> ¶	<code>Sequence[str]</code>	Properties of the rules project to include in the dictionary. Can be one of {'id', 'desc', 'app_id', 'tenancy_type', 'schema_id', 'target_field', 'actions', 'rules'}	<code>None</code>

Returns:

Type	Description
<code>dict[str, Any]</code>	A DataFrame with the properties for all rules in the project, along with the project's properties.

`update_rule` [¶](#)

```
update_rule(rule: str, find_by: str, reload: bool = True, **kwargs: Mapping[str, Any]) -> None
```

Update several properties of a rule associated with this project.

Warning

This method doesn't check whether the properties you pass as a kwarg is one of the existing modifiable ones, so it's up to the caller to make sure the keys are correct.

Parameters:

Name	Type	Description	Default
<code>rule</code> ¶	<code>str</code>	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to update.	<i>required</i>

Name	Type	Description	Default
<code>find_by</code> ¶	<code>('id', 'desc')</code>	Name of the property of a Rule, which uniquely identifies the rule to update, which describes the given rule : 'id' : update rule with the given id. 'desc' : update rule with the given user visible name.	<code>'id'</code>
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation. By default True.	<code>True</code>
<code>**kwargs</code> ¶	<code>Mapping[str, Any]</code>	Map between rule property names and the values we want to update them to	<code>{}</code>

Raises:

Type	Description
<code>TypeError</code>	If any of the given property values aren't of the required type

RulesProjectTenancyTypeEnum [¶](#)

Bases: `JavaEnumWrapper`

Possible types of tenancy for rules projects.

MULTI_TENANT_DISTINCT `class-attribute` `instance-attribute` [¶](#)

```
MULTI_TENANT_DISTINCT = auto()
```

Multi-tenant distinct. Each tenant may have its individual rules.

MULTI_TENANT_SHARED `class-attribute` `instance-attribute` [¶](#)

```
MULTI_TENANT_SHARED = auto()
```

Multi-tenant shared. Rules are common to all tenants but are processed independently by each tenant. This means events are filtered by tenant on evaluations, and that on stateful rules each tenant has a dedicated rules engine.

SINGLE_TENANT `class-attribute` `instance-attribute` [¶](#)

```
SINGLE_TENANT = auto()
```

Single tenant, the default. There is no tenant segregation.

rules_project_evaluation

RulesProjectEvaluation

RulesProjectEvaluation(pyjnius_java)

Bases: [JavaWrapper](#)

Represents a Rules Project Snapshot Evaluation in Pulse.

created_at property

created_at: datetime

Get the datetime from when the evaluation was created.

desc property

desc: str

Get the user-facing name for the evaluation

exported_datasource property

exported_datasource: Optional[[DataSource](#)]

Get the DataSource exported with the evaluation results, if applicable.

input_datasource property

input_datasource: [DataSource](#)

Get the DataSource that was used to run the evaluation.

subclasses class-attribute instance-attribute

subclasses: List[[JavaWrapper](#)] = []

has_finished_successfully

has_finished_successfully() -> bool

wait_until_finished

wait_until_finished() -> bool

Waits for the snapshot evaluation to finish and then reloads it.

Returns:

Type	Description
bool	True if the evaluation finished successfully.

rules_project_evaluation_config

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

RulesProjectEvaluationConfig

```
RulesProjectEvaluationConfig(output_datasource_uri: str, output_datasource_name: str, filter_expression: str | None = None, cost_field: str | None = None, breakdowns: dict[str, set[str]] | None = None, timestamp_field: str | None = None, output_datasource_config: DataSourceConfig | None = None, file_format: str | None = None, **kwargs)
```

Bases: [JavaWrapper](#)

A configuration used for Rules Project Snapshot evaluation.

Create a new Rules Project Snapshot evaluation configuration.

Parameters:

Name	Type	Description	Default
output_datasource_uri	str	The URI of the datasource with the exported evaluation.	required
output_datasource_name	str	The name of the datasource with the exported evaluation.	required
filter_expression	str	The expression that filters events from being evaluated.	None
cost_field	str	The cost field used in the evaluation.	None
breakdowns	dict[str, set[str]]	The breakdown fields and values.	None
timestamp_field	str	The timestamp field used in the evaluation.	None
output_datasource_config	DataSourceConfig	The datasource config that includes the .csv writing options, such as delimiter, and encoding, of the datasource with the exported evaluation. Deprecated: Please use file_format instead.	None
	str		None

Name	Type	Description	Default
file_format 🔗		Output file format. When not defined, it will reuse the format of the input data source. Possible options are: "parquet" (preferred) or "csv".	

Other Parameters:

Name	Type	Description
has_header	bool	Flags whether the dataset is to be read with header. Defaults to True. <i>CSV config</i> .
sep	str	Delimiter to use. Must contain exactly one character. Defaults to ','. <i>CSV config</i> .
missing_values	set of str	Set of strings with values for empty value. Defaults to {"", "NA", "NULL"}. <i>CSV config</i> .
quote	str	The character used to denote the start and end of a quoted item. Defaults to '"'. <i>CSV config</i> .
line_separator	str	The string used mark the end of line. Defaults to '\n'. <i>CSV config</i> .
encoding	str	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'. <i>CSV config</i> .
merge_schemas	bool	Specify if schema merging should be allowed. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging <i>Parquet config</i> .

breakdowns [property](#) [🔗](#)

```
breakdowns: dict[str, set[str]]
```

Get the breakdown fields and values.

cost_field [property](#) [🔗](#)

```
cost_field: str | None
```

Get the cost field used in the evaluation.

exported_datasource [property](#) [🔗](#)

```
exported_datasource: DataSource | None
```

Get datasource with the exported evaluation data.

exported_datasource_name [property](#) [🔗](#)

```
exported_datasource_name: str | None
```

Get the name of the datasource with the exported evaluation.

exported_datasource_uri [property](#) [🔗](#)

```
exported_datasource_uri: str | None
```

Get the URI of the datasource with the exported evaluation.

filter_expression property [🔗](#)

```
filter_expression: str | None
```

Get the expression that filters events from being evaluated.

subclasses class-attribute instance-attribute [🔗](#)

```
subclasses: List[JavaWrapper] = []
```

timestamp_field property [🔗](#)

```
timestamp_field: str | None
```

Get the timestamp field used in the evaluation.

rules_project_snapshot

Java ds-api counterpart in :java: `com.feedzai.commons.dsapi.RulesProjectSnapshot` .

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

RulesProjectSnapshot

```
RulesProjectSnapshot(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Rules Project Snapshot in Pulse.

actions property

```
actions: Dict[str, Any]
```

Get the actions that the snapshot can take.

app property

```
app: App
```

desc property

```
desc: str
```

Gets the user visible name of this rules project snapshot.

evaluation property

```
evaluation: Optional[RulesProjectEvaluation]
```

Gets the last evaluation ran for this snapshot, if any.

id property

```
id: str
```

Gets the id of this rules project snapshot.

original_rules_project property

```
original_rules_project: RulesProject
```

Gets the rules project from which this snapshot was generated.

rules property

```
rules: List[Rule]
```

Gets the snapshot's rules.

schema property [¶](#)

schema: [Schema](#)

Gets the schema associated with the snapshot.

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

target_field property [¶](#)

target_field: str

Gets the target field of RulesProject associated to this snapshot.

evaluate [¶](#)

```
evaluate(input_datasource: DataSource, evaluation_config: RulesProjectEvaluationConfig, profile_name: Optional[str] = None, spark_job_properties: Optional[Mapping] = None, wait: bool = True) -> RulesProjectEvaluation
```

Run a snapshot evaluation on an input data source.

Both spark job profile name and profile properties can be passed at the same time, but the profile name is prioritized.

Parameters:

Name	Type	Description	Default
input_datasource ¶	DataSource	Data to run through the rules project to calculate alerts.	<i>required</i>
evaluation_config ¶	RulesProjectEvalC onfig	Configuration for the snapshot evaluation.	<i>required</i>
profile_name ¶	str	Job profile name, by default None.	None
spark_job_properties ¶	Mapping	Job profile properties, by default None.	None
wait ¶	bool	Waits for the snapshot evaluation to finish before returning the evaluation object.	True

Returns:

Type	Description
RulesProjectEvaluation	The evaluation object containing its status and results.

sampling

Efficient sampling utilities through PySpark. This sub-package provides a stratified and a random sampling strategy, but can be easily extended to user defined strategies.

It supports smart optimized sampling that preserves the minimal history of entities, so historical profiles can be correctly computed while maximizing the sampling reduction power.

If you intend to implement your own sampling algorithm, that's fine: there's a `BaseSampler` that takes care of the integration with Pulse through the ds-api.

schema

Java ds-api counterpart in :java: `com.feedzai.commons.dsapi.Schema` .

Field module-attribute

```
Field = autoclass(DS_API_CLASSPATH + 'Field')
```

FieldType module-attribute

```
FieldType = autoclass('com.feedzai.pulse.esp.FieldType')
```

logger module-attribute

```
logger = getLogger(__name__)
```

Schema

```
Schema(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Schema in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_field

```
add_field(field: FieldTemplate, index: int | None = None)
```

Adds a new field to the schema.

Parameters:

Name	Type	Description	Default
field 	FieldTemplate	The new field, which is created according to the field template and added to the schema.	<i>required</i>
index 	int	The index of the new field in the Schema's list of fields. For example, if <code>index = 0</code> , the new schema will be added in the first position. If no index is specified, the field is appended to the end of the Schema.	None

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="target-schema")
>>> schema.add_field(field=FieldTemplate(
...     "numeric_field",
...     NativeType.FLOAT,
...     NumericalField())
```

```
... )
>>> schema.add_field(field=
...     FieldTemplate("categorical_field",
...         NativeType.STRING,
...         CategoricalField({"value1", "value2"}))
... ), index=0)
```

add_fields [¶](#)

```
add_fields(fields: list[FieldTemplate])
```

Adds a list of new fields to the schema. The fields are added at the end of the schema following the order of the provided list.

Parameters:

Name	Type	Description	Default
fields ¶	list of <code>FieldTemplate</code>	The list of new fields, which are created according to the corresponding field template and added to the schema.	<i>required</i>

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="target-schema")
>>> fields = [
...     FieldTemplate("numeric_field", NativeType.FLOAT, NumericalField()),
...     FieldTemplate(
...         "timestamp_msec_field",
...         NativeType.LONG,
...         DateTimeField(TimestampFormat("MILLISECONDS")))
...     ),
... ]
>>> schema.add_fields(fields)
```

create classmethod [¶](#)

```
create(app, name)
```

Create a new Schema in `app` with the given `name` .

Parameters:

Name	Type	Description	Default
app ¶	<code>App</code>	The App where the schema will be created.	<i>required</i>
name ¶	<code>str</code>	The name of the schema. Has to start with a non-special character.	<i>required</i>

Returns:

Type	Description
<code>Schema</code>	Schema object corresponding to the Pulse schema created.

desc_to_name [¶](#)

```
desc_to_name(desc: str) -> str
```

Return the internal (sanitized) representation of a field name as it appears in the UI. If `desc` is already the sanitized version of the name, that is returned instead.

Raises:

Type	Description
<code>ValueError</code>	When <code>desc</code> is not part of the schema.

duplicate [¶](#)

```
duplicate(new_name: str | None = None) -> Schema
```

Duplicates the schema.

Parameters:

Name	Type	Description	Default
<code>new_name</code> ¶	<code>str</code>	Name of the schema duplicate, by default will be the name of the source schema followed by <code>'_duplicate'</code> .	<code>None</code>

Returns:

Type	Description
<code>Schema</code>	The schema duplicate.

name_to_desc [¶](#)

```
name_to_desc(name: str) -> str
```

Return the description of a field as it appears in Pulse UI from its internal name representation.

Raises:

Type	Description
<code>ValueError</code>	When <code>name</code> is not part of the schema.

remove_fields [¶](#)

```
remove_fields(fields_to_remove: set[str])
```

Removes a list of fields from the schema.

Parameters:

Name	Type	Description	Default
<code>fields_to_remove</code> ¶	<code>Set[str]</code>	A set with the names of the fields to remove.	<i>required</i>

Raises:

Type	Description
<code>ValueError</code>	If at least one of the provided field names does not exist in the schema.

reorder_fields [¶](#)

```
reorder_fields(order: tuple[int, ...])
```

Reorders the fields in the schema according to the specified list of indices. The new order is specified with a tuple of integers where each element is the index of the field in the original list of fields.

Parameters:

Name	Type	Description	Default
<code>order</code> ¶	<code>tuple of int</code>	Tuple with the field indices in the new order.	<i>required</i>

Raises:

Type	Description
<code>(ValueError, TypeError)</code>	If the specified order is not valid.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="four-field-schema")
>>> schema.reorder_fields((1, 0, 2, 3))
>>> schema.reorder_fields((3, 2, 1, 0))
```

reorder_fields_by_name [¶](#)

```
reorder_fields_by_name(ordered_names: tuple[str, ...])
```

Reorders the fields in the schema according to the specified list of names.

Parameters:

Name	Type	Description	Default
<code>ordered_names</code> ¶	<code>tuple[str, ...]</code>	Tuple with the field names in the new order. The tuple must be an exact match to the schema field names.	<i>required</i>

Raises:

Type	Description
<code>(ValueError, TypeError)</code>	If the specified list of names is not an exact match to the schema field names.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="four-field-schema")
>>> schema.reorder_fields_by_name(("field2", "field1", "field3", "field4"))
>>> schema.reorder_fields_by_name(("field4", "field3", "field2", "field1"))
```

get_internal_name

```
get_internal_name(desc: str, schema: Schema | None = None) -> str
```

Return the internal (sanitized) name of a field.

This is a utility function to improve the UX of ds-api. Through ds_api the user should only care about the names as they appear in the UI ('desc').

Note

If a name is not found or the schema is not passed, this function acts as a no-op.

get_internal_names

```
get_internal_names(names: _T, schema: Schema | None = None) -> _T
```

Return a sequence of internal (sanitized) names based on names as they appear in the UI.

Examples:

```
>>> get_internal_names({"A Fancy Name here", "and another there"}, schema)
{"a_fancy_name_here", "and_another_there"}
```

See Also

[ds_api.schema.get_internal_name](#)

server

KEYTAB_SPARK_PROP module-attribute

```
KEYTAB_SPARK_PROP = 'spark.yarn.keytab'
```

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

PulseGlobalConfiguration module-attribute

```
PulseGlobalConfiguration = autoclass('com.feedzai.pulse.server.api.config.PulseGlobalConfigurati  
on')
```

Server

```
Server(pyjnius_java)
```

Bases: [JavaWrapper](#)

Server class that connects to the Pulse API.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

connect classmethod

```
connect(address: str = 'localhost', port: int = 1099, timeout: int = 10000, **kwargs)
```

Creates a new instance of com.feedzai.commons.dsapi.Server to connect to Pulse

Parameters:

Name	Type	Description	Default
<code>address</code> 	<code>str</code>	The hostname.	<code>'localhost'</code>
<code>port</code> 	<code>int</code>	The RMI port.	<code>1099</code>
<code>timeout</code> 	<code>int</code>	Maximum time to wait until timeout.	<code>10000</code>

Returns:

Type	Description
<code>Server</code>	The reference to the Pulse API.

get_datascience_config [↗](#)

```
get_datascience_config()
```

Return the DataScienceConfig object.

get_distributed_job_config [↗](#)

```
get_distributed_job_config()
```

Return the DistributedJobServiceConfig object.

get_kerberos_config [↗](#)

```
get_kerberos_config() -> dict
```

Return the DataScience Kerberos configuration as a dict.

spark_job_config [↗](#)

JOB_BACKEND_IMPL module-attribute [↗](#)

```
JOB_BACKEND_IMPL = autoclass('com.feedzai.pulse.service.datascience.config.JobConfigurator$JobBackendImplementation')
```

JOB_TYPE module-attribute [↗](#)

```
JOB_TYPE = autoclass('com.feedzai.pulse.service.datascience.JobType')
```

KEY_TAB_DIR module-attribute [↗](#)

```
KEY_TAB_DIR = 'KEY_TAB_DIR'
```

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

SparkJobConfig [↗](#)

```
SparkJobConfig(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a spark job configuration.

subclasses class-attribute instance-attribute [↗](#)

```
subclasses: List[JavaWrapper] = []
```

create classmethod [↗](#)

```
create(sparkJobName, sparkConfig, kerberosConfig)
```

SparkMasterURL dataclass [↗](#)

```
SparkMasterURL(master: str, deploy_mode: str | None)
```

Stores information about the Spark master URL.

This class is useful to parse master URLs from Pulse Configuration, while maintaining compatibility with different Spark versions/distributions.

See more at [Submitting Applications / Master URLs](#)

Attributes:

Name	Type	Description
master	str	Spark master URL. One of 'local', 'spark', 'mesos', 'yarn', or 'k8s'.

Name	Type	Description
<code>deploy_mode</code>	(str, optional)	Deploy mode for the different master URLs. One of 'client' or 'cluster'.

deploy_mode instance-attribute [¶](#)

```
deploy_mode: str | None
```

master instance-attribute [¶](#)

```
master: str
```

from_pulse_config classmethod [¶](#)

```
from_pulse_config(config: Any) -> SparkMasterURL
```

Return the master URL compatible with Spark 2 onwards.

Previous URLs would be called local, yarn-client, yarn-cluster. However, Spark 2 introduced a "special" Spark master which is just YARN and delegates the deployment mode to Hadoop. The new options are::

```
--master yarn --deploy-mode client|cluster
```

This function translates 'yarn-client' and 'yarn-cluster' to their respective Spark 2+ configs.

Parameters:

Name	Type	Description	Default
<code>config</code> ¶	SparkConfig	Spark Config as returned by :func: get_spark_session_with_app_name or :func: get_spark_session_without_app_name .	<i>required</i>

add_kerberos_to_spark_properties [¶](#)

```
add_kerberos_to_spark_properties(pulse: Server, spark_config: dict) -> None
```

Adds kerberos configuration to `spark_config` .

Note

This function mutates the contents of `spark_config` .

get_spark_config_with_app_name [¶](#)

```
get_spark_config_with_app_name(pulse: Server, app_name: str | None, job_profile_name: str | None = None, group_name: str | None = None, clean_spark_properties: bool = True) -> Any
```

Retrieves the effective 'com.feedzai.pulse.service.datascience.config.SparkConfig' from Pulse Server to be used by an external Spark job. These configurations will depend on a given optional job profile and group model.

Parameters:

Name	Type	Description	Default
<code>pulse</code> ¶	<code>Server</code>	The ds_api.Server used to communicate with Pulse.	<i>required</i>
	str		<i>required</i>

Name	Type	Description	Default
<code>app_name</code> ↗		An optional name of a pulse application. This parameter should be used when we want to have access to application specific properties. This allows to use the same properties being used by Spark jobs executed in Pulse.	
<code>job_profile_name</code> ↗	str	The name of the job profile to reuse the Spark properties defined there. If 'None' then it will retrieve the properties associated to default job profile for jobs of type 'EXTERNAL_DS_JOB'.	None
<code>group_name</code> ↗	str	The name of the group model used to retrieve the configurations defined in its properties.	None
<code>clean_spark_properties</code> ↗	bool	True if it should clean the Spark properties specific to Pulse use case, false otherwise.	True

Returns:

Type	Description
SparkConfig	A com.feedzai.pulse.service.datascience.config.SparkConfig with Pulse's configuration.

get_spark_config_without_app_name [↗](#)

```
get_spark_config_without_app_name(pulse: Server) -> Any
```

Retrieves the effective 'com.feedzai.pulse.service.datascience.config.SparkConfig' from configurations defined in the global scope of 'com.feedzai.pulse.service.datascience.config.DataScienceConfig' and com.feedzai.pulse.service.distributed.job.config.DistributedJobServiceConfig .

The DataScienceConfig has priority over DistributedJobServiceConfig.

Parameters:

Name	Type	Description	Default
<code>pulse</code> ↗	Server	The ds_api.Server used to communicate with Pulse.	<i>required</i>

Returns:

Type	Description
unknown	A 'com.feedzai.pulse.service.datascience.config.SparkConfig' from configurations defined in the global scope of data science service configuration.

string_indexing

type_conversion [↗](#)

Type conversion helper function between Java and Python types.

JDouble module-attribute [↗](#)

```
JDouble = autoclass('java.lang.Double')
```

JLong module-attribute [↗](#)

```
JLong = autoclass('java.lang.Long')
```

JString module-attribute [↗](#)

```
JString = autoclass('java.lang.String')
```

default_transformation module-attribute [↗](#)

```
default_transformation: Optional[Callable] = None
```

list_class module-attribute [↗](#)

```
list_class = find_javaclass('java.util.List')
```

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

map_class module-attribute [↗](#)

```
map_class = find_javaclass('java.util.Map')
```

option_class module-attribute [↗](#)

```
option_class = find_javaclass(_option_javaref)
```

optional_class module-attribute [↗](#)

```
optional_class = find_javaclass('java.util.Optional')
```

pair_class module-attribute [↗](#)

```
pair_class = find_javaclass(_pair_javaref)
```

set_class module-attribute [↗](#)

```
set_class = find_javaclass('java.util.Set')
```

transformations module-attribute [↗](#)

```
transformations = {list_class: _transform_list, set_class: _transform_set, map_class: _transform_map, optional_class: _transform_optional, option_class: _transform_optional, pair_class: _transform_feedzai_pair}
```

cast_java_subclass [↗](#)

```
cast_java_subclass(java_object)
```

Helper method that casts a java class to its real subclass implementation.

For context, for example, imagine the class `Format` that has as subclasses `DateTimeFormat` and `TimestampFormat`.

When java returns an object with type `Format` and we know it is a `TimestampFormat`, we first need to cast it to this subclass so we can for example access its `getUnit()` specific method.

get_optional_or_fail [↗](#)

```
get_optional_or_fail(optional, fail_message, mapper=None) -> Any
```

Fetches the value stored inside a `com.feedzai.commons.Option`.

Otherwise, fails and logs an error message with 'fail_message'

Parameters:

Name	Type	Description	Default
<code>optional</code> ↗		The optional from which the value will be extracted.	<i>required</i>
<code>fail_message</code> ↗		Message to be logged in the optional is empty.	<i>required</i>
<code>mapper</code> ↗		Mapper function.	<code>None</code>

Returns:

Type	Description
<code>Any</code>	The value stored inside the optional. Fails if the optional is empty.

get_optional_or_none [↗](#)

```
get_optional_or_none(java_optional, mapper=None)
```

optional_char_or_else [↗](#)

```
optional_char_or_else(value, or_else)
```

Workaround utility method for `com.feedzai.commons.Option` type parameters.

Parameters:

Name	Type	Description	Default
<code>value</code> Â¶		Option	<i>required</i>
<code>or_else</code> Â¶		Default value, if the option is empty	<i>required</i>

Returns:

Type	Description
Any	Option value or 'or_else' if the option is empty

to_feedzai_option [Â¶](#)

```
to_feedzai_option(obj: Any)
```

Creates an Option.of(obj).

to_java_list [Â¶](#)

```
to_java_list(pylist: Iterable)
```

Create a java list from a python iterable.

to_java_map [Â¶](#)

```
to_java_map(py_dict: Mapping)
```

Creates a java HashMap from a python dict.

Parameters:

Name	Type	Description	Default
<code>py_dict</code> Â¶	Mapping	the dictionary to be converted into an HashMap	<i>required</i>

Returns:

Type	Description
HashMap	A Java HashMap

to_java_set [Â¶](#)

```
to_java_set(pylist: Iterable)
```

Create a java set from a python iterable.

to_python_list [Â¶](#)

```
to_python_list(java_list, mapper=None) -> List
```

Create a python list from a java list.

to_python_map [↗](#)

```
to_python_map(java_map, key_mapper=None, value_mapper=None) -> Dict
```

Create a python map from a java map.

to_python_set [↗](#)

```
to_python_set(java_set, mapper=None) -> Set
```

Create a python set from a java set.

to_serializable [↗](#)

```
to_serializable(obj: Any) -> Any
```

Convert int, float, str to their serializable Java version:

- `int` -> `java.lang.Long`
- `float` -> `java.lang.Double`
- `str` -> `java.lang.String`
- `None` -> `None`

transform_java_object [↗](#)

```
transform_java_object(java\_object)
```

Transforms a java object into the correspondent python object.

1. If the provided object is already a python object, then it will simply be returned without any change. A java object is defined as an object that implements `jnius.JavaClass`.
2. If the provided object matches any of the specific transformations in `transformations`, then the correspondent custom transformation is applied.
3. If none of the above rules apply, then the object will be wrapped in a `JavaWrapper` object.

Parameters:

Name	Type	Description	Default
<code>java_object</code> ↗		The java object to be transformed.	<i>required</i>

Returns:

Type	Description
<code>Any</code>	The correspondent python object.

types

TIME_UNITS module-attribute

```
TIME_UNITS = {(toString()) for value in (values())}
```

TimeUnit module-attribute

```
TimeUnit = autoclass('com.feedzai.commons.util.time.TimeUnit')
```

CategoricalField

```
CategoricalField(possible\_values: Optional[Set[str]] = None)
```

Bases: [DataScienceField](#)

Representation of a Pulse categorical field.

Parameters:

Name	Type	Description	Default
possible_values 	Set of str	Possible value for the categorical field.	None

[possible_values](#) instance-attribute

```
possible\_values = possible\_values
```

[subclasses](#) class-attribute [instance-attribute](#)

```
subclasses: List[JavaWrapper] = []
```

[auto_cast](#) staticmethod

```
auto\_cast(java_object) -> CategoricalField
```

Helper method called when a return obj comes from java to be auto converted to python.

ComplexField

```
ComplexField()
```

Bases: [DataScienceField](#)

Representation of a Pulse ComplexField

[subclasses](#) class-attribute [instance-attribute](#)

```
subclasses: List[JavaWrapper] = []
```

[auto_cast](#) staticmethod

```
auto\_cast(_) -> ComplexField
```

Helper method called when a return obj comes from java to be auto converted to python.

DataScienceField [↗](#)

```
DataScienceField(data\_science\_field)
```

Bases: [JavaWrapper](#)

Base class for every type.

Default constructor.

Parameters:

Name	Type	Description	Default
data_science_field ↗		Pulse's DataScienceField class.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

DateTimeField [↗](#)

```
DateTimeField(time\_format: Format)
```

Bases: [DataScienceField](#)

Representation of a Pulse DateTimeField.

Parameters:

Name	Type	Description	Default
time_format ↗	Format	The time format for the date time field.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

time_format **instance-attribute** [↗](#)

```
time_format = time\_format
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> DateTimeField
```

Helper method called when a return obj comes from java to be auto converted to python.

DateTimeFormat [↗](#)

```
DateTimeFormat(time\_format: str)
```

Bases: [Format](#)

Representation of a date time format (equivalent to Pulse's DateTimeFormat).

Parameters:

Name	Type	Description	Default
<code>time_format</code> ¶	<code>str</code>	The date time pattern associated with this format.	<i>required</i>

subclasses `class-attribute` `instance-attribute` [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast `staticmethod` [¶](#)

```
auto_cast(java_object) -> DateFormat
```

Helper method called when a return obj comes from java to be auto converted to python.

validate [¶](#)

```
validate()
```

Format [¶](#)

```
Format(time\_format: JavaClass)
```

Bases: `JavaWrapper`

Base class for every format.

Parameters:

Name	Type	Description	Default
<code>time_format</code> ¶	<code>JavaClass</code>	The time format for the date time field.	<i>required</i>

subclasses `class-attribute` `instance-attribute` [¶](#)

```
subclasses: List[JavaWrapper] = []
```

NativeType [¶](#)

Bases: `Enum`

Supported Native Types in the creation of schema fields

BOOLEAN `class-attribute` `instance-attribute` [¶](#)

```
BOOLEAN = 'BOOLEAN'
```

DOUBLE `class-attribute` `instance-attribute` [¶](#)

```
DOUBLE = 'DOUBLE'
```

FLOAT `class-attribute` `instance-attribute` [¶](#)

```
FLOAT = 'FLOAT'
```

INT `class-attribute` `instance-attribute` [¶](#)

```
INT = 'INT'
```

LONG [class-attribute](#) [instance-attribute](#) [¶](#)

```
LONG = 'LONG'
```

STRING [class-attribute](#) [instance-attribute](#) [¶](#)

```
STRING = 'STRING'
```

UTF8STRING [class-attribute](#) [instance-attribute](#) [¶](#)

```
UTF8STRING = 'UTF8STRING'
```

NumericalField [¶](#)

```
NumericalField()
```

Bases: [DataScienceField](#)

Representation of a Pulse NumericalField

subclasses [class-attribute](#) [instance-attribute](#) [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast [staticmethod](#) [¶](#)

```
auto_cast(_) -> NumericalField
```

Helper method called when a return obj comes from java to be auto converted to python.

StringField [¶](#)

```
StringField()
```

Bases: [DataScienceField](#)

Representation of a Pulse StringField

subclasses [class-attribute](#) [instance-attribute](#) [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast [staticmethod](#) [¶](#)

```
auto_cast(_) -> StringField
```

Helper method called when a return obj comes from java to be auto converted to python.

TimeModel [¶](#)

```
TimeModel(value: int, unit: TimeUnit)
```

Bases: [JavaWrapper](#)

Representation of a Pulse TimeModel to construct a time window.

Parameters:

Name	Type	Description	Default
value ↗	int	Value of the time window.	<i>required</i>
unit ↗	TimeUnit	Enum object to define the window, possible values: MILLISECOND SECOND MINUTE HOUR DAY WEEK	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> TimeModel
```

Helper method called when a return obj comes from java to be auto converted to python.

TimestampFormat [↗](#)

```
TimestampFormat(unit: str)
```

Bases: [Format](#)

Representation of a timestamp format (equivalent to Pulse's TimestampFormat).

Parameters:

Name	Type	Description	Default
unit ↗	str	The time format identifier. The domain values are (the java.concurrent.TimeUnit values): NANoseconds, MICROseconds, MILLIseconds, SECONDS, MINUTES, HOURS, DAYS.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> TimestampFormat
```

Helper method called when a return obj comes from java to be auto converted to python.

get_dtype [↗](#)

```
get_dtype(ds_field, pyspark_context: bool = False) -> DataScienceField
```

Get the python representation of a DataScienceField from a Pulse field.

Parameters:

Name	Type	Description	Default
ds_field ↗			<i>required</i>

Name	Type	Description	Default
<code>pyspark_context</code> 	boolean	Indicates whether this is a pyspark context or a Pulse context. In a pyspark context the type of time fields will be given as timestamp in seconds (this is the default unit after the pulse_to_pyspark transformation), otherwise the actual type from Pulse will be returned.	False

Returns:

Type	Description
<code>DataScienceField</code>	The corresponding python DataScienceField representation.

`get_time_format`

```
get_time_format(ds_field) -> Format
```

Get the python TimeFormat representation from a Pulse field.

Parameters:

Name	Type	Description	Default
<code>ds_field</code> 		A Pulse DataScienceField corresponding to a DateTimeField.	<i>required</i>

Returns:

Type	Description
<code>Format</code>	The associated time Format.

utils

Helper utilities.

DEFAULT_SPARK_APP_NAME module-attribute

```
DEFAULT_SPARK_APP_NAME = 'pySparkJob-created-by-ds_api'
```

logger module-attribute

```
logger = getLogger(__name__)
```

ResourceSearchTypeEnum

Bases: `str`, `Enum`

Procedures to search for given resource within a parent resource.

DESC class-attribute instance-attribute

```
DESC = 'desc'
```

ID class-attribute instance-attribute

```
ID = 'id'
```

OBJECT class-attribute instance-attribute

```
OBJECT = 'object'
```

camel_to_snake

```
camel_to_snake(string: str) -> str
```

Convert camel case to snake case.

Excludes strings starting with underscore, or in upper case eg: ["createObj", "create_obj", "**dict**", "CONSTANT"] = ["create_obj", 'create_obj', '**dict**', 'CONSTANT']

Parameters:

Name	Type	Description	Default
<code>string</code> 	<code>str</code>	input string	<i>required</i>

Returns:

Type	Description
<code>str</code>	snake case string

create_spark_app_name [↗](#)

```
create_spark_app_name() -> str
```

Creates an application name for spark.

The created name will take in consideration the following aspects:

1. If the env variable `ds_api.USER_NAME_ENV` is defined, then it will be used as part of the created name. This environment variable is assumed to contain the name of the user that spawned the current process.
2. If the env variable in 1) is not defined, then the env variable `ds_api.USER_ID_ENV` will be used (if defined) as part of the created name. This environment variable is assumed to contain the identifier of the user that spawned the current process.
3. If the env variable in 2) is not defined, then this function calls `os.getpid`, and uses such pid in the created application name.

Returns:

Type	Description
<code>str</code>	A spark application name.

get_optional_or_fail [↗](#)

```
get_optional_or_fail(optional, fail_message, mapper=None)
```

get_optional_or_none [↗](#)

```
get_optional_or_none(java_optional, mapper=None)
```

optional_char_or_else [↗](#)

```
optional_char_or_else(value, or_else)
```

to_java_list [↗](#)

```
to_java_list(pylist)
```

to_java_map [↗](#)

```
to_java_map(pymap)
```

to_java_set [↗](#)

```
to_java_set(pylist)
```

to_python_list [↗](#)

```
to_python_list(java_list, mapper=None)
```

to_python_map [↗](#)

```
to_python_map(java_map, key_mapper=None, value_mapper=None)
```

to_python_set [¶](#)

```
to_python_set(java_set, mapper=None)
```

transform_java_object [¶](#)

```
transform_java_object(java_object)
```

config_factory

logger module-attribute

```
logger = getLogger(__name__)
```

ConfigFactory

```
ConfigFactory(pyjnius_java)
```

Bases: [JavaWrapper](#)

A DataSource Configuration Factory representation.

configs_dict class-attribute instance-attribute

```
configs_dict = {CSV: partial(_set_csv_conf, config_ref=createCsvConf), PARQUET: partial(_set_parquet_conf, config_ref=createParquetConf), INFER: partial(_set_infer_schema_conf, config_ref=createInferSchemaConf)}
```

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

get_config staticmethod

```
get_config(config: ConfigType, **kwargs: Union[bool, str, int, Dict[str, str], Set[str]]) -> JavaWrapper
```

Getter for Pulse datasource configuration.

Parameters:

Name	Type	Description	Default
<code>config</code>	<code>str</code>	The desired Pulse configuration. Options are: <code>csv</code> , <code>parquet</code> , <code>infer</code> .	<i>required</i>
<code>kwargs</code>	<code>Dict</code>	Keyword arguments used for the configuration setup.	<code>{}</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Defaults to ','.
<code>missing_values</code>	<code>set of str</code>	Set of strings with values for empty value. Defaults to { "", "NA", "NULL" }.
<code>quote</code>	<code>str</code>	The character used to denote the start and end of a quoted item. Defaults to ""

Name	Type	Description
<code>line_separator</code>	<code>str</code>	The string used mark the end of line. Defaults to '\n'.
<code>encoding</code>	<code>str</code>	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'.
<code>cat_threshold</code>	<code>int</code>	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
<code>job_properties</code>	<code>Dict</code>	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified.
<code>n_records</code>	<code>int</code>	The number of records that will be read per partition when performing datasource schema inference. Defaults to 15000. Will only be used if no Schema is specified.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified.
<code>merge_schemas</code>	<code>bool</code>	Specify if schema merging should be allowed. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
JavaWrapper	Returns the requested datasource configuration depending on the config passed.

ConfigType

Bases: `str`, `Enum`

CSV `class-attribute` `instance-attribute` 

CSV = 'csv'

INFER `class-attribute` `instance-attribute` 

INFER = 'infer'

PARQUET `class-attribute` `instance-attribute` 

PARQUET = 'parquet'

datasource_adapter [↗](#)

DefaultParameters module-attribute [↗](#)

```
DefaultParameters = namedtuple('DefaultParameters', ('has_header', 'sep', 'quote', 'line_separat  
or', 'encoding', 'missing_values', 'merge_schemas', 'n_records', 'cat_threshold', 'dtypes'), def  
aults=(True, ',', '"', '\n', name(), {'', 'NA', 'NULL'}, False, DEFAULT_NUMBER_OF_RECORDS, DEFAU  
LT_MAX_CATEGORIES, {}))
```

MS module-attribute [↗](#)

```
MS = 1000
```

The number of milliseconds in one second. Used to convert to/from spark timestamps.

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

SparkDatasourceAdapter [↗](#)

```
SparkDatasourceAdapter(spark\_session)
```

Bases: `object`

A class that follows the adapter pattern. This class is responsible for:

- adapting pulse data sources into spark dataframes, through `from_pulse_datasource`
- adapting spark dataframes into pulse data sources, through `to_pulse_datasource`

Parameters:

Name	Type	Description	Default
<code>spark_session</code> ↗	<code>SparkSession</code>	Session to which the adapter is bound to.	<i>required</i>

spark_session instance-attribute [↗](#)

```
spark_session = spark\_session
```

from_pulse_datasource [↗](#)

```
from_pulse_datasource(app, name)
```

Adapts a pulse data source into a pyspark dataframe.

Parameters:

Name	Type	Description	Default
<code>app</code> ↗	<code>App</code>	The app in which the data source exists.	<i>required</i>
	<code>str</code>		<i>required</i>

Name	Type	Description	Default
<code>name</code> ↗		The Data Source name as it appears in the UI.	

Returns:

Type	Description
<code>DataFrame</code>	DataFrame equivalent to the referenced pulse data source.

`to_pulse_datasource` [↗](#)

```
to_pulse_datasource(dataframe: _TDataFrame, app: App, name: str, path: str, file_format: str = 'csv', schema: Optional[Schema] = None, **kwargs: Union[bool, str, int, Dict[str, Any], Set[str]])
```

Adapts a pyspark dataframe into a pulse datasource. The dataframe will be written in the path specified before being added to Pulse. DateType and TimestampType fields are written as unix time (in milliseconds). The schema will be automatically inferred using `AbstractDataSource#inferSchema` approach if the Schema is not specified.

All pySpark types but DecimalType and MLlib types are supported.

Parameters:

Name	Type	Description	Default
<code>dataframe</code> ↗	<code>DataFrame</code>	DataFrame which will be used to create a Pulse datasource.	<i>required</i>
<code>app</code> ↗	<code>App</code>	The App where the datasource will be created.	<i>required</i>
<code>name</code> ↗	<code>str</code>	The name of the datasource.	<i>required</i>
<code>path</code> ↗	<code>str</code>	The path of the file containing the dataset.	<i>required</i>
<code>file_format</code> ↗	<code>str</code>	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	<code>'csv'</code>
<code>schema</code> ↗	<code>Schema</code>	The Pulse Schema to use. Optional. If not specified, the Schema will be inferred by Pulse. Only used for csv files.	<code>None</code>
<code>kwargs</code> ↗	<code>Dict</code>	Keyword arguments used for the configuration of the DataSource and for the SchemaInference job.	<code>{}</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Only used for csv files. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Only used for csv files. Defaults to <code>','</code> .

Name	Type	Description
missing_values	set of str	Set of strings with values for empty value. Only used for csv files. Defaults to {"", "NA", "NULL"}.
quote	str	The character used to denote the start and end of a quoted item. Only used for csv files. Defaults to "".
line_separator	str	The string used mark the end of line. Only used for csv files. Defaults to "\n".
encoding	str	Encoding to use for UTF when reading/writing. Only used for csv files. Defaults to 'UTF-8'.
cat_threshold	int	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
job_properties	Dict	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse PropertiesProfile.
n_records	int	The number of records that will be read per partition when performing datasource schema inference. Will only be used if no Schema is specified. Defaults to 15000.
dtypes	Dict	The dictionary with the field overriding information when inferring the datasource schema. Will only be used for csv files and if no Schema is specified. Defaults to None.
merge_schemas	bool	Specify if schema merging should be allowed. Only used for parquet files. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
DataSource	The new DataSource instance.

datasource_transformations [↗](#)

Module that handles datasource/feature related transformations/conversion.

DEFAULT_SPARK_APP_NAME module-attribute [↗](#)

```
DEFAULT_SPARK_APP_NAME = 'pySparkJob-created-by-ds_api'
```

FINAL_TIMESTAMP module-attribute [↗](#)

```
FINAL_TIMESTAMP = datetime(2200, 1, 1)
```

INITIAL_TIMESTAMP module-attribute [↗](#)

```
INITIAL_TIMESTAMP = datetime(1970, 1, 1)
```

TagUtils module-attribute [↗](#)

```
TagUtils = autoclass(_AUTOML_TRANSFORMATIONS_CLASSPATH + 'TagUtils')
```

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

CompositeField dataclass [↗](#)

```
CompositeField(desc: str, tags: Sequence[str], fields: Sequence[str])
```

Bases: [SemanticField](#)

Defines a composite field to use in the semantics construction.

Please check the `available_tags` property to get the list of supported tags.

TAGS_MAP class-attribute instance-attribute [↗](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [↗](#)

```
available_tags: List[str]
```

Returns the list of available tags.

desc instance-attribute [↗](#)

```
desc: str
```


fields instance-attribute [¶](#)

```
fields: Sequence[str]
```

tags instance-attribute [¶](#)

```
tags: Sequence[str]
```

to_pulse [¶](#)

```
to_pulse(tagged_fields: Sequence)
```

Turns our CompositeField into its pulse representation.

Parameters:

Name	Type	Description	Default
<code>tagged_fields</code> ¶	sequence of converted TaggedField	Set of possible simple tagged fields that can be used to compose the composite field.	<i>required</i>

Returns:

Type	Description
A pulse representation of the composite field.	

SemanticField dataclass [¶](#)

```
SemanticField()
```

Superclass for TaggedField and CompositeField

TAGS_MAP class-attribute instance-attribute [¶](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [¶](#)

```
available_tags: List[str]
```

Returns the list of available tags.

Semantics [¶](#)

Utility class to convert to/from a json semantic file.

from_semantic_file staticmethod [¶](#)

```
from_semantic_file(filename: str, path: Optional[str] = None) -> Tuple[SemanticField, ...]
```

Takes a semantic file and converts it into a usable collection of SemanticField.

Parameters:

Name	Type	Description	Default
<code>filename</code> ¶	<code>str</code>	Filename of the semantic file.	<i>required</i>
<code>path</code> ¶	<code>str</code>	Path where the file is to be saved, default is the working directory	<code>None</code>

Returns:

Type	Description
<code>tuple of SemanticField</code>	A collection of SemanticField.

`to_semantic_file` `staticmethod` [¶](#)

```
to_semantic_file(semantic: Sequence[SemanticField], filename: str = 'semantic_file.json', path: Optional[str] = None)
```

Takes a collection of semantics and writes a json semantic file.

Parameters:

Name	Type	Description	Default
<code>semantic</code> ¶	<code>sequence of SemanticField</code>	Semantics to write.	<i>required</i>
<code>filename</code> ¶	<code>str</code>	Name for the semantic file, default is "semantic_file.json"	<code>'semantic_file.json'</code>
<code>path</code> ¶	<code>str</code>	Path where the file is to be saved, default is the working directory	<code>None</code>

`Split` `dataclass` [¶](#)

```
Split(name: str, start_timestamp: Optional[Union[int, datetime]] = None, end_timestamp: Optional[Union[int, datetime]] = None, path: Optional[str] = None)
```

Defines a datasource split.

Attributes:

Name	Type	Description
<code>name</code>	<code>str</code>	Name of the datasource created by the split
<code>start_time_stamp</code>	<code>(optional, int or datetime)</code>	Timestamp starting the splitting interval. If not specified, the beginning of the dataset will be included and end_timestamp must be specified.
<code>end_timestamp</code>	<code>(optional, int or datetime)</code>	Timestamp ending the splitting interval. If not specified, the end of the dataset will be included and start_timestamp must be specified.
<code>path</code>	<code>(optional, str)</code>	The writing path for the split.

end_timestamp class-attribute instance-attribute [¶](#)

```
end_timestamp: Optional[Union[int, datetime]] = None
```

name instance-attribute [¶](#)

```
name: str
```

path class-attribute instance-attribute [¶](#)

```
path: Optional[str] = None
```

start_timestamp class-attribute instance-attribute [¶](#)

```
start_timestamp: Optional[Union[int, datetime]] = None
```

to_pulse [¶](#)

```
to_pulse()
```

TaggedField dataclass [¶](#)

```
TaggedField(desc: str, tags: Sequence[str])
```

Bases: [SemanticField](#)

Defines a tagged field to use in the semantics construction.

Please check the `available_tags` property to get the list of supported tags.

TAGS_MAP class-attribute instance-attribute [¶](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [¶](#)

```
available_tags: List[str]
```

Returns the list of available tags.

desc instance-attribute [¶](#)

```
desc: str
```

tags instance-attribute [¶](#)

```
tags: Sequence[str]
```

to_pulse [¶](#)

```
to_pulse()
```

constants

A set of useful numerical, and categorical constants.

HyperLogLog constants

We use 2^{12} registers for the HyperLogLog algorithm, equivalent to a cardinality error of 1.625%. The relative error can be estimated from

$$\frac{1.04}{\sqrt{m}},$$

with $m = 2^k$ the number of registers.

Attributes:

Name	Type	Description
<code>HLL_K</code>	int	Number of registries for HyperLogLog.
<code>HLL_ERROR_RATIO</code>	float	Cardinality relative error.

AGGREGATION_MAPPER module-attribute

```
AGGREGATION_MAPPER = {'min': BIN_MIN_COL, 'max': BIN_MAX_COL, 'count': COUNT_COL, 'avg': MEAN_COL, 'sum': SUM_COL}
```

BD_COL module-attribute

```
BD_COL = 'breakdowns'
```

BIN_MAX_COL module-attribute

```
BIN_MAX_COL = 'fz$bmax'
```

BIN_MIN_COL module-attribute

```
BIN_MIN_COL = 'fz$bmin'
```

BIN_RANGE_COL module-attribute

```
BIN_RANGE_COL = 'fz$bin_range'
```

CAT_COL module-attribute

```
CAT_COL = 'fz$category'
```

COUNT_COL module-attribute

```
COUNT_COL = 'fz$counts'
```

FIELD_TYPE_KEY module-attribute [Â¶](#)

```
FIELD_TYPE_KEY = 'field_type'
```

FIRST_QTL_COL module-attribute [Â¶](#)

```
FIRST_QTL_COL = '25%'
```

FR_COL module-attribute [Â¶](#)

```
FR_COL = 'fraud_rate'
```

HISTOGRAM_KEY module-attribute [Â¶](#)

```
HISTOGRAM_KEY = 'histogram'
```

HLL_ERROR_RATIO module-attribute [Â¶](#)

```
HLL_ERROR_RATIO = 1.04 / sqrt(2 ** HLL_K)
```

HLL_K module-attribute [Â¶](#)

```
HLL_K = 12
```

MAX_COL module-attribute [Â¶](#)

```
MAX_COL = 'max'
```

MEAN_COL module-attribute [Â¶](#)

```
MEAN_COL = 'mean'
```

MEDIAN_COL module-attribute [Â¶](#)

```
MEDIAN_COL = 'median'
```

MIN_COL module-attribute [Â¶](#)

```
MIN_COL = 'min'
```

MISSING_COLOR module-attribute [Â¶](#)

```
MISSING_COLOR = '#990000'
```

MISSING_VAL module-attribute [Â¶](#)

```
MISSING_VAL = '(missing)'
```

NEG_COL module-attribute [Â¶](#)

```
NEG_COL = 'negatives'
```

OTHER_COLOR module-attribute [Â¶](#)

```
OTHER_COLOR = '#999'
```

OTHER_VAL module-attribute [Â¶](#)

```
OTHER_VAL = '(other)'
```

POS_COL module-attribute [Â¶](#)

```
POS_COL = 'positives'
```

SECONDARY_COLOR module-attribute [Â¶](#)

```
SECONDARY_COLOR = '#ff7f0e'
```

SEMANTIC_TAG_KEY module-attribute [Â¶](#)

```
SEMANTIC_TAG_KEY = 'semantic_tag'
```

STATS_KEY module-attribute [Â¶](#)

```
STATS_KEY = 'stats'
```

SUM_COL module-attribute [Â¶](#)

```
SUM_COL = 'sum'
```

SUPPORTED_AGGREGATIONS module-attribute [Â¶](#)

```
SUPPORTED_AGGREGATIONS = list(keys())
```

TERTIARY_COLOR module-attribute [Â¶](#)

```
TERTIARY_COLOR = '#2ca02c'
```

THIRD_QTL_COL module-attribute [Â¶](#)

```
THIRD_QTL_COL = '75%'
```

TIME_COL module-attribute [Â¶](#)

```
TIME_COL = 'fz$time'
```

VALIDS_KEY module-attribute [Â¶](#)

```
VALIDS_KEY = 'validations'
```

exploration_results

EXPLORATION_OBJECT_VERSION

module-attribute

```
EXPLORATION_OBJECT_VERSION = 'v0.0'
```

LOGGER

module-attribute

```
LOGGER = getLogger(__name__)
```

SAMPLE_SIZE

module-attribute

```
SAMPLE_SIZE = 500
```

Sample size.

ExplorationResults

```
ExplorationResults(results_dict: dict, max_bins: int, time_resolution: str, breakdowns: list, n_rows: int, timestamp_field: str | None = None, cost_field: str | None = None, target=(None, None), data: DataFrame | _TDataFrame | None = None, sample_size: int | None = 100000, report_html: str | None = None)
```

Bases: object

Results of the exploration compute_stats.

Parameters:

Name	Type	Description	Default
results_dict	dict	dictionary-like results obtained from exploration.stats.compute_stats().	required
max_bins	int	binning resolution used for the histogram computation of numerical fields. In practice, this value defines the maximum number of bins that can be used to draw histograms for numerical fields.	required
time_resolution	str	time resolution used for the histogram computation of datetime fields.	required
breakdowns	list	a list of field names that were used to calculate breakdowns over.	required

Name	Type	Description	Default
<code>n_rows</code> API	int	number of rows of the explored dataset.	<i>required</i>
<code>times_tamp_field</code> API	str None	name of the timestamp field. Default: None.	None
<code>cost_field</code> API	str None	name of the cost field. Default: None.	None
<code>target</code> API		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. Default: None.	(None, None)
<code>data</code> API	DataFrame TDataFrame None	Pandas or PySpark DataFrame. A small sample (500 rows) will be stored as a Pandas DataFrame. If a target exists then the sample will be balanced with respect to the target value. This is useful for the sample section of the html report and to create scatter plots. Depending on the specification of parameter <code>sample_size</code> , this data will be used as well to compute the time dependencies. Default: None.	None
<code>sample_size</code> API	int None	number of instances taken from <code>data</code> to use in the computation of time dependencies. If -1 is given then all instances will be used. If <code>None</code> then time dependencies won't be computed. Default: 100k.	100000
<code>report_html</code> API	str None	an existing html report in string format. If None is passed it will generate it. It's mainly useful when creating an ExplorationResults from serialized results using the 'load' method. Default: None.	None

Attributes:

Name	Type	Description
<code>field_names</code>		List of field names.
<code>constant_fields</code>		List of constant fields.
<code>n_unhealthy</code>		Number of unhealthy values (missing, nan, infs, empty) summed over all fields.
<code>target_rate</code>		Rate of positives.

breakdowns `instance-attribute` [API](#)

breakdowns = `breakdowns`

constant_fields instance-attribute [¶](#)

```
constant_fields = list((name) for field in (values()) if is_constant)
```

cost_field instance-attribute [¶](#)

```
cost_field = cost_field
```

field_names instance-attribute [¶](#)

```
field_names = list(keys())
```

field_results property writable [¶](#)

```
field_results: Dict[str, FieldResults]
```

max_bins instance-attribute [¶](#)

```
max_bins = max_bins
```

n_fields instance-attribute [¶](#)

```
n_fields = len(results_dict)
```

n_rows instance-attribute [¶](#)

```
n_rows = n_rows
```

n_unhealthy instance-attribute [¶](#)

```
n_unhealthy = sum((n_unhealthy) for field in (values()))
```

report property writable [¶](#)

```
report
```

sample_df property writable [¶](#)

```
sample_df
```

target_rate instance-attribute [¶](#)

```
target_rate = compute_target_rate()
```

time_dependencies instance-attribute [¶](#)

```
time_dependencies = compute_time_dependencies(drop(*(constant_fields)), timestamp_field, sample_size)
```

time_resolution instance-attribute [¶](#)

```
time_resolution = time_resolution
```

timestamp_field instance-attribute [¶](#)

```
timestamp_field = timestamp_field
```

compute_target_rate [¶](#)

```
compute_target_rate()
```

Method to retrieve the target rate if a target is specified.

generate_report [¶](#)

```
generate_report(show_all_checks=False, embed_dependencies=False)
```

Generates an HTML report of the exploration results.

Parameters:

Name	Type	Description	Default
<code>show_all_checks</code> ¶		Boolean value indicating whether all checks that were run should be displayed in the report, even if they triggered no warnings. Default is False.	False
<code>embed_dependencies</code> ¶		Boolean value indicating if the report html file should include all css and js dependencies (bootstrap, jquery, vega). Embedding them increases significantly the size of the file, but in security-closed environments where there is no internet access, the dependencies might need to be embedded in order to be able to render the report. Default is False.	False

Returns:

Type	Description
unknown	Overview report of the data in HTML format.

load_classmethod [¶](#)

```
load(file_name)
```

Loads results object from given pickle file.

Parameters:

Name	Type	Description	Default
<code>file_name</code> ¶		name of file to deserialize results from.	<i>required</i>

Returns:

Type	Description
<code>ExplorationResults</code>	an ExplorationResults instance.

save [¶](#)

```
save(file_name='exploration_results.pkl', overwrite=False)
```

Saves results object into given pickle file.

Parameters:

Name	Type	Description	Default
<code>file_name</code> ¶		name of file to serialize results into.	'exploration_results.pkl'

Name	Type	Description	Default
<code>overwrite</code> ¶		bool indicating whether to overwrite the file if it already exists. Default: False.	<code>False</code>

`save_report_html` [¶](#)

```
save_report_html(file_name='report.html', overwrite=False)
```

Saves the HTML report.

Parameters:

Name	Type	Description	Default
<code>file_name</code> ¶		name of the file to save the report into.	<code>'report.html'</code>
<code>overwrite</code> ¶		bool indicating whether to overwrite the file if it already exists. Default: False.	<code>False</code>

feature_stats

collect_value_counts

```
collect_value_counts(data: _TDataFrame, string_fields: list[str], dist_sampling: float | None) -> dict[str, dict[str, int]]
```

Compute sampled value counts for string fields via groupBy.
It identifies the frequency of unique values within specified string columns.

Parameters:

Name	Type	Description	Default
data	DataFrame	The source DataFrame containing string columns to analyze.	required
string_fields	list of str	The names of the columns for which to compute value distributions.	required
dist_sampling	float	The sampling fraction (0.0 to 1.0). If None or 0.0, the function returns an empty dictionary immediately.	required

Returns:

Type	Description
dict of (str, dict of (str, int))	A mapping where each key is a field name, and the value is a dictionary containing unique strings and their respective occurrence counts.

datetime_stat_exprs

```
datetime_stat_exprs(field_name: str) -> list[tuple[str, str, Column]]
```

Generate Spark aggregate expressions that compute statistics for a given datetime column.

Parameters:

Name	Type	Description	Default
field_name	str	The name of the datetime column to analyze. The name is automatically escaped using backticks for SQL safety.	required

Returns:

Type	Description
list of tuple of (str, str, pyspark.sql.Column)	A list of triples containing (field_name, stat_description, spark_expression). The statistics included are: - count: Total number of non-null datetime entries. - missing: Count of NULL values in the column. - min: The earliest (minimum) timestamp in the dataset. - max: The latest (maximum) timestamp in the dataset.

get_expected_stats_set [↗](#)

```
get_expected_stats_set(field_type)
```

Returns the set of stats retrieved by compute_stats for a given field type.

Parameters:

Name	Type	Description	Default
field_type ↗		type of field.	<i>required</i>

Returns:

Type	Description
unknown	set of expected computed stats.

numerical_stat_exprs [↗](#)

```
numerical_stat_exprs(field_name: str) -> list[tuple[str, str, Column]]
```

Generate Spark aggregate expressions that compute statistics for a given numerical column.

Parameters:

Name	Type	Description	Default
field_name ↗	str	The name of the numerical column to analyze. The name is automatically escaped using backticks for SQL safety.	<i>required</i>

Returns:

Type	Description
list of tuple of (str, str, pyspark.sql.Column)	A list of triples containing (field_name, stat_description, spark_expression). The statistics included are: - sum: Sum of all finite values. - min: Minimum finite value. - max: Maximum finite value. - nan: Count of NaN values. - zeros: Count of values equal to 0. - infinities: Count of positive and negative infinity values. - missing: Count of null values. - count: Count of finite values. - mean: Arithmetic mean of finite values. - var: Sample variance of finite values.

string_stat_exprs [↗](#)

```
string_stat_exprs(field_name: str) -> list[tuple[str, str, Column]]
```

Generate Spark aggregate expressions that compute statistics for a given string column.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ¶	str	The name of the string column to analyze. The name is automatically escaped using backticks for SQL safety.	<i>required</i>

Returns:

Type	Description
list of tuple of (str, str, pyspark.sql.Column)	A list of triples containing (field_name, stat_description, spark_expression). The statistics included are: - count: Count of values that are neither NULL nor empty strings. - empty: Count of empty string ("") values. - missing: Count of NULL values. - hll_cardinality: An approximate count of distinct values using the HyperLogLog++ algorithm for memory efficiency.

validation_exprs [¶](#)

```
validation_exprs(field_name: str, validations: list[Validation]) -> list[tuple[str, str, Column]]
```

Build Spark aggregate expressions for custom validations on a field..

Iterates over a provided set of validations and generates a SQL expression that encompasses all provided validations.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ¶	str	The name of the column to be validated. The name is automatically escaped using backticks for SQL safety.	<i>required</i>
<code>validations</code> ¶	list of Validation	A list of Validation objects, each containing a description and a 'sql_builder' method that accepts a Column and returns a Spark aggregate expression.	<i>required</i>

Returns:

Type	Description
list of tuple of (str, str, pyspark.sql.Column)	A list of triples containing (field_name, validation_description, spark_expression). These are typically collected into a single data.agg() call alongside standard statistics.

field_results

LOGGER module-attribute

LOGGER = getLogger(__name__)

DatetimeFieldResults

DatetimeFieldResults([field_name](#), [field_results](#), exploration_results)

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a time field.

Parameters:

Name	Type	Description	Default
field_name		field name.	<i>required</i>
field_results		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	<i>required</i>

constant_value instance-attribute

constant_value = _constant_value()

count instance-attribute

count = _stats['count']

field_type instance-attribute

field_type = [field_results](#)[[FIELD_TYPE_KEY](#)]

histogram instance-attribute

histogram = _histogram_df()

histogram_computed instance-attribute

histogram_computed = [HISTOGRAM_KEY](#) in [field_results](#)

histogram_reduced instance-attribute

histogram_reduced = [reduce_histogram](#)(histogram, [result_type](#)=field_type) if histogram_computed else None

is_constant instance-attribute

is_constant = _is_constant()

max instance-attribute [Alt](#)

```
max = _stats['max']
```

min instance-attribute [Alt](#)

```
min = _stats['min']
```

n_missing instance-attribute [Alt](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [Alt](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [Alt](#)

```
name = field_name
```

semantic_tag instance-attribute [Alt](#)

```
semantic_tag = get(SEMANTIC_TAG_KEY)
```

stats instance-attribute [Alt](#)

```
stats = Series(_stats)
```

validations instance-attribute [Alt](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALID_KEY in field_results else None
```

plot [Alt](#)

```
plot(time_unit='yearmonthdate', time_type='T', breakdown_by=None, logy=False, interactive=None, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
time_unit Alt		a time unit supported by Vega-Lite. Recommended: 'yearmonthdate' or 'yearmonthdatehours'. For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'
time_type Alt		Vega Data Type for time variable (default: 'T'). Recommended 'T' (= 'temporal') for high granularity time resolution, and 'O' (= 'ordinal') for low cardinality time_unit like 'month'. For more details: https://vega.github.io/vega-lite/docs/type.html	'T'
breakdown_by Alt		str specifying field to breakdown histograms by. Default is None: no breakdown.	None
logy Alt		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
			None

Name	Type	Description	Default
<code>interactive</code> ¶		bool (or None) indicating whether to call the <code>interactive()</code> function on the Altair chart (default: None = False for ordinal <code>time_type</code> otherwise True).	
<code>mini</code> ¶		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

`plot_calendar` [¶](#)

```
plot_calendar(xaxis='date', yaxis='yearmonth', color='fr', size='freq', color_scheme=None)
```

Altair chart with calendar-like plot of the field.

Parameters:

Name	Type	Description	Default
<code>xaxis</code> ¶		time unit of the xaxis. Default: 'hours'. Recommended combinations of units are (xaxis='date', yaxis='yearmonth') and (xaxis='hours', yaxis='yearmonthdate')	'date'
<code>yaxis</code> ¶		time unit of the yaxis. Default: 'yearmonthdate'.	'yearmonth'
<code>color</code> ¶		color encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'fr'
<code>size</code> ¶		size encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'freq'
<code>color_scheme</code> ¶		Vega color scheme to be used. Available schemes can be found here: https://vega.github.io/vega/docs/schemes/ . Default is None, meaning 'blues' for 'freq' and diverging 'blueorange' centered at the overall fraud rate for 'fr'.	None

Returns:

Type	Description
unknown	an Altair chart.

`plot_fraud_rate` [¶](#)

```
plot_fraud_rate(target=(None, None), time_unit='yearmonthdate', time_type='T', logy=False, interactive=None)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> Altair		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>time_unit</code> Altair		a time unit supported by Vega-Lite. Recommended: 'yearmonthdate' or 'yearmonthdatehours'. For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'
<code>time_type</code> Altair		Vega Data Type for time variable (default: 'T'). Recommended 'T' (='temporal') for high granularity time resolution, and 'O' (='ordinal') for low cardinality time_unit like 'month'. For more details: https://vega.github.io/vega-lite/docs/type.html	'T'
<code>logy</code> Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Altair		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	None

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal [Altair](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> Altair		type of chart. Options are: "", 'boxplot', 'scatter' and 'missing_values'. can be any of constants.SUPPORTED_AGGREGATIONS, or 'all' meaning that aggregations 'min', 'max' and 'avg' will be overlaid.	<i>required</i>
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>**kwargs</code> Altair		plotting options. Look at dedicated plotting functions for reference.	{}

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_aggregation [↗](#)

```
plot_temporal_aggregation(aggregate, time\_field=None, time\_resolution='1D', show\_points=False, interactive=True, zero\_scale=False, logy=False)
```

Altair chart of the field aggregated with respect to time.

Parameters:

Name	Type	Description	Default
time_field ↗		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
aggregate ↗		aggregation function to be used. Can be any of constants.SUPPORTED_AGGREGATIONS.	<i>required</i>
time_resolution ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here ↗ . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
show_points ↗		bool indicating whether to display the data points (default: False).	False
interactive ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
zero_scale ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False
logy ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_boxplot [↗](#)

```
plot_temporal_boxplot(time\_field=None, time\_resolution=None, bar\_size=10, interactive=True, zero\_scale=False)
```

Altair boxplot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: None, meaning that it will be dynamically chosen to divide the time range in 15 periods.	None
<code>bar_size</code> ↗		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_missing_values [↗](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None or that are not of type datetime.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False

Name	Type	Description	Default
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

`plot_temporal_scatter` [¶](#)

```
plot_temporal_scatter(time_field=None, breakdown_field=None, interactive=True)
```

Altair scatter plot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>breakdown_field</code> ¶		name of the field used as a breakdown field. The points will be color encoded by this field.	None
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

`plot_with_fraud_rate` [¶](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (<code>target_field</code> , <code>target_value</code>). Element <code>target_field</code> is the name of the target field. Element <code>target_value</code> indicate the value to be considered as 'positives'. The <code>target_value</code> will be used for the numerator of the fraud rate. Default: (None, None), meaning use the <code>exploration_results.target_field/value</code> if existing.	(None, None)
			{}

Name	Type	Description	Default
**kwargs ¶		plotting options to pass to plot() method.	

Returns:

Type	Description
unknown	an Altair chart.

to_dict [¶](#)

```
to_dict()
```

FieldResults [¶](#)

```
FieldResults(field_name, field_results, exploration_results)
```

Bases: `ABC`

Generic class containing the results of the exploration compute_stats for a particular field.

Parameters:

Name	Type	Description	Default
field_name ¶		field name.	<i>required</i>
field_results ¶		dictionary-like results of the given field obtained from ds_api.exploration.stats.compute_stats .	<i>required</i>

constant_value instance-attribute [¶](#)

```
constant_value = _constant_value()
```

count instance-attribute [¶](#)

```
count = _stats['count']
```

field_type instance-attribute [¶](#)

```
field_type = field_results[FIELD_TYPE_KEY]
```

histogram instance-attribute [¶](#)

```
histogram = _histogram_df()
```

histogram_computed instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM_KEY in field_results
```

histogram_reduced instance-attribute [¶](#)

```
histogram_reduced = reduce_histogram(histogram, result_type=field_type) if histogram_computed else None
```

is_constant instance-attribute [¶](#)

```
is_constant = _is_constant()
```

n_missing instance-attribute [¶](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [¶](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [¶](#)

```
name = field\_name
```

semantic_tag instance-attribute [¶](#)

```
semantic_tag = get(SEMANTIC\_TAG\_KEY)
```

stats instance-attribute [¶](#)

```
stats = Series(_stats)
```

validations instance-attribute [¶](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS\_KEY i  
n field\_results else None
```

plot abstractmethod [¶](#)

```
plot()
```

To be overridden. Altair chart of the field.

plot_fraud_rate abstractmethod [¶](#)

```
plot_fraud_rate(target)
```

To be overridden. Altair fraud rate chart of the field.

plot_temporal abstractmethod [¶](#)

```
plot_temporal(chart_type, time_field, **kwargs)
```

To be overridden. Altair chart of the field over time.

plot_temporal_missing_values abstractmethod [¶](#)

```
plot_temporal_missing_values(time_field)
```

To be overridden. Altair chart of the missing values over time.

plot_with_fraud_rate abstractmethod [¶](#)

```
plot_with_fraud_rate(target)
```

To be overridden. Altair chart of the field with the fraud rate overlaid.

to_dict [¶](#)

```
to_dict()
```

NumericalFieldResults [¶](#)

```
NumericalFieldResults(field\_name, field\_results, exploration_results)
```

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a numerical field.

Parameters:

Name	Type	Description	Default
field_name ¶		field name.	<i>required</i>
field_results ¶		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	<i>required</i>

[bin_width](#) instance-attribute [¶](#)

```
bin_width = _bin_width()
```

[constant_value](#) instance-attribute [¶](#)

```
constant_value = _constant_value()
```

[count](#) instance-attribute [¶](#)

```
count = _stats['count']
```

[field_type](#) instance-attribute [¶](#)

```
field_type = field\_results[FIELD\_TYPE\_KEY]
```

[histogram](#) instance-attribute [¶](#)

```
histogram = _histogram_df()
```

[histogram_computed](#) instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM\_KEY in field\_results
```

[histogram_reduced](#) instance-attribute [¶](#)

```
histogram_reduced = reduce\_histogram(histogram, result\_type=field_type) if histogram_computed else None
```

[is_constant](#) instance-attribute [¶](#)

```
is_constant = _is_constant()
```

[max](#) instance-attribute [¶](#)

```
max = _stats['max']
```

[mean](#) instance-attribute [¶](#)

```
mean = _stats['mean']
```


min instance-attribute [Altair](#)

```
min = _stats['min']
```

n_infinities instance-attribute [Altair](#)

```
n_infinities = _stats['infinities']
```

n_missing instance-attribute [Altair](#)

```
n_missing = _stats['missing']
```

n_nan instance-attribute [Altair](#)

```
n_nan = _stats['nan']
```

n_unhealthy instance-attribute [Altair](#)

```
n_unhealthy = _n_unhealthy()
```

n_zeros instance-attribute [Altair](#)

```
n_zeros = _stats['zeros']
```

name instance-attribute [Altair](#)

```
name = field\_name
```

semantic_tag instance-attribute [Altair](#)

```
semantic_tag = get(SEMANTIC\_TAG\_KEY)
```

stats instance-attribute [Altair](#)

```
stats = Series(_stats)
```

std instance-attribute [Altair](#)

```
std = _stats['stddev']
```

sum instance-attribute [Altair](#)

```
sum = _stats['sum']
```

validations instance-attribute [Altair](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS\_KEY i  
n field\_results else None
```

plot [Altair](#)

```
plot(nbins=50, fix\_step=False, normalize=None, breakdown\_by=None, stack=False, logy=False, inter  
active=True, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
nbins Altair		approximate number of bins for the desired histogram visualization. Default: 50	50
fix_step Altair		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	False
normalize Altair		bool (or None) indicating if the histogram is to be normalized. If set as True, the histogram is normalized to unity, otherwise the y-axis represents the actual counts. If set as True and breakdown_by is not None, the histogram of every breakdown value will be normalized to unity. Default is None: this means that the histogram will be normalized if breakdown_by is not None, otherwise it will not be normalized.	None
breakdown_by Altair		string specifying a field to breakdown histograms by. Default is None: no breakdown.	None
stack Altair		whether to stack histograms or not. Default: None. Only applies to the case where breakdown_by is not None.	False
logy Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
interactive Altair		bool indicating whether to call the function interactive() on the Altair chart (default: True).	True
mini Altair		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

plot_fraud_rate [Altair](#)

```
plot_fraud_rate(target=(None, None), nbins=50, fix_step=False, bar=True, logy=False, interactive=True)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> Altair		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>bins</code> Altair		approximate number of bins for the desired histogram visualization. Default: 50.	50
<code>fix_step</code> Altair		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	False
<code>bar</code> Altair		bool indicating whether to return a bar chart (True) or a line chart (False). Default: True.	True
<code>logy</code> Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function interactive() on the Altair chart (default: True).	True

Returns:

Type	Description
An Altair chart.	

plot_temporal [Altair](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> Altair		type of chart. Options are: "", 'boxplot', 'scatter' and 'missing_values'. can be any of constants.SUPPORTED_AGGREGATIONS, or 'all' meaning that aggregations 'min', 'max' and 'avg' will be overlaid.	<i>required</i>
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
		plotting options. Look at dedicated plotting functions for reference.	{}

Name	Type	Description	Default
<code>**kwargs</code> ↗			

Returns:

Type	Description
An Altair chart.	

plot_temporal_aggregation [↗](#)

```
plot_temporal_aggregation(aggregate, time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=False, logy=False)
```

Altair chart of the field aggregated with respect to time.

Parameters:

Name	Type	Description	Default
<code>aggregate</code> ↗		aggregation function to be used. Can be any of constants.SUPPORTED_AGGREGATIONS.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here ↗ . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description

Type	Description
An Altair chart.	

plot_temporal_boxplot [¶](#)

```
plot_temporal_boxplot(time_field=None, time_resolution=None, bar_size=10, interactive=True, zero_scale=False)
```

Altair boxplot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>time_resolution</code> ¶		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: None, meaning that it will be dynamically chosen to divide the time range in 15 periods.	None
<code>bar_size</code> ¶		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ¶		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False

Returns:

Type	Description
An Altair chart.	

plot_temporal_missing_values [¶](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None or nan.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None

Name	Type	Description	Default
<code>time_resolution</code> Alt		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> Alt		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> Alt		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> Alt		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

plot_temporal_scatter [Alt](#)

```
plot_temporal_scatter(time_field=None, breakdown_field=None, interactive=True)
```

Altair scatter plot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> Alt		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>breakdown_field</code> Alt		name of the field used as a breakdown field. The points will be color encoded by this field.	None
<code>interactive</code> Alt		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Returns:

Type	Description
An Altair chart.	

plot_with_fraud_rate [Alt](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>**kwargs</code> ¶		plotting options to pass to plot() method.	{}

Returns:

Type	Description
An Altair chart.	

quantile [¶](#)

```
quantile(q=(0.0, 0.25, 0.5, 0.75, 1.0))
```

Return approximate value at the given quantile(s). Values are computed from the histograms, therefore the associated uncertainty is half the bin width of the histogram.

Parameters:

Name	Type	Description	Default
<code>q</code> ¶		float or list/tuple of floats with quantiles to compute. Values must be between 0 and 1. Default is (0.0, 0.25, 0.5, 0.75, 1.0).	(0.0, 0.25, 0.5, 0.75, 1.0)

Returns:

Type	Description
Pandas Series with the quantiles.	

to_dict [¶](#)

```
to_dict()
```

StringFieldResults [¶](#)

```
StringFieldResults(field_name, field_results, exploration_results)
```

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a categorical field.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ¶		field name.	required
<code>field_results</code> ¶		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	required

cardinality instance-attribute [¶](#)

```
cardinality = _cardinality()
```

cardinality_error instance-attribute [¶](#)

```
cardinality_error = _stats['hll_error'] if is_approximate else 0
```

constant_value instance-attribute [¶](#)

```
constant_value = _constant_value()
```

count instance-attribute [¶](#)

```
count = _stats['count']
```

field_type instance-attribute [¶](#)

```
field_type = field_results[FIELD_TYPE_KEY]
```

histogram instance-attribute [¶](#)

```
histogram = _histogram_df()
```

histogram_computed instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM_KEY in field_results
```

histogram_reduced instance-attribute [¶](#)

```
histogram_reduced = reduce_histogram(histogram, result_type=field_type) if histogram_computed else None
```

is_approximate instance-attribute [¶](#)

```
is_approximate = not histogram_computed or OTHER_VAL in [(k[0]) for k in (_histogram)]
```

is_constant instance-attribute [¶](#)

```
is_constant = _is_constant()
```

n_empty instance-attribute [¶](#)

```
n_empty = _stats['empty']
```

n_missing instance-attribute [¶](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [¶](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [¶](#)

```
name = field_name
```

semantic_tag instance-attribute [¶](#)

```
semantic_tag = get(SEMANTIC_TAG_KEY)
```


stats instance-attribute [Altair](#)

```
stats = Series(_stats)
```

validations instance-attribute [Altair](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS_KEY in field_results else None
```

categories [Altair](#)

```
categories(top_n=None)
```

List of top-n most frequent categories ordered by frequency.

Parameters:

Name	Type	Description	Default
<code>top_n</code> Altair		if given return only the top_n most frequent categories.	<code>None</code>

Returns:

Type	Description
<code>list</code>	list of top-n categories ordered by frequency.

frequencies [Altair](#)

```
frequencies(top_n: Optional[int] = None) -> Optional[Series]
```

Frequency table of top-n categories.

Parameters:

Name	Type	Description	Default
<code>top_n</code> Altair	<code>Optional[int]</code>	if given return only the top_n most frequent categories. The rest of the categories will be merge into the '(other)' category.	<code>None</code>

Returns:

Type	Description
<code>Series</code>	Table containing the frequency table of the top_n categories. Indices are category labels, values are frequencies. At most it will have a length of top_n + 2, including the other and the missing keys. If the field histogram was not computed it returns None.

plot [Altair](#)

```
plot(sort_by_freq=True, breakdown_by=None, cat_threshold=None, vertical=True, logy=False, interactive=True, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
<code>sort_by_freq</code> ↗		bool indicating whether to sort by frequency, i.e. categories with higher frequency first. Otherwise, sort alphabetically (default: True).	True
<code>breakdown_by</code> ↗		str specifying field to breakdown histograms by. Default is None: no breakdown.	None
<code>cat_threshold</code> ↗		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold-1 most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	None
<code>vertical</code> ↗		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function interactive() on the Altair chart. Default: True.	True
<code>mini</code> ↗		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

`plot_fraud_rate` [↗](#)

```
plot_fraud_rate(target=(None, None), sort_by='fr', cat_threshold=None, vertical=True, logy=False, interactive=True)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> ↗		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>sort_by</code> ↗		string indicating sorting type. Supported sorting are by frequency ('freq'), by fraud rate ('fr') and alphabetical ('alpha'). Default: 'fr'.	'fr'
<code>cat_threshold</code> ↗		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold-1 most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	None
			True

Name	Type	Description	Default
<code>vertical</code> ↗		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart. Default: True.	True

Returns:

Type	Description
An Altair chart.	

`plot_temporal` [↗](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> ↗		type of chart. Options are: 'categories', 'cardinality' and 'missing_values'.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>**kwargs</code> ↗		other options. Look at dedicated plotting functions for reference.	<code>{}</code>

Returns:

Type	Description
An Altair chart.	

`plot_temporal_cardinality` [↗](#)

```
plot_temporal_cardinality(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the field cardinality with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None

Name	Type	Description	Default
<code>time_resolution</code> Altair		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> Altair		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_temporal_categories` [Altair](#)

```
plot_temporal_categories(time_field=None, time_resolution='1D', top_n=5, show_points=False, interactive=True, zero_scale=True, logy=False)
```

Altair chart of the categories of a string field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> Altair		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>top_n</code> Altair		the number of categorical values, with highest count, that should be displayed.	5
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Name	Type	Description	Default
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description
An Altair chart.	

`plot_temporal_missing_values` [↗](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_with_fraud_rate` [↗](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	<code>(None, None)</code>
<code>**kwargs</code> ¶		plotting options to pass to plot() method.	<code>{}</code>

Returns:

Type	Description
<code>unknown</code>	an Altair chart.

`to_dict` [¶](#)

```
to_dict()
```

field_types [↗](#)

FieldType [↗](#)

Bases: `str`, `Enum`

CATEGORICAL `class-attribute` `instance-attribute` [↗](#)

```
CATEGORICAL = 'string'
```

DATETIME `class-attribute` `instance-attribute` [↗](#)

```
DATETIME = 'timestamp'
```

NUMERICAL `class-attribute` `instance-attribute` [↗](#)

```
NUMERICAL = 'numeric'
```

get_values `classmethod` [↗](#)

```
get_values() -> set[str]
```

formatters

Formatters are utilities for formatting numbers/tables appropriately in HTML code or CSS classes.

CARDINALITY_SEVERITY_THRESHOLD module-attribute

```
CARDINALITY_SEVERITY_THRESHOLD = 0.9
```

FRACTION_SEVERITY_THRESHOLD module-attribute

```
FRACTION_SEVERITY_THRESHOLD = 0.1
```

fmt

```
fmt(name, value, exact=False)
```

Format for a (name, value).

If name starts with 'f_' it'll be considered as a fraction to be shown as a percent.

fmt_amount

```
fmt_amount(number, significant_digits=3, currency='$')
```

Format for amount values. Prefixed by (kMBTQ).

fmt_cardinality_severity

```
fmt_cardinality_severity(v)
```

Return the severity of a cardinality.

More than 90% is considered an alert, and a 0 is considered to be ignored.

fmt_class

```
fmt_class(text, cls)
```

Format a text with a given class.

fmt_color

```
fmt_color(text, color)
```

Format a text with a given color.

fmt_float

```
fmt_float(v)
```

Format for float values

fmt_fraction_severity

```
fmt_fraction_severity(v)
```

Return the severity of a fraction.

More than 10% is considered an alert, and a 0 is considered to be ignored.

fmt_integer

```
fmt_integer(v, exact=False)
```

Format for integer values

fmt_percent

```
fmt_percent(v)
```

Format for fractions values to be formatted as a percent.

formatted

```
formatted(values)
```

Return a dictionary similar to the input dictionary but appropriately formatted.

histograms

ConstantFieldError

Bases: Exception

Raised when numerical field is constant.

NumericalHistogramConfig

```
NumericalHistogramConfig(minv, maxv, max_bins=1000)
```

Bases: object

Configuration object for numerical histograms.

It requires three configurable values: minv/maxv the minimum and maximum histogram values, and max_bins the number of bins or resolution of the histogram.

The minv and maxv configurables are private accessible via get_minmax() and set_minmax(), while nbins is a property.

Parameters:

Name	Type	Description	Default
minv		minimum value of the histogram. Smaller values will be counted in the underflow bin.	required
maxv		maximum value of the histogram. The last bin is closed on the right side, i.e. entries with value == maxv will be counted in the last bin. Larger values will be counted in the overflow bin.	required
max_bins		resolution of the histogram, i.e. number of bins within minv and maxv. There will be two extra bins to account for underflow and overflow.	1000

max_bins property writable

```
max_bins
```

get_minmax

```
get_minmax()
```

set_minmax

```
set_minmax(minv, maxv)
```

Setter function for minv and maxv. It raises exceptions if they are not finite numbers or if minv > maxv.

TimeHistogramConfig

```
TimeHistogramConfig(time_resolution='h')
```

Bases: object

Configuration object for datetime histograms. It requires one configurable value: the `time_resolution` of the plot.

Parameters:

Name	Type	Description	Default
<code>time_resolution</code> ↗		time resolution of the histogram.	<code>'h'</code>

`time_resolution` [property](#) [writable](#) [↗](#)

`time_resolution`

`build_numerical_bin_expr` [↗](#)

`build_numerical_bin_expr(field: StructField, stats: dict, max_bins: int) -> Tuple[str, Column, Callable] | None`

Builds a bin expression for a numerical field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>) where <code>bin_expr</code> maps each value to a string-encoded bin index and <code>decode_fn</code> converts the string back to the (<code>bin_range</code> , <code>breakdown_values</code>) key used in the histogram dict. If the field is constant, returns <code>None</code> .

`build_string_bin_expr` [↗](#)

`build_string_bin_expr(field: StructField, stats: dict, cardinality_threshold: int | None, dist_sampling: float | None) -> Tuple[str, Column, Callable]`

Builds a bin expression for a string field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>).

`build_timestamp_bin_expr` [↗](#)

`build_timestamp_bin_expr(field: StructField, time_config: TimeHistogramConfig) -> Tuple[str, Column, Callable]`

Builds a bin expression for a timestamp field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>).

compute_all_histograms [↗](#)

```
compute_all_histograms(data: _TDataFrame, field_bin_exprs: list[Tuple[str, Column, Callable]], breakdown_cols: list[Column], breakdowns: list[str]) -> dict[str, dict]
```

Compute histograms for all fields in a single pass:

1. Builds an array-of-structs with one element per field, each containing `(field_name_literal, bin_expression)`.
2. Explodes the array so every input row produces N rows (one per field).
3. Runs a single `groupBy("__field", "__bin", *breakdowns).count()`.
4. Partitions the collected rows back into per-field histogram dicts using the field-specific `decode_fn`.

Parameters:

Name	Type	Description	Default
<code>data</code> ↗	<code>DataFrame</code>	Source DataFrame.	<i>required</i>
<code>field_bin_exprs</code> ↗	list of (str, Column, callable)	One tuple per field, as returned by <code>build_*_bin_expr</code> . Each tuple is <code>(field_name, bin_column_expr, decode_fn)</code> .	<i>required</i>
<code>breakdown_cols</code> ↗	list of Column	Pre-built breakdown column expressions for the <code>groupBy</code> .	<i>required</i>
<code>breakdowns</code> ↗	list of str	Field names corresponding to <code>breakdown_cols</code> .	<i>required</i>

Returns:

Type	Description
<code>dict[str, dict]</code>	Mapping from field name to histogram dict. Each histogram dict maps <code>(bucket_key, breakdown_values) -> count</code> .

plots

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

quiet_mode module-attribute

```
quiet_mode = False
```

plot_calendar

```
plot_calendar(field_name, histogram_df, xaxis='date', yaxis='yearmonth', color='fr',
size='freq', color_scheme=None, target_field=None, target_value=None)
```

Function to plot a calendar-like plot of a time variable using Altair.

Recommended combinations of axis units are (xaxis="date", yaxis="yearmonth") and (xaxis="hours", yaxis="yearmonthdate").

Parameters:

Name	Type	Description	Default
<code>field_name</code>		field name.	<i>required</i>
<code>histogram_df</code>		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>xaxis</code>		time unit of the xaxis. Default: "hours". Recommended combinations of units are (xaxis="date", yaxis="yearmonth") and (xaxis="hours", yaxis="yearmonthdate")	'date'
<code>yaxis</code>		time unit of the yaxis. Default: "yearmonthdate".	'yearmonth'
<code>color</code>		color encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'fr'
<code>size</code>		size encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'freq'
<code>color_scheme</code>		Vega color scheme to be used. Available schemes can be found here: https://vega.github.io/vega/docs/schemes/ . Default is None, meaning 'blues' for 'freq' and diverging 'blueorange' centered at the overall fraud rate for 'fr'.	None
			None

Name	Type	Description	Default
<code>target_field</code> ↗		name of the target field. Must be given if color/size encoding contain 'fr'. Otherwise the given fraud rate encoding will be disabled.	
<code>target_value</code> ↗		value to be considered as 'positives' labels. Must be given if color/size encoding contain 'fr'. Otherwise the given fraud rate encoding will be disabled.	None

Returns:

Type	Description
unknown	an Altair chart.

plot_fraud_rate_numerical [↗](#)

```
plot_fraud_rate_numerical(field_name, histogram_df, target_field, target_value, nbins=50, fix_step=False, bar=True, logy=False, interactive=True)
```

Function for default fraud rate plotting of numerical variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ↗		field name.	<i>required</i>
<code>histogram_df</code> ↗		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>target_field</code> ↗		name of the target field.	<i>required</i>
<code>target_value</code> ↗		value to be considered as 'positives' labels.	<i>required</i>
<code>nbins</code> ↗		approximate number of bins for the desired histogram visualization. Default: 50	50
<code>fix_step</code> ↗		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not	False

Name	Type	Description	Default
		fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	
<code>bar</code> Alt		bool indicating whether to return a bar chart (True) or a line chart (False). Default: True.	True
<code>logy</code> Alt		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Alt		bool indicating whether to call the function interactive() on the Altair chart (default: True).	True

Returns:

Type	Description
unknown	an Altair chart. If bar=False, this will be a layered Altair chart.

plot_fraud_rate_string [Alt](#)

```
plot_fraud_rate_string(field_name, histogram_df, target_field, target_value, sort_by='fr', cat_threshhold=None, vertical=True, logy=False, interactive=True)
```

Function for default fraud rate plotting of categorical/string variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> Alt		field name.	<i>required</i>
<code>histogram_df</code> Alt		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>target_field</code> Alt		name of the target field.	<i>required</i>
<code>target_value</code> Alt		value to be considered as 'positives' labels.	<i>required</i>
<code>sort_by</code> Alt		string indicating sorting type. Supported sorting are by frequency ('freq'), by fraud rate ('fr') and alphabetical ('alpha'). Default: 'fr'.	'fr'
			None

Name	Type	Description	Default
<code>cat_threshold</code> ↗		number of categories to keep. In case the underlying cardinality is larger, keep the <code>cat_threshold</code> most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	
<code>vertical</code> ↗		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart. Default: True.	True

Returns:

Type	Description
unknown	an Altair chart.

plot_fraud_rate_time [↗](#)

```
plot_fraud_rate_time(field_name, histogram_df, target_field, target_value, time_unit='yearmonthdate', time_type='T', logy=False, interactive=None)
```

Function for default fraud rate plotting of time variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ↗		field name.	<i>required</i>
<code>histogram_df</code> ↗		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>target_field</code> ↗		name of the target field.	<i>required</i>
<code>target_value</code> ↗		value to be considered as 'positives' labels.	<i>required</i>
<code>time_unit</code> ↗		a time unit supported by Vega-Lite. Recommended: "yearmonthdate" or "yearmonthdatehours". For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'

Name	Type	Description	Default
<code>time_type</code> Alt		Vega Data Type for time variable (default: "T"). Recommended "T" ("temporal") for high granularity time resolution, and "O" ("ordinal") for low cardinality time_unit like "month". For more details: https://vega.github.io/vega-lite/docs/type.html	'T'
<code>logy</code> Alt		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Alt		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	None

Returns:

Type	Description
unknown	an Altair chart.

plot_numerical [Alt](#)

```
plot_numerical(field_name, histogram_df, nbins=50, fix_step=False, normalize=None, breakdown_by=None, stack=False, logy=False, interactive=True, mini=False)
```

Function for default plotting of numerical variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> Alt		field name.	<i>required</i>
<code>histogram_df</code> Alt		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>nbins</code> Alt		approximate number of bins for the desired histogram visualization. Default: 50	50
<code>fix_step</code> Alt		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	False

Name	Type	Description	Default
<code>normalize</code> ↗		bool (or None) indicating if the histogram is to be normalized. If set as True, the histogram is normalized to unity, otherwise the y-axis represents the actual counts. If set as True and <code>breakdown_by</code> is not None, the histogram of every breakdown value will be normalized to unity. Default is None: this means that the histogram will be normalized if <code>breakdown_by</code> is not None, otherwise it will not be normalized.	None
<code>breakdown_by</code> ↗		string specifying a field to breakdown histograms by. Default is None: no breakdown.	None
<code>stack</code> ↗		whether to stack histograms or not. Default: None. Only applies to the case where <code>breakdown_by</code> is not None.	False
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>mini</code> ↗		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
unknown	An Altair chart.

plot_string [↗](#)

```
plot_string(field_name, histogram_df, sort_by_freq=True, breakdown_by=None, cat_threshold=None, vertical=True, logy=False, interactive=True, mini=False)
```

Function for default plotting of categorical/string variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ↗		field name.	<i>required</i>
<code>histogram_df</code> ↗		Pandas DataFrame representation of a histogram.	<i>required</i>

Name	Type	Description	Default
<code>sort_by_freq</code> ↗		bool indicating whether to sort by frequency, i.e. categories with higher frequency first. Otherwise, sort alphabetically (default: True).	True
<code>breakdown_by</code> ↗		str specifying field to breakdown histograms by. Default is None: no breakdown.	None
<code>cat_threshold</code> ↗		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	None
<code>vertical</code> ↗		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function interactive() on the Altair chart. Default: True.	True
<code>mini</code> ↗		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_aggregation [↗](#)

```
plot_temporal_aggregation(df, time_field, field_name, aggregate='avg', time_resolution='1 day', show_points=False, interactive=True, zero_scale=True)
```

Function to process a PySpark DataFrame, computing a given aggregate function on a given field over time windows defined by a time resolution, and plotting it using a line plot.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	required
<code>time_field</code> ↗		name of the PySpark DataFrame column associated to the time field.	required

Name	Type	Description	Default
<code>field_name</code> Altair		name of the PySpark DataFrame column associated to the field used to calculate the aggregation over.	<i>required</i>
<code>aggregate</code> Altair		aggregation function to be used. The available functions are "avg", "max", "min", "sum", "count". UDF created with <code>pyspark.sql.functions.pandas_udf()</code> may also be used. See https://spark.apache.org/docs/latest/api/python/pyspark.sql.html#pyspark.sql.GrouppedData for more details. (default: avg).	'avg'
<code>time_resolution</code> Altair		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the <code>windowDuration</code> parameter of <code>pyspark.sql.functions.window</code> are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False)	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> Altair		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_boxplot [Altair](#)

```
plot_temporal_boxplot(df, field_name, bar_size=10, interactive=True, zero_scale=True)
```

Function for default plotting a box plot of numerical or timestamp variables over time, using Altair.

Parameters:

Name	Type	Description	Default
<code>df</code> Altair		Pandas DataFrame with the aggregated data. The expected format is six columns. The first should be of type datetime and the remaining should contain the min, max, 25%, median and 75% values of the field of interest in a time window.	<i>required</i>
<code>field_name</code> Altair		field name.	<i>required</i>

Name	Type	Description	Default
<code>bar_size</code> ↗		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_cardinality [↗](#)

```
plot_temporal_cardinality(df, time_field, field_name, show_cardinality_over_trx=True,
show_points=False, interactive=True, zero_scale=True)
```

Function for default plotting two line plots of the cardinality of a categorical variable over time, using Altair.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame with the aggregated data. The expected format is three columns. The first should be a datetime and the second and third should be numerical. The column names will be used as the axis titles.	<i>required</i>
<code>time_field</code> ↗		field name for the temporal dimension.	<i>required</i>
<code>field_name</code> ↗		field name.	<i>required</i>
<code>show_cardinality_over_trx</code> ↗		bool display a second plot of the cardinality divided by the total number of transactions in that time period.	True
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_lines [↗](#)

```
plot_temporal_lines(df, field_name, show_points=False, interactive=True, zero_scale=True, logy=False)
```

Function for default plotting line plots of given numeric data points, over time, using Altair.

Multiple lines can be drawn representing different categories. It may be used to plot, for example, the values of an aggregation function (min, max, mean, ...) of numerical variables or the cardinality of a categorical variable over time.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pandas DataFrame with the data. The expected format is two or three columns. The first corresponds to the time dimension (x-axis) and should be of datetime type. The second corresponds to an arbitrary numerical quantity (y-axis), for instance a mean or a cardinality. If the third, and optional, column is passed, it will be used to draw multiple lines using Altair's color encoding and should be of type string. The column names will be used as the axis titles.	<i>required</i>
<code>field_name</code> ↗		field name.	<i>required</i>
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_points [↗](#)

```
plot_temporal_points(df, time_field, field_name, breakdown_field=None, interactive=True)
```

Function for plotting points given numeric data, over time, using Altair.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pandas DataFrame with the data.	<i>required</i>
<code>time_field</code> ↗		name of the column associated to the time field.	<i>required</i>
<code>field_name</code> ↗		name of the column associated to the numerical field.	<i>required</i>
<code>breakdown_field</code> ↗		name of the column used as a breakdown field. The points will be color encoded by this field.	None
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_quantiles [↗](#)

```
plot_temporal_quantiles(df, time_field, field_name, time_resolution='1 day', bar_size=10, interactive=True, zero_scale=True)
```

Function to process a PySpark DataFrame, computing the min, max and quantiles of a given field over time windows defined by a time resolution, and plotting it using a boxplot.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame. It should contain at least two columns, one with the value of <code>time_field</code> as column name and another with the value of <code>field_name</code> .	<i>required</i>
<code>time_field</code> ↗		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> ↗		name of the PySpark DataFrame column associated to the field used to calculate the quantiles over.	<i>required</i>

Name	Type	Description	Default
<code>time_resolution</code> ↗		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of <code>pyspark.sql.functions.window</code> are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'
<code>bar_size</code> ↗		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_top_n [↗](#)

```
plot_temporal_top_n(df, time_field, field_name, top_n=5, time_resolution='1 day', show_points=False, interactive=True, zero_scale=True)
```

Function for plotting a line plot for the counts of the top-n most frequent categories of a given string field over a time window defined by a time resolution.

It shows the evolution over time of a given quantity for each category, using Altair.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame. It should contain at least two columns, one with the value of <code>time_field</code> as column name. It corresponds to the time dimension (x-axis) and should be of datetime type. The other should have the value of <code>field_name</code> as the column name. It contains categorical values of str type and corresponds to the categorical dimension (color encoding).	<i>required</i>
<code>time_field</code> ↗		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> ↗		name of the PySpark DataFrame column associated to the string field.	<i>required</i>

Name	Type	Description	Default
<code>top_n</code> Altair		the number of categorical values, with highest count, that should be used in the aggregation.	5
<code>time_resolution</code> Altair		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the <code>windowDuration</code> parameter of <code>pyspark.sql.functions.window</code> are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False)	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> Altair		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

plot_time [Altair](#)

```
plot_time(field_name, histogram_df, time_unit='yearmonthdate', time_type='T', breakdown_by=None,
logy=False, interactive=None, mini=False)
```

Function for default plotting of time variables using Altair.

Parameters:

Name	Type	Description	Default
<code>field_name</code> Altair		field name.	<i>required</i>
<code>histogram_df</code> Altair		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>time_unit</code> Altair		a time unit supported by Vega-Lite. Recommended: "yearmonthdate" or "yearmonthdatehours". For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'

Name	Type	Description	Default
<code>time_type</code> Altair		Vega Data Type for time variable (default: "T"). Recommended "T" ("temporal") for high granularity time resolution, and "O" ("ordinal") for low cardinality time_unit like "month". For more details: https://vega.github.io/vega-lite/docs/type.html	<code>'T'</code>
<code>breakdown_by</code> Altair		str specifying field to breakdown histograms by. Default is None: no breakdown.	<code>None</code>
<code>logy</code> Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	<code>False</code>
<code>interactive</code> Altair		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	<code>None</code>
<code>mini</code> Altair		bool indicating if this is a mini-histogram.	<code>False</code>

Returns:

Type	Description
<code>unknown</code>	an Altair chart.

`set_quiet_mode` [Altair](#)

```
set_quiet_mode(mode)
```

report

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

report

```
report(exploration_results, show_all_checks=False, embed_dependencies=False)
```

Function that prints the report for stats.

Parameters:

Name	Type	Description	Default
exploration_results		ExplorationResults instance created from the stats.compute_stats method.	required
show_all_checks		Boolean value indicating whether all checks that were run should be displayed in the report, even if they triggered no warnings. Default is False.	False
embed_dependencies		Boolean value indicating if the report html file should include all css and js dependencies (bootstrap, jquery, vega). Embedding them increases significantly the size of the file, but in security-closed environments where there is no internet access, the dependencies might need to be embedded in order to be able to render the report. Default is False.	False

Returns:

Type	Description
str	Overview report of the data in HTML format.

stats

CHECKPOINT_FILE module-attribute

```
CHECKPOINT_FILE = 'checkpoint.pkl'
```

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

compute_histograms

```
compute_histograms(data, stats, breakdowns, cardinality_threshold, dist_sampling, max_bins, time_resolution)
```

Computes histograms.

Parameters:

Name	Type	Description	Default
data 		the Spark Dataframe with the data to be used in order to compute histograms.	<i>required</i>
stats 		dictionary with statistics per field relevant for the computation of histograms.	<i>required</i>
breakdowns 		a list of field names to calculate breakdowns over.	<i>required</i>
cardinality_threshold 		cardinality threshold to decide if the full value counts is to be computed for string histograms. If the estimated cardinality is smaller than this value, then the full string histogram computation will be performed. Otherwise, only the top most frequent values are to be counted.	<i>required</i>
dist_sampling 		sampling factor between 0 and 1 used for the estimation of value_counts for string fields during the compute_stats phase.	<i>required</i>
max_bins 		binning resolution for the histogram computation of numerical fields. This value define the number of uniform buckets to be computed between to minimum and the maximum value of the field.	<i>required</i>
time_resolution 		time resolution for the histogram computation of datetime fields. Supported resolutions: 'h' for hours, 'd' for days.	<i>required</i>

Returns:

Type	Description
dict	a dictionary with the descriptive statistics.

compute_stats [¶](#)

```
compute_stats(data: _TDataFrame, semantics_map: dict | None = None, validations: Any = None, timestamp_field: str | None = None, cost_field: str | None = None, target: tuple | None = None, breakdowns: list[str] | None = None, breakdown_card_threshold: int = 10, cardinality_sample_size: int = 100000, max_bins: int = 1000, time_resolution: str = 'h', time_dependencies_sample_size: int = 100000, checkpoint_file: str = CHECKPOINT_FILE)
```

Computes the descriptive statistics and optional validations for all the fields contained in the data. The statistics/validations will be returned in a dictionary whose keys are the fields in the dataset and the values will be dictionaries with the descriptive statistics/validations for those fields.

The computation will be performed over the RDD representation of the dataset in a single pass over the data through a map and a reduce step. The map operation will be applied to an entire row and will result in a dictionary with the mapped values for each feature and statistic/validation per row. Then a single reduce operation will be applied, reducing the dataset to a single dictionary with the aggregated statistics/validations.

Parameters:

Name	Type	Description	Default
data ¶	<code>_TDataFrame</code>	the Spark Dataframe with the data to be used in order to compute statistics.	<i>required</i>
semantics_map ¶	<code>dict None</code>	dictionary matching fields (keys) to semantic tags (values). Default: None.	None
validations ¶	<code>Any</code>	list/tuple/set containing list of Validation. It will be appended to the built-in set of validations. Default: None.	None
timestamp_field ¶	<code>str None</code>	name of the column to be associated as timestamp field. Will be tagged with the <code>TIMESTAMP</code> semantic and associated built-in validations will be performed. Default: None.	None
cost_field ¶	<code>str None</code>	name of the column to be associated as cost field. Will be tagged with the <code>AMOUNT</code> semantic and associated built-in validations will be performed. Default: None.	None
target ¶	<code>tuple None</code>	tuple containing (target_field, target_value). Target field is the name of the columns to be associated as the target field, will be included in the list of breakdowns. Will be tagged with the <code>AMOUNT</code> semantic and associated built-in validations will be performed. Target value indicate the value to be considered as 'positives'. Default: None.	None
breakdowns ¶	<code>list[str] None</code>	a list of field names to calculate breakdowns over. So far breakdowns are only relevant for histograms.	None
breakdown_card_threshold ¶	<code>int</code>	cardinality threshold for breakdown fields. A protection is added so if a specified breakdown field has a cardinality larger than this threshold, then it will be removed from the list of fields to calculate breakdowns over. Default value is 10.	10

Name	Type	Description	Default
<code>cardinality_threshold</code> ↗	int None	cardinality threshold to decide if the full value counts is to be computed for string histograms. If the estimated cardinality is smaller than this value, then the full string histogram computation will be performed. Otherwise, only the top most frequent values are to be counted. Default is None, meaning that the full computation will be performed.	None
<code>cardinality_sample_size</code> ↗	int	sample size used for the estimation of value_counts for string fields. If a negative value is given then the full sample will be used (not recommended for samples larger than 1M instances). If None is given, then no value_counts computation will be performed. Default is 100k.	100000
<code>max_bins</code> ↗	int	binning resolution for the histogram computation of numerical fields. This value defines the number of uniform buckets to be computed between the minimum and the maximum value of the field. Default is 1000.	1000
<code>time_resolution</code> ↗	str	time resolution for the histogram computation of datetime fields. Supported resolutions: 'h' for hours, 'd' for days. Default is 'd'.	'h'
<code>time_dependencies_sample_size</code> ↗	int	number of instances to use in the computation of time dependencies. If a negative value is given then all instances will be used. If None is given then time dependencies won't be computed. Default: 100k.	100000

Returns:

Type	Description
unknown	an instance of ExplorationResults containing the results of the computed statistics, validations and histograms.

compute_stats_from_checkpoint [↗](#)

```
compute_stats_from_checkpoint(checkpoint_file=CHECKPOINT_FILE, data=None)
```

create_exploration_results [↗](#)

```
create_exploration_results(params, data, checkpoint_file=CHECKPOINT_FILE)
```

run_validations [↗](#)

```
run_validations(data, semantics_map, validations=None, timestamp_field=None, cost_field=None, target=None)
```

Computes the specified validations per field contained in the data. The validations will be returned in a dictionary whose keys are the fields in the dataset and the values will be dictionaries with the validations for those fields.

The computation will be performed over the RDD representation of the dataset in a single pass over the data through a map and a reduce step. The map operation will be applied to an entire row and will result in a dictionary with the mapped values for each feature and validation per row. Then a single reduce operation will be applied, reducing the dataset to a single dictionary with the aggregated validations.

Parameters:

Name	Type	Description	Default
<code>data</code> ↗		the Spark Dataframe with the data to be used in order to compute statistics.	<i>required</i>
<code>semantic s_map</code> ↗		dictionary matching fields (keys) to semantic tags (values).	<i>required</i>
<code>validati ons</code> ↗		list/tuple/set containing list of Validation. It will be appended to the built-in set of validations.	None
<code>timestamp p_field</code> ↗		name of the column to be associated as timestamp field. Will be tagged with the TIMESTAMP semantic and associated built-in validations will be performed. Default: None.	None
<code>cost_fie ld</code> ↗		name of the column to be associated as cost field. Will be tagged with the AMOUNT semantic and associated built-in validations will be performed. Default: None.	None
<code>target</code> ↗		tuple containing (target_field, target_value). Target field is the name of the columns to be associated as the target field. Will be tagged with the TARGET semantic and associated built-in validations will be performed. Target value indicate the value to be considered as 'positives'. Default: None.	None

Returns:

Type	Description
<code>dict</code>	a dictionary with the descriptive validations.

templates

Contains all templates and auxiliary functions used for generating the HTML report.

base_template module-attribute

```
base_template = get_template('base.html')
```

env module-attribute

```
env = Environment(loader=pl)
```

freq_template module-attribute

```
freq_template = get_template('mini_freq_table_row.html')
```

overview_template module-attribute

```
overview_template = get_template('overview.html')
```

pl module-attribute

```
pl = PackageLoader('ds_api.exploration', 'html_templates')
```

plot_template module-attribute

```
plot_template = get_template('plot.html')
```

row_cat_template module-attribute

```
row_cat_template = get_template('row_cat.html')
```

row_constant_template module-attribute

```
row_constant_template = get_template('row_constant.html')
```

row_num_template module-attribute

```
row_num_template = get_template('row_num.html')
```

row_time_template module-attribute

```
row_time_template = get_template('row_time.html')
```

row_var_template module-attribute

```
row_var_template = {NUMERICAL: row\_num\_template, CATEGORICAL: row\_cat\_template, DATETIME: row\_time\_template, 'constant': row\_constant\_template}
```


sample_template module-attribute [↗](#)

```
sample_template = get_template('sample.html')
```

time_dependencies_template module-attribute [↗](#)

```
time_dependencies_template = get_template('time_dependencies.html')
```

validation_template module-attribute [↗](#)

```
validation_template = get_template('validation.html')
```

warning_messages module-attribute [↗](#)

```
warning_messages = {'const': warning_message_fn('Constant value {0[value]} - '), 'cardinality': warning_message_fn('{0[cardinality]} distinct values - '), 'missing': warning_message_fn('{0[f_missing]} - '), 'empty': warning_message_fn('{0[f_empty]} - '), 'infinite': warning_message_fn('{0[f_infinities]} - '), 'zeros': warning_message_fn('{0[f_zeros]} - '), 'nan': warning_message_fn('{0[f_nan]} - ')}
```

wrapper_template module-attribute [↗](#)

```
wrapper_template = get_template('wrapper.html')
```

initialize_warnings [↗](#)

```
initialize_warnings()
```

Returns a correctly initialized dictionary to store the warnings

make_freq_table [↗](#)

```
make_freq_table(freq_table)
```

Function that returns a formatted HTML code corresponding to a frequency table.

:param freq_table: Pandas Series containing the frequency table. Indices are the category labels, values are the frequencies. :returns: frequency table in formatted HTML code.

make_time_dependencies_table [↗](#)

```
make_time_dependencies_table(time_dependencies)
```

Function that returns a formatted HTML code corresponding to the time dependencies table table.

Parameters:

Name	Type	Description	Default
<code>time_dependencies</code> ↗		Pandas Series containing the time dependencies as returned by <code>time_dependencies.compute_time_dependencies</code> .	<i>required</i>

Returns:

Type	Description
unknown	frequency table in formatted HTML code.

render_plot [↗](#)

```
render_plot(chart, chart_id, is_mini=False)
```

Given an altair chart renders an html using the plot template.

warning_message_fn [↗](#)

```
warning_message_fn(message_value, message_key=None)
```

time_dependencies

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

compute_time_dependencies

```
compute_time_dependencies(df, time_field, sample_size=100000, missing_value_strategy='impute')
```

Computes a list of fields ordered by their dependency with time. To obtain the degree of dependency, a model is trained to predict the timestamp field. The dependency score of each feature is its feature importance on that regression model.

Parameters:

Name	Type	Description	Default
df	DataFrame	a PySpark DataFrame.	required
time_field	str	name of the reference time field over which the dependencies are calculated.	required
sample_size	int	Number of instances to keep after sampling. The sampled dataset will be fed to the model. This is needed in order for the computation to run in a feasible time, for large datasets. If the dataframe is smaller than sample_size, then all instances are used. Default is 100k.	100000
missing_value_strategy	str	method to use for missing value treatment. Possible values are "impute" and "drop". "Impute" fills missing values with the mean, for numerical and datetime fields, and with the constant values "(missing)", for string fields. "Impute" may cause the dependency to be inflated if the missing values of a field are concentrated in the same region of time. Default is "impute".	'impute'

Returns:

Type	Description
Series	Pandas Series where each row contains the name of a feature (as the index) and a value representing its dependency with time. Sorted in descending order. The higher the value, the stronger the dependency.

transformations

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

aggregate_function_over_time

```
aggregate_function_over_time(df, time_field, field_name, aggregate='avg', time_resolution='1 day')
```

Function to process a PySpark DataFrame, computing a given aggregate function on a given field over time windows defined by a time resolution.

Parameters:

Name	Type	Description	Default
df		PySpark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	required
time_field		name of the PySpark DataFrame column associated to the time field.	required
field_name		name of the PySpark DataFrame column associated to the field used to calculate the aggregation over.	required
aggregate		aggregation function to be used. The available functions are "avg", "max", "min", "sum", "count". UDF created with pyspark.sql.functions.pandas_udf() may also be used. See https://spark.apache.org/docs/latest/api/python/pyspark.sql.html#pyspark.sql.GroupedData for more details. (default: avg).	'avg'
time_resolution		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
DataFrame	Pandas DataFrame with two columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second column contains the values of the aggregation function and its name is the name of the aggregation function that was used.

count_top_n_over_time [↗](#)

```
count_top_n_over_time(df, time_field, field_name, top_n=5, time_resolution='1 day')
```

Function to process a PySpark DataFrame, computing the counts of the top n most frequent categories of a given string field over a time windows defined by a time resolution.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		Pyspark DataFrame. It should contain at least two columns, one with the value of time_field as column name and another with the value of field_name.	<i>required</i>
<code>time_field</code> ↗		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> ↗		name of the PySpark DataFrame column associated to the string field used to calculate the top N values aggregation over time.	<i>required</i>
<code>top_n</code> ↗		the number of the top most frequent categorical values to be drawn.	5
<code>time_resolution</code> ↗		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the windowDuration parameter of pyspark.sql.functions.window are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
Data Frame	Pandas Dataframe with three columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second contains the frequency counts of the top n values of the string field, over that time window, and is called "count". Lastly, the third contains the name of that categorical value, and the column name is the value of field_name.

datetime_to_unix_epoch_int [↗](#)

```
datetime_to_unix_epoch_int(df, col=None)
```

fix_out_of_bounds_timestamp [↗](#)

```
fix_out_of_bounds_timestamp(time_series)
```

Fixes the time series to be within bounds.

Parameters:

Name	Type	Description	Default
<code>time_series</code> ↗	Series	Pandas.Series representing time.	<i>required</i>

Returns:

Type	Description
Series	series where out of bounds values are fixed to the bound limits.

get_quantiles [↗](#)

```
get_quantiles(df_hist, q=(0.0, 0.25, 0.5, 0.75, 1.0), half_bin_width=0,
is_timestamp_field=False) -> Optional[DataFrame]
```

Computes approximate quantiles from a histogram in its DataFrame representation.

Parameters:

Name	Type	Description	Default
<code>df_hist</code> ↗		histogram as a pandas DataFrame.	<i>required</i>
<code>q</code> ↗		float or tuple/list of floats with quantiles to compute. Values must be between 0 and 1. Default is (0.0, 0.25, 0.5, 0.75, 1.0).	(0.0, 0.25, 0.5, 0.75, 1.0)
<code>half_bin_width</code> ↗		half of the bin width. It will be used to correct the quantile approximation to give the center of the bin. Default: 0 (no correction).	0
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False

Returns:

Type	Description
DataFrame	DataFrame with a single row containing the quantiles.

get_sampled_df [↗](#)

```
get_sampled_df(data, n_sample=500, balance=True, target=(None, None))
```

Returns a subsample of a PySpark/Pandas dataframe as a Pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>data</code> ↗		Either a PySpark or a Pandas DataFrame.	<i>required</i>
			500

Name	Type	Description	Default
<code>n_sample</code> ↗		number of rows to sample from the DataFrame. If negative, all instances will be sampled. Default: 500.	
<code>balance</code> ↗		boolean indicating if the sampled dataframe should be balanced or not with respect to the target field, so as many positives as non-positives will be sampled. Only relevant if target attribute is set. Default: False.	<code>True</code>
<code>target</code> ↗		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. Default: (None, None).	<code>(None, None)</code>

Returns:

Type	Description
<code>DataFrame</code>	Sampled Pandas DataFrame.

`group_by_sum_with_missing` [↗](#)

```
group_by_sum_with_missing(df, groupby_cols)
```

Applies a group-by sum keeping track of missing values in group-by columns

`histogram2df_fraud_rate` [↗](#)

```
histogram2df_fraud_rate(histogram_df, field_type, target_field, target_value,
cat_threshold=None, time_resolution=None)
```

Transforms the given histogram into a convenient Pandas DataFrame including a fraud rate column.

Parameters:

Name	Type	Description	Default
<code>histogram_df</code> ↗		Pandas DataFrame representation of a histogram.	<i>required</i>
<code>target_field</code> ↗		name of the target field.	<i>required</i>
<code>target_value</code> ↗		value to be considered as 'positives' labels.	<i>required</i>

Name	Type	Description	Default
<code>cat_threshold</code> Altair		number of categories to keep. In case the underlying cardinality is larger, keep the <code>cat_threshold</code> most frequent categories and create a new category 'other' as the aggregation of the rest of categories. The 'missing' category is always treated as a special category, meaning that it will not be merged into 'other' and it will be ignored when finding the <code>cat_threshold</code> most frequent categories. Default is None: keep all categories. Only relevant for string histograms.	None
<code>time_resolution</code> Altair		project time field to given <code>time_resolution</code> . Default is None: keep underlying time resolution. Only relevant for time histograms.	None

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram including a fraud rate column.

histogram2df_numerical [Altair](#)

```
histogram2df_numerical(histogram, breakdowns)
```

Transforms the numerical histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair	dict	dictionary representation of a histogram.	<i>required</i>
<code>breakdowns</code> Altair	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

histogram2df_string [Altair](#)

```
histogram2df_string(histogram, breakdowns)
```

Transforms the string histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair		dictionary representation of a histogram.	<i>required</i>
<code>breakdowns</code> Altair		list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

histogram2df_time [¶](#)

```
histogram2df_time(histogram, breakdowns)
```

Transforms the time histogram into a convenient pandas DataFrame for plotting with Altair.

Parameters:

Name	Type	Description	Default
histogram ¶	dict	dictionary representation of a histogram.	<i>required</i>
breakdowns ¶	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	pandas DataFrame representation of the histogram.

normalize_histogram [¶](#)

```
normalize_histogram(df, breakdown\_by=None)
```

Function to normalize the histogram to unity.

Parameters:

Name	Type	Description	Default
df ¶	DataFrame	pandas DataFrame representation of a histogram.	<i>required</i>
breakdown_by ¶	str	string that specifies the breakdown field. The histogram of every breakdown value will be normalized to unity. Default is None: normalize without breakdown.	None

Returns:

Type	Description
DataFrame	Normalized DataFrame representation of the histogram.

quantiles_over_time [¶](#)

```
quantiles_over_time(df, time\_field, field\_name, time\_resolution='1 day')
```

Function to process a PySpark DataFrame, computing the min, max and quantiles of a given field over time windows defined by a time resolution. The quantiles are approximated, yielding a relative error of 0.01%. See https://spark.apache.org/docs/2.3.0/api/sql/index.html#percentile_approx for more details.

Parameters:

Name	Type	Description	Default
<code>df</code> Altair		PySpark DataFrame. It should contain at least two columns, one with the value of <code>time_field</code> as column name and another with the value of <code>field_name</code> .	<i>required</i>
<code>time_field</code> Altair		name of the PySpark DataFrame column associated to the time field.	<i>required</i>
<code>field_name</code> Altair		name of the PySpark DataFrame column associated to the field used to calculate the quantiles over.	<i>required</i>
<code>time_resolution</code> Altair		time resolution to be used is provided as a string containing a number and an interval or a combination of those. For example, '1 day 12 hours'. Valid intervals are 'week', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond'. All valid values for the <code>windowDuration</code> parameter of <code>pyspark.sql.functions.window</code> are accepted. See http://spark.apache.org/docs/latest/api/python/pyspark.sql.html?highlight=window#module-pyspark.sql.functions for more details. (default: 1 day).	'1 day'

Returns:

Type	Description
<code>DataFrame</code>	Pandas DataFrame with six columns. One is the time field column, it has the value of <code>time_field</code> as the column name and contains the start datetime for each time window. The others contain the values of the min, max and the 25%, 50% and 75% quantiles. They are called min, 25%, median, 75% and max and appear in this order.

reduce_categorical_histogram [Altair](#)

```
reduce_categorical_histogram(histogram: DataFrame, breakdown_by: str | None = None,
                             cat_threshold: int | None = None, **kwargs)
```

reduce_histogram [Altair](#)

```
reduce_histogram(histogram: DataFrame, result_type: str, **kwargs) -> DataFrame
```

Function to reduce a histogram DataFrame to the minimal information to pass to Altair.

Parameters:

Name	Type	Description	Default
<code>histogram</code> Altair	<code>DataFrame</code>	DataFrame representation of a histogram.	<i>required</i>
<code>result_type</code> Altair	<code>str</code>	Type of histogram result.	<i>required</i>

Other Parameters:

Name	Type	Description
<code>breakdown_by</code>	<code>str</code>	field to breakdown by. If None (default) then don't breakdown. If the original histogram has multiple breakdown fields, project all fields except the breakdown field of interest.
<code>cat_threshold</code>	<code>int</code>	number of categories to keep. In case the underlying cardinality is larger, keep the <code>cat_threshold</code> most frequent categories and create a new category 'other' as the aggregation of the rest of categories. The 'missing' category is always treated as a special category, meaning that it will not be merged into 'other' and it will be ignored when finding the <code>cat_threshold</code> most frequent categories. Default is None: keep all categories. Only relevant for string histograms.
<code>time_resolution</code>	<code>str</code>	project time field to given <code>time_resolution</code> . Default is None: keep underlying time resolution. Only relevant for time histograms.

Returns:

Type	Description
<code>DataFrame</code>	Reduced histogram.

reduce_numerical_histogram [↗](#)

```
reduce_numerical_histogram(histogram: DataFrame, breakdown_by: str | None = None, **kwargs) -> DataFrame
```

reduce_temporal_histogram [↗](#)

```
reduce_temporal_histogram(histogram: DataFrame, breakdown_by: str | None = None, time_resolution: str | None = None, **kwargs) -> DataFrame
```

temporal_aggregation [↗](#)

```
temporal_aggregation(df, time_field, aggregate, time_resolution='1D', is_timestamp_field=False)
```

Aggregates a dataframe with respect to time.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗		dataframe.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension.	<i>required</i>
<code>aggregate</code> ↗		aggregation function to be used. Can be any of <code>plots.SUPPORTED_AGGREGATIONS</code> .	<i>required</i>
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.Grouper.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> <code>_</code> . Default: '1D'.	'1D'

Name	Type	Description	Default
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False

Returns:

Type	Description
DataFrame	Pandas DataFrame with two columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The second column contains the values of the aggregation function and its name is the name of the aggregation function that was used.

temporal_aggregation_all [↗](#)

```
temporal_aggregation_all(df, time_field, time_resolution='1D', is_timestamp_field=False)
```

temporal_quantiles [↗](#)

```
temporal_quantiles(df_hist, time_field, is_timestamp_field=False, time_resolution='1W', half_bin_width=0)
```

Estimates the quantiles of a field with respect to time.

Parameters:

Name	Type	Description	Default
<code>df_hist</code> ↗		dataframe.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension.	<i>required</i>
<code>is_timestamp_field</code> ↗		boolean indicating if it is a datetime field (default: False).	False
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas . Grouper freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Default: '1W'.	'1W'
<code>half_bin_width</code> ↗		half of the bin width. It will be used to correct the quantile approximation to give the center of the bin. Default: 0 (no correction).	0

Returns:

Type	Description
DataFrame	Pandas DataFrame with six columns. One is the time field column, it has the value of time_field as the column name and contains the start datetime for each time window. The others contain the values of the min, max and the 25%, 50% and 75% quantiles. They are called min, 25%, median, 75% and max and appear in this order.

unpack_breakdowns [¶](#)

```
unpack_breakdowns(df, breakdowns)
```

Unpacks the DataFrame column 'breakdowns' into its components.

Parameters:

Name	Type	Description	Default
df ¶	DataFrame	a pandas DataFrame with a column named 'breakdowns' containing a tuple with the breakdown values.	<i>required</i>
breakdowns ¶	list	list of breakdown names.	<i>required</i>

Returns:

Type	Description
DataFrame	unpacked pandas DataFrame.

utils

friendly_type

```
friendly_type(datatype) -> str
```

Checks if the given pyspark datatype matches any of the supported types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> 		pyspark dataType.	<i>required</i>

Returns:

Type	Description
<code>str</code>	friendly type ('numerical', 'timestamp', 'string', 'unsupported').

get_pyspark_column_type

```
get_pyspark_column_type(df, column\_name)
```

Returns the type of a column with a given name in a Pyspark dataframe

Parameters:

Name	Type	Description	Default
<code>df</code> 		PySpark dataframe	<i>required</i>
<code>column_name</code> 		String containing the name of the column to get the type	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	a string with the data type of the column with column_name. If the column does not exist in the dataframe, does not return anything.

is_finite_col

```
is_finite_col(c: Column) -> Column
```

Column expression: not null, not NaN, not infinite.

Parameters:

Name	Type	Description	Default
<code>c</code> 	Column	The numerical column to evaluate.	<i>required</i>

Returns:

Type	Description
Column	A BooleanType Column expression that evaluates to True only for finite, non-null numerical values.

is_finite_number

```
is_finite_number(num)
```

Checks if num is a finite number.

Parameters:

Name	Type	Description	Default
<code>num</code> 		any object.	<i>required</i>

Returns:

Type	Description
unknown	True if num is a finite number.

is_number

```
is_number(num)
```

Checks if num is a number.

Parameters:

Name	Type	Description	Default
<code>num</code> 		any object.	<i>required</i>

Returns:

Type	Description
unknown	True if num is a number (int, float).

is_pandas_categorical

```
is_pandas_categorical(datatype)
```

Checks if the given pandas datatype matches our supported categorical types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pandas data type. This is a numpy.dtype and can be found with <code>df[].dtype</code> .	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported categorical types, False otherwise.

is_pandas_numerical [↗](#)

```
is_pandas_numerical(datatype)
```

Checks if the given pandas datatype matches our supported numerical types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pandas data type. This is a numpy.dtype and can be found with <code>df[].dtype</code> .	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported numerical types, False otherwise.

is_pandas_timestamp [↗](#)

```
is_pandas_timestamp(datatype)
```

Checks if the given pandas datatype matches our supported timestamp types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pandas data type. This is a numpy.dtype and can be found with <code>df[].dtype</code> .	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported timestamp types, False otherwise.

is_pyspark_numerical [↗](#)

```
is_pyspark_numerical(datatype)
```

Checks if the given pyspark datatype matches our supported numerical types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pyspark dataType.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported numerical types, False otherwise.

`is_pyspark_string` [↗](#)

```
is_pyspark_string(datatype)
```

Checks if the given pyspark datatype matches our supported string types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pyspark dataType.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported string types, False otherwise.

`is_pyspark_supported_type` [↗](#)

```
is_pyspark_supported_type(datatype)
```

Checks if the given pyspark datatype matches any of the supported types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pyspark dataType.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported types, False otherwise.

`is_pyspark_timestamp` [↗](#)

```
is_pyspark_timestamp(datatype)
```

Checks if the given pyspark datatype matches our supported timestamp types.

Parameters:

Name	Type	Description	Default
<code>datatype</code> ↗		pyspark dataType.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if matches supported timestamp types, False otherwise.

`is_string` [↗](#)

```
is_string(x)
```

Checks if x is a string.

Parameters:

Name	Type	Description	Default
<code>x</code> ↗		any object.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if x is a string.

`is_string_pair` [↗](#)

```
is_string_pair(x)
```

Checks if x is a pair of strings.

Parameters:

Name	Type	Description	Default
<code>x</code> ↗		any object.	<i>required</i>

Returns:

Type	Description
<code>unknown</code>	True if x is a tuple (string, string).

`print_with_timestamp` [↗](#)

```
print_with_timestamp(text)
```

validations

DEFAULT_VALIDATIONS module-attribute

```
DEFAULT_VALIDATIONS = [VAL_NEGATIVE_AMOUNT, VAL_ZERO_AMOUNT, VAL_HIGH_AMOUNT, VAL_FUTURE_DATETIME, VAL_PREHISTORIC_DATETIME]
```

VAL_FUTURE_DATETIME module-attribute

```
VAL_FUTURE_DATETIME = Validation(desc='datetime in future', semantic=TIMESTAMP, sql_builder=lambda c: count(when(c > current_timestamp(), lit(True))))
```

VAL_HIGH_AMOUNT module-attribute

```
VAL_HIGH_AMOUNT = Validation(desc='amount > 1e8', semantic=AMOUNT, sql_builder=lambda c: count(when(isNotNull() & ~isnan(c) & (c > 100000000.0), lit(True))))
```

VAL_NEGATIVE_AMOUNT module-attribute

```
VAL_NEGATIVE_AMOUNT = Validation(desc='amount < 0', semantic=AMOUNT, sql_builder=lambda c: count(when(isNotNull() & (c < 0), lit(True))))
```

VAL_PREHISTORIC_DATETIME module-attribute

```
VAL_PREHISTORIC_DATETIME = Validation(desc='ancient transaction (<1950)', semantic=TIMESTAMP, sql_builder=lambda c: count(when(isNotNull() & (c < lit(datetime(1950, 1, 1))), lit(True))))
```

VAL_ZERO_AMOUNT module-attribute

```
VAL_ZERO_AMOUNT = Validation(desc='amount == 0', semantic=AMOUNT, sql_builder=lambda c: count(when(isNotNull() & (c == 0), lit(True))))
```

Semantics

Bases: `object`

Set of pre-defined semantics.

AMOUNT class-attribute instance-attribute

```
AMOUNT = Tag('AMOUNT', NUMERICAL)
```

TARGET class-attribute instance-attribute

```
TARGET = Tag('TARGET', CATEGORICAL)
```

TIMESTAMP class-attribute instance-attribute

```
TIMESTAMP = Tag('TIMESTAMP', DATETIME)
```

Tag [↗](#)

```
Tag(desc, ftype)
```

Bases: `object`

A Tag or semantic consist of a descriptive tag (desc) and a field type (ftype) associated to it.

desc `instance-attribute` [↗](#)

```
desc = desc
```

ftype `instance-attribute` [↗](#)

```
ftype = ftype
```

Validation [↗](#)

```
Validation(desc: str, semantic: Tag, sql_builder: Callable[[Column], Column], warning_fn: Callable[[int], bool] = lambda x: x > 0)
```

Bases: `object`

Class that represents a descriptive validation to be computed. These validations will be computed in a map and reduce approach.

Parameters:

Name	Type	Description	Default
desc ↗	<code>str</code>	the description/name of the validation.	<i>required</i>
semantic ↗	<code>Tag</code>	the semantic type to be applied to, e.g. Tag("AMOUNT", FieldType.NUMERICAL).	<i>required</i>
sql_builder ↗	<code>Callable[[Column], Column]</code>	Factory that receives a Spark <code>Column</code> and returns the Spark SQL validation expression	<i>required</i>
warning_fn ↗	<code>Callable[[int], bool]</code>	the function to be used to asses when a validation should be alerted.	<code>lambda x: x > 0</code>

desc `instance-attribute` [↗](#)

```
desc = desc
```

semantic `instance-attribute` [↗](#)

```
semantic = semantic
```

sql_builder `instance-attribute` [↗](#)

```
sql_builder = sql_builder
```

warning_fn instance-attribute [¶](#)

```
warning_fn = warning_fn
```

build_validations_dict [¶](#)

```
build_validations_dict( semantics_map: dict[str, Tag], validations: Any | None = None) -> dict[str, list[Validation]]
```

Helper function that builds a dict with the validations to be performed per field.

Parameters:

Name	Type	Description	Default
semantics_map ¶	dict[str, Tag]	dictionary matching fields (keys) to semantic tags (values).	<i>required</i>
validations ¶	Any None	set/list/tuple containing list of Validation.	None

Returns:

Type	Description
unknown	dictionary with list of validations (values) associated to each field (keys).

check_validations_type [¶](#)

```
check_validations_type(validations, ftype, fname='')
```

Helper function to check if the list of validations matches the expected field type, otherwise it raises an exception.

Parameters:

Name	Type	Description	Default
validations ¶		list of Validation.	<i>required</i>
ftype ¶		FieldType.	<i>required</i>
fname ¶		(optional) field name.	''

show_validations_by_field [¶](#)

```
show_validations_by_field(semantics_map, validations=None)
```

Helper function that prints the validations to be performed per field given a semantics map and a list of validations.

Parameters:

Name	Type	Description	Default
semantics_map ¶		dictionary matching fields (keys) to semantic tags (values).	<i>required</i>

Name	Type	Description	Default
<code>validations</code> 		set/list/tuple containing list of Validation.	<code>None</code>

fair_gbm

FairGBMConfig

```
FairGBMConfig(constraint\_group\_column: str = '', constraint\_type: Optional[str] = 'FPR', global\_constraint\_type: Optional[str] = 'FPR,FNR', global\_target\_fpr: Optional[float] = 0.05, global\_target\_fnr: Optional[float] = 0.5, objective: Optional[str] = 'constrained_cross_entropy', constraint\_fpr\_threshold: Optional[float] = 0.0, constraint\_fnr\_threshold: Optional[float] = 0.0, multiplier\_learning\_rate: Optional[float] = 1000.0, init\_multipliers: Optional[str] = '', constraint\_stepwise\_proxy: Optional[str] = 'cross_entropy', objective\_stepwise\_proxy: Optional[str] = '', **kwargs)
```

Bases: [GenericLightGBMConfig](#)

The FairGBM (LightGBM with fairness constraints) configuration object used for model training.

Parameters:

Name	Type	Description	Default
constraint_group_column	str	(Fairness) Sensitive group column. Must be categorical.	' '
constraint_type	str	(Fairness) Constraint type. Either "FPR", "FNR", "FPR,FNR".	' FPR '
global_constraint_type	str	(Fairness) Global constraint type. Either "FPR", "FNR", "FPR,FNR".	' FPR, FNR '
global_target_fpr	float	(Fairness) Global target FPR.	0.05
global_target_fnr	float	(Fairness) Global target FNR.	0.5
objective	str	(Fairness) Objective function. Either "constrained_cross_entropy", "constrained_recall_objective".	' constrained_cross_entropy '
constraint_fpr_threshold	float	(Fairness) FPR slack for fairness.	0.0
constraint_fnr_threshold	float	(Fairness) FNR slack for fairness.	0.0
	float	(Fairness) Multipliers' learning rate.	1000.0

Name	Type	Description	Default
<code>multiplier_learning_rate</code> ¶			
<code>init_multipliers</code> ¶	str	(Fairness) Initial multipliers.	<code>''</code>
<code>constraint_stepwise_proxy</code> ¶	str	(Fairness) Stepwise proxy for fairness constraints. Either "cross_entropy", "quadratic", "hinge".	<code>'cross_entropy'</code>
<code>objective_stepwise_proxy</code> ¶	str	(Fairness) Stepwise proxy for global constraints. Either "cross_entropy", "quadratic", "hinge".	<code>''</code>

Other Parameters:

Name	Type	Description
<code>boosting_type</code>	str	Boosting type.
<code>num_iterations</code>	int	Number of booster iterations.
<code>learning_rate</code>	float	Learning rate.
<code>num_leaves</code>	int	Maximum tree leaves.
<code>lightgbm_seed</code>	int	Seed used to generate other seeds.
<code>max_depth</code>	int	Maximum tree depth.
<code>min_data_in_leaf</code>	int	Minimum leaf data points.
<code>min_sum_hessian_in_leaf</code>	float	Minimal sum of hessian per leaf.
<code>bagging_fraction</code>	float	Bagging sample fraction.
<code>bagging_freq</code>	int	Bagging frequency.
<code>feature_fraction</code>	float	Feature fraction by tree.
<code>feature_fraction_by_node</code>	float	Feature fraction by tree node.
<code>lambda_l1</code>	float	L1 regularization.
<code>lambda_l2</code>	float	L2 regularization.

Name	Type	Description
min_gain_to_split	float	Minimal gain to split a leaf.
drop_rate	float	DART drop rate.
max_drop	int	DART max drop.
skip_drop	float	DART skip drop.
xgboost_dart_mode	bool	DART XGBoost mode.
uniform_drop	bool	DART uniform drop.
top_rate	float	GOSS top retain rate.
other_rate	float	GOSS other retain rate.
min_data_per_group	int	Minimum data per group.
max_cat_threshold	int	Maximum categories thresholds.
cat_l2	float	Categorical L2 regularization.
cat_smooth	float	Categorical smoothing.
max_cat_to_onehot	int	Max categories to one-hot encode.
verbosity	int	Verbosity of the LightGBM backend.
max_bin	int	Max bins.
min_data_in_bin	int	Minimum bin data.
bin_construct_sample_cnt	int	Bin construction sample count.
enable_bundle	bool	Exclusive Feature Bundling (EFB).
zero_as_missing	bool	Use zeroes as missing.
is_unbalance	bool	Unbalanced label.
undersampling	float	The value for undersampling. Only this or oversampling can be used.
oversampling	float	

Name	Type	Description
		The value for oversampling. Only this or undersampling can be used.
seed	int	The seed value.
ignored_fields	set of str	Set of names corresponding to the fields to be ignored when training a model.
schema	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.

Examples:

```
>>> model_config = FairGBMConfig(
...     constraint_group_column= "age",
...     bagging_fraction=0.9,
...     undersampling=0.85
... )
>>> model_config.num_leaves # gets default value of num_leaves
>>> model_config.num_leaves = 10 # sets num_leaves to 10
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

bagging_fraction property writable [AI](#)

bagging_fraction

bagging_freq property writable [AI](#)

bagging_freq

bin_construct_sample_cnt property writable [AI](#)

bin_construct_sample_cnt

boosting_type property writable [AI](#)

boosting_type

cat_l2 property writable [AI](#)

cat_l2

cat_smooth property writable [AI](#)

cat_smooth

constraint_fnr_threshold property writable [AI](#)

constraint_fnr_threshold

constraint_fpr_threshold property writable [¶](#)

constraint_fpr_threshold

constraint_group_column property writable [¶](#)

constraint_group_column

constraint_stepwise_proxy property writable [¶](#)

constraint_stepwise_proxy

constraint_type property writable [¶](#)

constraint_type

drop_rate property writable [¶](#)

drop_rate

enable_bundle property writable [¶](#)

enable_bundle

feature_fraction property writable [¶](#)

feature_fraction

feature_fraction_bynode property writable [¶](#)

feature_fraction_bynode

global_constraint_type property writable [¶](#)

global_constraint_type

global_target_fnr property writable [¶](#)

global_target_fnr

global_target_fpr property writable [¶](#)

global_target_fpr

ignored_fields property writable [¶](#)

ignored_fields

init_multipliers property writable [¶](#)

init_multipliers

is_unbalance property writable [¶](#)

is_unbalance

lambda_l1 property writable [¶](#)

lambda_l1

lambda_l2 property writable [↗](#)

lambda_l2

learning_rate property writable [↗](#)

learning_rate

lightgbm_seed property writable [↗](#)

lightgbm_seed

max_bin property writable [↗](#)

max_bin

max_cat_threshold property writable [↗](#)

max_cat_threshold

max_cat_to_onehot property writable [↗](#)

max_cat_to_onehot

max_depth property writable [↗](#)

max_depth

max_drop property writable [↗](#)

max_drop

min_data_in_bin property writable [↗](#)

min_data_in_bin

min_data_in_leaf property writable [↗](#)

min_data_in_leaf

min_data_per_group property writable [↗](#)

min_data_per_group

min_gain_to_split property writable [↗](#)

min_gain_to_split

min_sum_hessian_in_leaf property writable [↗](#)

min_sum_hessian_in_leaf

multiplier_learning_rate property writable [↗](#)

multiplier_learning_rate

num_iterations property writable [↗](#)

num_iterations

num_leaves property writable [¶](#)

num_leaves

objective property writable [¶](#)

objective

objective_stepwise_proxy property writable [¶](#)

objective_stepwise_proxy

other_rate property writable [¶](#)

other_rate

oversampling property writable [¶](#)

oversampling

seed property writable [¶](#)

seed

skip_drop property writable [¶](#)

skip_drop

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

top_rate property writable [¶](#)

top_rate

undersampling property writable [¶](#)

undersampling

uniform_drop property writable [¶](#)

uniform_drop

verbosity property writable [¶](#)

verbosity

xgboost_dart_mode property writable [¶](#)

xgboost_dart_mode

zero_as_missing property writable [¶](#)

zero_as_missing

auto_cast staticmethod [¶](#)

auto_cast([config](#): [JavaWrapper](#)) -> [FairGBMConfig](#)

Creates a FairGBMConfig from the java object.

Parameters:

Name	Type	Description	Default
<code>config</code> ⓘ	<code>JavaWrapper</code>	Java configuration object.	<i>required</i>

Returns:

Type	Description
<code>GeneralizedLinearModelConfig</code>	Configuration object according to its python representation.

generic

GenericConfig

```
GenericConfig(provider: str, name: str, parameters: dict[str, str], undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

Training configuration object that can be used to train any supported algorithm in Pulse.

Parameters:

Name	Type	Description	Default
<code>provider</code>	<code>str</code>	Provider for the algorithm to be trained, for example "H2O".	<i>required</i>
<code>name</code>	<code>str</code>	Name of the algorithm to be trained, for example "Gradient Boosting Machine".	<i>required</i>
<code>parameters</code>	<code>dict[str, str]</code>		<i>required</i>
<code>undersampling</code>	<code>float</code>	The value for undersampling. Only this or oversampling can be used.	None
<code>oversampling</code>	<code>float</code>	The value for oversampling. Only this or undersampling can be used.	None
<code>seed</code>	<code>int</code>	The seed value.	None
<code>ignored_fields</code>	<code>set of str</code>	Set of names corresponding to the fields to be ignored when training a model.	None
<code>schema</code>	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> # training an H2O Gradient Boosting Machine
>>> model_config = GenericConfig(
...     name="Gradient Boosting Machine",
...     provider="H2O",
...     parameters={
...         "col_sample_rate": "0.9",
```

```
...     "max_depth": "5",
...     "learn_rate": "1.0",
...     "sample_rate": "1.0",
...     "nbins": "20",
...     "min_rows": "10.0",
...     "ntrees": "50",
... },
...     undersampling=0.98
... )
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

ignored_fields property writable [🔗](#)

ignored_fields

name property [🔗](#)

name

oversampling property writable [🔗](#)

oversampling

parameters property [🔗](#)

parameters

provider property [🔗](#)

provider

seed property writable [🔗](#)

seed

subclasses class-attribute instance-attribute [🔗](#)

subclasses: List[[JavaWrapper](#)] = []

undersampling property writable [🔗](#)

undersampling

auto_cast staticmethod [🔗](#)

auto_cast(config: [JavaWrapper](#)) -> [GenericConfig](#)

Creates a GenericConfig from the java object.

Parameters:

Name	Type	Description	Default
config 🔗	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
GenericConfig	Configuration object according to its python representation.

generic_light_gbm

GenericLightGBMConfig

```
GenericLightGBMConfig(builder, boosting_type: str = 'gbdt', num_iterations: int = 100, learning_rate: float = 0.1, num_leaves: int = 31, lightgbm_seed: int = 0, max_depth: int | None = None, min_data_in_leaf: int = 20, min_sum_hessian_in_leaf: float = 0.001, bagging_fraction: float = 1.0, bagging_freq: int = 0, feature_fraction: float = 1.0, feature_fraction_bynode: float = 1.0, lambda_l1: float = 0.0, lambda_l2: float = 0.0, min_gain_to_split: float = 0.0, drop_rate: float = 0.1, max_drop: int = 50, skip_drop: float = 0.5, xgboost_dart_mode: bool = False, uniform_drop: bool = False, top_rate: float = 0.2, other_rate: float = 0.1, min_data_per_group: int = 100, max_cat_threshold: int = 32, cat_l2: float = 10.0, cat_smooth: float = 10.0, max_cat_to_onehot: int = 4, verbosity: int = 1, max_bin: int = 255, min_data_in_bin: int = 3, bin_construct_sample_cnt: int = 200000, enable_bundle: bool = True, zero_as_missing: bool = False, is_unbalance: bool = False, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

Generic GBM configuration object. Serves as parent class for LightGBMConfig and FairGBMConfig.

Parameters:

Name	Type	Description	Default
builder		Builder that comes from the specific algorithm.	<i>required</i>
boosting_type	str	Boosting type.	'gbdt'
num_iterations	int	Number of booster iterations.	100
learning_rate	float	Learning rate.	0.1
num_leaves	int	Maximum tree leaves.	31
lightgbm_seed	int	Seed used to generate other seeds.	0
max_depth	int None	Maximum tree depth.	None
min_data_in_leaf	int	Minimum leaf data points.	20

Name	Type	Description	Default
<code>min_sum_hessian_in_leaf</code> ¶	float	Minimal sum of hessian per leaf.	0.001
<code>bagging_fraction</code> ¶	float	Bagging sample fraction.	1.0
<code>bagging_frequency</code> ¶	int	Bagging frequency.	0
<code>feature_fraction</code> ¶	float	Feature fraction by tree.	1.0
<code>feature_fraction_by_node</code> ¶	float	Feature fraction by tree node.	1.0
<code>lambda_l1</code> ¶	float	L1 regularization.	0.0
<code>lambda_l2</code> ¶	float	L2 regularization.	0.0
<code>min_gain_to_split</code> ¶	float	Minimal gain to split a leaf.	0.0
<code>drop_rate</code> ¶	float	DART drop rate.	0.1
<code>max_drop</code> ¶	int	DART max drop.	50
<code>skip_drop</code> ¶	float	DART skip drop.	0.5
<code>xgboost_dart_mode</code> ¶	bool	DART XGBoost mode.	False
<code>uniform_drop</code> ¶	bool	DART uniform drop.	False
<code>top_rate</code> ¶	float	GOSS top retain rate.	0.2
<code>other_rate</code> ¶	float	GOSS other retain rate.	0.1
	int	Minimum data per group.	100

Name	Type	Description	Default
<code>min_data_per_group</code> ↗			
<code>max_cat_thres hold</code> ↗	int	Maximum categories thresholds.	32
<code>cat_l2</code> ↗	float	Categorical L2 regularization.	10.0
<code>cat_smooth</code> ↗	float	Categorical smoothing.	10.0
<code>max_cat_to_on ehot</code> ↗	int	Max categories to one-hot encode.	4
<code>verbosity</code> ↗	int	Verbosity of the LightGBM backend.	1
<code>max_bin</code> ↗	int	Max bins.	255
<code>min_data_in_b in</code> ↗	int	Minimum bin data.	3
<code>bin_construct_ sample_cnt</code> ↗	int	Bin construction sample count.	200000
<code>enable_bundle</code> ↗	bool	Exclusive Feature Bundling (EFB).	True
<code>zero_as_missi ng</code> ↗	bool	Use zeroes as missing.	False
<code>is_unbalance</code> ↗	bool	Unbalanced label.	False
<code>undersampling</code> ↗	float	The value for undersampling. Only this or oversampling can be used.	None
<code>oversampling</code> ↗	float	The value for oversampling. Only this or undersampling can be used.	None
<code>seed</code> ↗	int	The seed value.	None

Name	Type	Description	Default
ignored_fields ↗	set of str	Set of names corresponding to the fields to be ignored when training a model.	None
schema ↗	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

bagging_fraction property writable [↗](#)

bagging_fraction

bagging_freq property writable [↗](#)

bagging_freq

bin_construct_sample_cnt property writable [↗](#)

bin_construct_sample_cnt

boosting_type property writable [↗](#)

boosting_type

cat_l2 property writable [↗](#)

cat_l2

cat_smooth property writable [↗](#)

cat_smooth

drop_rate property writable [↗](#)

drop_rate

enable_bundle property writable [↗](#)

enable_bundle

feature_fraction property writable [↗](#)

feature_fraction

feature_fraction_bynode property writable [↗](#)

feature_fraction_bynode

ignored_fields property writable [↗](#)

ignored_fields

is_unbalance property writable [↗](#)

is_unbalance

lambda_l1 property writable [↗](#)

lambda_l1

lambda_l2 property writable [↗](#)

lambda_l2

learning_rate property writable [↗](#)

learning_rate

lightgbm_seed property writable [↗](#)

lightgbm_seed

max_bin property writable [↗](#)

max_bin

max_cat_threshold property writable [↗](#)

max_cat_threshold

max_cat_to_onehot property writable [↗](#)

max_cat_to_onehot

max_depth property writable [↗](#)

max_depth

max_drop property writable [↗](#)

max_drop

min_data_in_bin property writable [↗](#)

min_data_in_bin

min_data_in_leaf property writable [↗](#)

min_data_in_leaf

min_data_per_group property writable [↗](#)

min_data_per_group

min_gain_to_split property writable [↗](#)

min_gain_to_split

min_sum_hessian_in_leaf property writable [↗](#)

min_sum_hessian_in_leaf

num_iterations property writable [↗](#)

num_iterations

num_leaves property writable [↗](#)

num_leaves

other_rate property writable [↗](#)

other_rate

oversampling property writable [↗](#)

oversampling

seed property writable [↗](#)

seed

skip_drop property writable [↗](#)

skip_drop

subclasses class-attribute instance-attribute [↗](#)

subclasses: List[[JavaWrapper](#)] = []

top_rate property writable [↗](#)

top_rate

undersampling property writable [↗](#)

undersampling

uniform_drop property writable [↗](#)

uniform_drop

verbosity property writable [↗](#)

verbosity

xgboost_dart_mode property writable [↗](#)

xgboost_dart_mode

zero_as_missing property writable [↗](#)

zero_as_missing

glm

GeneralizedLinearModelConfig

```
GeneralizedLinearModelConfig(lambda_search: bool = False, standardize: bool = True, max_after_balance_size: float = 5.0, max_hit_ratio_k: int = 0, compute_p_values: bool = False, balance_classes: bool = False, remove_collinear_columns: bool = False, missing_values_handling: str = 'MeanImputation', solver: str = 'AUTO', non_negative: bool = False, intercept: bool = True, link: str = 'family_default', beta_epsilon: float = 0.0001, theta: float = 1e-10, hglm: bool = False, calc_like: bool = False, max_active_predictors: int | None = None, prior: float | None = None, max_ite_rations: int | None = None, lambda_min_ratio: float | None = None, objective_epsilon: float | None = None, gradient_epsilon: float | None = None, obj_reg: float | None = None, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

The H2O Generalized Linear Model configuration object used for model training.

Parameters:

Name	Type	Description	Default
<code>lambda_search</code>	bool	Use lambda search starting at lambda max, given lambda is then interpreted as lambda min.	False
<code>standardize</code>	bool	Standardize numeric columns to have zero mean and unit variance.	True
<code>max_after_balance_size</code>	float	Maximum relative size of the training data after balancing class counts (can be less than 1.0). Requires balance_classes.	5.0
<code>max_hit_ratio_k</code>	int	Maximum number (top K) of predictions to use for hit ratio computation (for multi-class only, 0 to disable).	0
<code>compute_p_values</code>	bool	Request p-values computation, p-values work only with IRLSM solver and no regularization.	False
<code>balance_classes</code>	bool	Balance training data class counts via over/under-sampling (for imbalanced data).	False
	bool	In case of linearly dependent columns, remove some of the dependent columns.	False

Name	Type	Description	Default
<code>remove_collinear_columns</code> ¶			
<code>missing_values_handling</code> ¶	str	Handling of missing values. Either MeanImputation or Skip.	'MeanImputation'
<code>solver</code> ¶	str	AUTO will set the solver based on given data and the other parameters. IRLSM is fast on problems with small number of predictors and for lambda-search with L1 penalty, L_BFGS scales better for datasets with many columns.	'AUTO'
<code>non_negative</code> ¶	bool	Restrict coefficients (not intercept) to be non-negative.	False
<code>intercept</code> ¶	bool	Include constant term in the model.	True
<code>link</code> ¶	str	Link.	'family_default'
<code>beta_epsilon</code> ¶	float	Converge if beta changes less (using L-infinity norm) than beta esilon, ONLY applies to IRLSM solver.	0.0001
<code>max_active_predictors</code> ¶	int None	Maximum number of active predictors during computation. Use as a stopping criterion to prevent expensive model building with many predictors. Default indicates: If the IRLSM solver is used, the value of max_active_predictors is set to 5000 otherwise it is set to 100000000.	None
<code>prior</code> ¶	float None	Prior probability for y==1. To be used only for logistic regression if the data has been sampled and the mean of response does not reflect reality.	None
<code>max_iterations</code> ¶	int None	Maximum number of iterations.	None
<code>lambda_min_ratio</code> ¶	float None	Minimum lambda used in lambda search, specified as a ratio of lambda_max (the smallest lambda that drives all coefficients to zero). Default indicates: if the number of observations is greater than the number of variables, then lambda_min_ratio is set to 0.0001; If the number of observations is less than the number of variables, then lambda_min_ratio is set to 0.01.	None
<code>objective_epsilon</code> ¶	float None	Converge if objective value changes less than this. Default indicates: If lambda_search is set to True the value of objective_epsilon is set to .0001. If the lambda_search is set to False and lambda is equal to zero, the value of objective_epsilon is set to .000001, for any other value of lambda the default value of objective_epsilon is set to .0001.	None

Name	Type	Description	Default
gradient_epsilon ¶	float None	Converge if objective changes less (using L-infinity norm) than this, ONLY applies to L-BFGS solver. Default indicates: If lambda_search is set to False and lambda is equal to zero, the default value of gradient_epsilon is equal to .000001, otherwise the default value is .0001. If lambda_search is set to True, the conditional values above are 1E-8 and 1E-6 respectively.	None
obj_reg ¶	float None	Likelihood divider in objective value computation, default is 1/nobs.	None
theta ¶	float	Theta.	1e-10
hglm ¶	bool	If set to true, will return HGLM model. Otherwise, normal GLM model will be returned.	False
calc_likelihood ¶	bool	If true, will return likelihood function value for HGLM.	False
undersampling ¶	float	The value for undersampling. Only this or oversampling can be used.	None
oversampling ¶	float	The value for oversampling. Only this or undersampling can be used.	None
seed ¶	int	The seed value.	None
ignored_fields ¶	set of str	Set of names corresponding to the fields to be ignored when training a model.	None
schema ¶	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> model_config = GeneralizedLinearModelConfig()
>>> model_config.remove_collinear_columns
>>> model_config.remove_collinear_columns = True # sets it to True
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

balance_classes [property](#) [writable](#) [¶](#)

balance_classes

beta_epsilon property writable [↗](#)

beta_epsilon

calc_like property writable [↗](#)

calc_like

compute_p_values property writable [↗](#)

compute_p_values

gradient_epsilon property writable [↗](#)

gradient_epsilon

hglm property writable [↗](#)

hglm

ignored_fields property writable [↗](#)

ignored_fields

intercept property writable [↗](#)

intercept

lambda_min_ratio property writable [↗](#)

lambda_min_ratio

lambda_search property writable [↗](#)

lambda_search

link property writable [↗](#)

link

max_active_predictors property writable [↗](#)

max_active_predictors

max_after_balance_size property writable [↗](#)

max_after_balance_size

max_hit_ratio_k property writable [↗](#)

max_hit_ratio_k

max_iterations property writable [↗](#)

max_iterations

missing_values_handling property writable [↗](#)

missing_values_handling

non_negative property writable [⌕](#)

non_negative

obj_req property writable [⌕](#)

obj_req

objective_epsilon property writable [⌕](#)

objective_epsilon

oversampling property writable [⌕](#)

oversampling

prior property writable [⌕](#)

prior

remove_collinear_columns property writable [⌕](#)

remove_collinear_columns

seed property writable [⌕](#)

seed

solver property writable [⌕](#)

solver

standardize property writable [⌕](#)

standardize

subclasses class-attribute instance-attribute [⌕](#)

subclasses: List[[JavaWrapper](#)] = []

theta property writable [⌕](#)

theta

undersampling property writable [⌕](#)

undersampling

auto_cast staticmethod [⌕](#)

auto_cast([config](#): [JavaWrapper](#)) -> [GeneralizedLinearModelConfig](#)

Creates a GeneralizedLinearModelConfig from the java object.

Parameters:

Name	Type	Description	Default
config ⌕	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
GeneralizedLinearModelConfig	Configuration object according to its python representation.

isolation_forest

IsolationForestConfig

```
IsolationForestConfig(ntrees: int = 50, sample_size: int = 100, min_rows: int = 1, max_depth: int = 8, sample_rate: float = -1.0, mtries: int = -1, nbins: int = 2, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

The H2O Isolation Forest configuration object used for model training.

This is an unsupervised learning algorithm used for anomaly detection. The target variable will be automatically excluded from features during training.

Parameters:

Name	Type	Description	Default
<code>ntrees</code>	int	Number of trees in the forest, defaults to 50.	50
<code>sample_size</code>	int	Number of samples to use per tree, defaults to 100.	100
<code>min_rows</code>	int	Minimum number of rows in a leaf, defaults to 1.	1
<code>max_depth</code>	int	Maximum depth of trees, defaults to 8.	8
<code>sample_rate</code>	float	Row sampling rate. Use -1 for automatic, defaults to -1.0.	-1.0
<code>mtries</code>	int	Number of features to try per split. Use -1 for automatic, defaults to -1.	-1
<code>nbins</code>	int	Number of bins for numerical features, defaults to 2.	2
<code>seed</code>	int	The seed value.	None
<code>ignored_fields</code>	set of str	Set of names corresponding to the fields to be ignored when training a model.	None

Name	Type	Description	Default
<code>schema</code> ↗	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> model_config = IsolationForestConfig(
...     ntrees=100,
...     sample_size=500,
...     max_depth=10
... )
>>> model_config.ntrees # gets the ntrees: 100
>>> model_config.ntrees = 50 # sets the ntrees to 50
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field", # Will be excluded from features automatically
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

`ignored_fields` `property` `writable` [↗](#)

`ignored_fields`

`max_depth` `property` `writable` [↗](#)

`max_depth`

`min_rows` `property` `writable` [↗](#)

`min_rows`

`mtries` `property` `writable` [↗](#)

`mtries`

`nbins` `property` `writable` [↗](#)

`nbins`

`ntrees` `property` `writable` [↗](#)

`ntrees`

`oversampling` `property` `writable` [↗](#)

`oversampling`

`sample_rate` `property` `writable` [↗](#)

`sample_rate`

`sample_size` `property` `writable` [↗](#)

`sample_size`

seed property writable [¶](#)

seed

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

undersampling property writable [¶](#)

undersampling

auto_cast staticmethod [¶](#)

auto_cast([config](#): [JavaWrapper](#)) -> [IsolationForestConfig](#)

Creates an IsolationForestConfig from the java object.

Parameters:

Name	Type	Description	Default
config ¶	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
IsolationForestConfig	Configuration object according to its python representation.

light_gbm

LightGBMConfig

LightGBMConfig(**kwargs)

Bases: [GenericLightGBMConfig](#)

The Microsoft LightGBM configuration object used for model training.

Other Parameters:

Name	Type	Description
boosting_type	str	Boosting type.
num_iterations	int	Number of booster iterations.
learning_rate	float	Learning rate.
num_leaves	int	Maximum tree leaves.
lightgbm_seed	int	Seed used to generate other seeds.
max_depth	int	Maximum tree depth.
min_data_in_leaf	int	Minimum leaf data points.
min_sum_hessian_in_leaf	float	Minimal sum of hessian per leaf.
bagging_fraction	float	Bagging sample fraction.
bagging_freq	int	Bagging frequency.
feature_fraction	float	Feature fraction by tree.
feature_fraction_bynode	float	Feature fraction by tree node.
lambda_l1	float	L1 regularization.
lambda_l2	float	L2 regularization.
min_gain_to_split	float	Minimal gain to split a leaf.

Name	Type	Description
drop_rate	float	DART drop rate.
max_drop	int	DART max drop.
skip_drop	float	DART skip drop.
xgboost_dart_mode	bool	DART XGBoost mode.
uniform_drop	bool	DART uniform drop.
top_rate	float	GOSS top retain rate.
other_rate	float	GOSS other retain rate.
min_data_per_group	int	Minimum data per group.
max_cat_thres hold	int	Maximum categories thresholds.
cat_l2	float	Categorical L2 regularization.
cat_smooth	float	Categorical smoothing.
max_cat_to_on ehot	int	Max categories to one-hot encode.
verbosity	int	Verbosity of the LightGBM backend.
max_bin	int	Max bins.
min_data_in_b in	int	Minimum bin data.
bin_construct_ sample_cnt	int	Bin construction sample count.
enable_bundle	bool	Exclusive Feature Bundling (EFB).
zero_as_missi ng	bool	Use zeroes as missing.
is_unbalance	bool	Unbalanced label.
undersampling	float	The value for undersampling. Only this or oversampling can be used.
oversampling	float	The value for oversampling. Only this or undersampling can be used.
seed	int	

Name	Type	Description
		The seed value.
<code>ignored_fields</code>	set of str	Set of names corresponding to the fields to be ignored when training a model.
<code>schema</code>	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.

Examples:

```
>>> model_config = LightGBMConfig(
...     bagging_fraction=0.9,
...     undersampling=0.85
...)
>>> model_config.num_leaves # gets num_leaves: 31
>>> model_config.num_leaves = 10 # sets num_leaves to 10
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

`bagging_fraction` property writable [API](#)

```
bagging_fraction
```

`bagging_freq` property writable [API](#)

```
bagging_freq
```

`bin_construct_sample_cnt` property writable [API](#)

```
bin_construct_sample_cnt
```

`boosting_type` property writable [API](#)

```
boosting_type
```

`cat_l2` property writable [API](#)

```
cat_l2
```

`cat_smooth` property writable [API](#)

```
cat_smooth
```

`drop_rate` property writable [API](#)

```
drop_rate
```

`enable_bundle` property writable [API](#)

```
enable_bundle
```

feature_fraction property writable [↗](#)

feature_fraction

feature_fraction_bynode property writable [↗](#)

feature_fraction_bynode

ignored_fields property writable [↗](#)

ignored_fields

is_unbalance property writable [↗](#)

is_unbalance

lambda_l1 property writable [↗](#)

lambda_l1

lambda_l2 property writable [↗](#)

lambda_l2

learning_rate property writable [↗](#)

learning_rate

lightgbm_seed property writable [↗](#)

lightgbm_seed

max_bin property writable [↗](#)

max_bin

max_cat_threshold property writable [↗](#)

max_cat_threshold

max_cat_to_onehot property writable [↗](#)

max_cat_to_onehot

max_depth property writable [↗](#)

max_depth

max_drop property writable [↗](#)

max_drop

min_data_in_bin property writable [↗](#)

min_data_in_bin

min_data_in_leaf property writable [↗](#)

min_data_in_leaf

min_data_per_group property writable [⌕](#)

min_data_per_group

min_gain_to_split property writable [⌕](#)

min_gain_to_split

min_sum_hessian_in_leaf property writable [⌕](#)

min_sum_hessian_in_leaf

num_iterations property writable [⌕](#)

num_iterations

num_leaves property writable [⌕](#)

num_leaves

other_rate property writable [⌕](#)

other_rate

oversampling property writable [⌕](#)

oversampling

seed property writable [⌕](#)

seed

skip_drop property writable [⌕](#)

skip_drop

subclasses class-attribute instance-attribute [⌕](#)

subclasses: List[[JavaWrapper](#)] = []

top_rate property writable [⌕](#)

top_rate

undersampling property writable [⌕](#)

undersampling

uniform_drop property writable [⌕](#)

uniform_drop

verbosity property writable [⌕](#)

verbosity

xgboost_dart_mode property writable [⌕](#)

xgboost_dart_mode

`zero_as_missing` `property` `writable` [¶](#)

`zero_as_missing`

`auto_cast` `staticmethod` [¶](#)

```
auto_cast(config: JavaWrapper) -> LightGBMConfig
```

Creates a LightGBMConfig from the java object.

Parameters:

Name	Type	Description	Default
<code>config</code> ¶	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
GeneralizedLinearModelConfig	Configuration object according to its python representation.

random_forest

RandomForestConfig

```
RandomForestConfig(num_trees: int = 100, max_depth: int = 20, split_criteria: str = 'One vs others', min_instance_split: int = 2, in_bag: float = 0.65, features_per_split: int = 0, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

The Feedzai Random Forest configuration object used for model training.

Parameters:

Name	Type	Description	Default
<code>num_trees</code>	int	The number of trees, defaults to 100.	100
<code>max_depth</code>	int	The depth of each tree, defaults to 20.	20
<code>split_criteria</code>	str	Split criteria for categorical values, can be "All vs all", or "One vs others". Defaults to "One vs others".	'One vs others'
<code>min_instance_split</code>	int	The minimum number of split instances, defaults to 2.	2
<code>in_bag</code>	float	The ratio used for sampling data into the training, defaults to 0.65.	0.65
<code>features_per_split</code>	int	The number of features per split, defaults to 0 which represents sqrt(N).	0
<code>undersampling</code>	float	The value for undersampling. Only this or oversampling can be used.	None
<code>oversampling</code>	float	The value for oversampling. Only this or undersampling can be used.	None
<code>seed</code>	int	The seed value.	None
	set of str	Set of names corresponding to the fields to be ignored when training a model.	None

Name	Type	Description	Default
ignored_fields ↗			
schema ↗	Schema	Pass a schema to enable conversion between descscs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> model_config = RandomForestConfig(
...     num_trees=200,
...     in_bag=0.5,
...     undersampling=0.98
... )
>>> model_config.num_trees # gets the num_trees: 200
>>> model_config.num_trees = 100 # sets the num_trees to 100
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

features_per_split property writable [↗](#)

features_per_split

ignored_fields property writable [↗](#)

ignored_fields

in_bag property writable [↗](#)

in_bag

max_depth property writable [↗](#)

max_depth

min_instance_split property writable [↗](#)

min_instance_split

num_trees property writable [↗](#)

num_trees

oversampling property writable [↗](#)

oversampling

seed property writable [↗](#)

seed

split_criteria property writable [¶](#)

split_criteria

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

undersampling property writable [¶](#)

undersampling

auto_cast staticmethod [¶](#)

auto_cast([config](#): [JavaWrapper](#)) -> [RandomForestConfig](#)

Creates a RandomForestConfig from the java object.

Parameters:

Name	Type	Description	Default
config ¶	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
RandomForestConfig	Configuration object according to its python representation.

xgboost

XGBoostConfig

```
XGBoostConfig(reg_alpha: float = 0.0, max_bins: int = 256, dmatrix_type: str = 'auto', skip_drop: float = 0.0, min_child_weight: float = 1.0, tree_method: str = 'auto', booster: str = 'gbtree', col_sample_rate: float = 1.0, max_delta_step: float = 0.0, max_leaves: int = 0, reg_lambda: float = 1.0, quiet_mode: bool = True, backend: str = 'auto', nthread: int | None = None, min_in_data_in_leaf: float = 0.0, rate_drop: float = 0.0, min_rows: float = 1.0, col_sample_rate_per_tree: float = 1.0, ntrees: int = 50, gpu_id: int = 0, learn_rate: float = 0.3, colsample_bytree: float = 1.0, min_split_improvement: float = 0.0, sample_rate: float = 1.0, eta: float = 0.3, subsample: float = 1.0, max_abs_leafnode_pred: float = 0.0, gamma: float = 0.0, max_depth: int = 6, colsample_bylevel: float = 1.0, score_tree_interval: int = 0, normalize_type: str = 'tree', min_sum_hessian_in_leaf: float = 100.0, grow_policy: str = 'depthwise', one_drop: bool = False, sample_type: str = 'uniform', build_tree_one_node: bool = False, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

The H2O XGBoost configuration object used for model training.

Parameters:

Name	Type	Description	Default
<code>reg_alpha</code>	float	L1 regularization.	0.0
<code>max_bins</code>	int	For tree_method=hist only: maximum number of bins.	256
<code>dmatrix_type</code>	str	Type of DMatrix. For sparse, NAs and 0 are treated equally.	'auto'
<code>skip_drop</code>	float	For booster=dart only: skip_drop (0..1).	0.0
<code>min_child_weight</code>	float	(same as min_rows) Fewest allowed (weighted) observations in a leaf.	1.0
<code>tree_method</code>	str	Tree method.	'auto'
<code>booster</code>	str	Booster type.	'gbtree'
	float	(same as colsample_bylevel) Column sample rate (from 0.0 to 1.0).	1.0

Name	Type	Description	Default
<code>col_sample_rate</code> ↗			
<code>max_delta_step</code> ↗	float	(same as max_abs_leafnode_pred) Maximum absolute value of a leaf node prediction.	0.0
<code>max_leaves</code> ↗	int	For tree_method=hist only: maximum number of leaves.	0
<code>reg_lambda</code> ↗	float	L2 regularization.	1.0
<code>quiet_mode</code> ↗	bool	Enable quiet mode.	True
<code>backend</code> ↗	str	Backend. By default (auto), a GPU is used if available..	'auto'
<code>nthread</code> ↗	int None	Number of parallel threads that can be used to run XGBoost. Cannot exceed H2O cluster limits (-nthreads parameter). Defaults to maximum available.	None
<code>min_data_in_leaf</code> ↗	float	For tree_method=hist only: the minimum data in a leaf to keep splitting.	0.0
<code>rate_drop</code> ↗	float	For booster=dart only: rate_drop (0..1).	0.0
<code>min_rows</code> ↗	float	(same as min_child_weight) Fewest allowed (weighted) observations in a leaf.	1.0
<code>col_sample_rate_per_tree</code> ↗	float	(same as colsample_bytree) Column sample rate per tree (from 0.0 to 1.0).	1.0
<code>ntrees</code> ↗	int	(same as n_estimators) Number of trees.	50
<code>gpu_id</code> ↗	int	Which GPU to use.	0
<code>learn_rate</code> ↗	float	(same as eta) Learning rate (from 0.0 to 1.0).	0.3
<code>colsample_bytree</code> ↗	float	(same as col_sample_rate_per_tree) Column sample rate per tree (from 0.0 to 1.0).	1.0

Name	Type	Description	Default
<code>min_split_improvement</code> ¶	float	(same as gamma) Minimum relative improvement in squared error reduction for a split to happen.	0.0
<code>sample_rate</code> ¶	float	(same as subsample) Row sample rate per tree (from 0.0 to 1.0).	1.0
<code>eta</code> ¶	float	(same as learn_rate) Learning rate (from 0.0 to 1.0).	0.3
<code>sub_sample</code> ¶	float	(same as sample_rate) Row sample rate per tree (from 0.0 to 1.0).	1.0
<code>max_abs_leaf_node_pred</code> ¶	float	(same as max_delta_step) Maximum absolute value of a leaf node prediction.	0.0
<code>gamma</code> ¶	float	(same as min_split_improvement) Minimum relative improvement in squared error reduction for a split to happen.	0.0
<code>max_depth</code> ¶	int	Maximum tree depth.	6
<code>colsample_by_level</code> ¶	float	(same as col_sample_rate) Column sample rate (from 0.0 to 1.0).	1.0
<code>score_tree_interval</code> ¶	int	Score the model after every so many trees. Disabled if set to 0.	0
<code>normalize_type</code> ¶	str	For booster=dart only: normalize_type.	'tree'
<code>min_sum_hessian_in_leaf</code> ¶	float	For tree_method=hist only: the minimum sum of hessian in a leaf to keep splitting.	100.0
<code>grow_policy</code> ¶	str	Grow policy - depthwise is standard GBM, lossguide is LightGBM.	'depthwise'
<code>one_drop</code> ¶	bool	For booster=dart only: one_drop.	False
<code>sample_type</code> ¶	str	For booster=dart only: sample_type.	'uniform'
	bool		False

Name	Type	Description	Default
<code>build_tree_one_node</code> ¶		Run on one node only.	
<code>undersampling</code> ¶	float	The value for undersampling. Only this or oversampling can be used.	None
<code>oversampling</code> ¶	float	The value for oversampling. Only this or undersampling can be used.	None
<code>seed</code> ¶	int	The seed value.	None
<code>ignored_fields</code> ¶	set of str	Set of names corresponding to the fields to be ignored when training a model.	None
<code>schema</code> ¶	Schema	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> model_config = XGBoostConfig()
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

`backend` `property` `writable` [¶](#)

backend

`booster` `property` `writable` [¶](#)

booster

`build_tree_one_node` `property` `writable` [¶](#)

build_tree_one_node

`col_sample_rate` `property` `writable` [¶](#)

col_sample_rate

`col_sample_rate_per_tree` `property` `writable` [¶](#)

col_sample_rate_per_tree

colsample_bylevel property writable [↗](#)

colsample_bylevel

colsample_bytree property writable [↗](#)

colsample_bytree

dmatrix_type property writable [↗](#)

dmatrix_type

eta property writable [↗](#)

eta

gamma property writable [↗](#)

gamma

gpu_id property writable [↗](#)

gpu_id

grow_policy property writable [↗](#)

grow_policy

ignored_fields property writable [↗](#)

ignored_fields

learn_rate property writable [↗](#)

learn_rate

max_abs_leafnode_pred property writable [↗](#)

max_abs_leafnode_pred

max_bins property writable [↗](#)

max_bins

max_delta_step property writable [↗](#)

max_delta_step

max_depth property writable [↗](#)

max_depth

max_leaves property writable [↗](#)

max_leaves

min_child_weight property writable [↗](#)

min_child_weight

min_data_in_leaf property writable [¶](#)

min_data_in_leaf

min_rows property writable [¶](#)

min_rows

min_split_improvement property writable [¶](#)

min_split_improvement

min_sum_hessian_in_leaf property writable [¶](#)

min_sum_hessian_in_leaf

normalize_type property writable [¶](#)

normalize_type

nthread property writable [¶](#)

nthread

ntrees property writable [¶](#)

ntrees

one_drop property writable [¶](#)

one_drop

oversampling property writable [¶](#)

oversampling

quiet_mode property writable [¶](#)

quiet_mode

rate_drop property writable [¶](#)

rate_drop

reg_alpha property writable [¶](#)

reg_alpha

reg_lambda property writable [¶](#)

reg_lambda

sample_rate property writable [¶](#)

sample_rate

sample_type property writable [¶](#)

sample_type

score_tree_interval property writable [¶](#)

score_tree_interval

seed property writable [¶](#)

seed

skip_drop property writable [¶](#)

skip_drop

sub_sample property writable [¶](#)

sub_sample

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

tree_method property writable [¶](#)

tree_method

undersampling property writable [¶](#)

undersampling

auto_cast staticmethod [¶](#)

auto_cast(config: [JavaWrapper](#)) -> [XGBoostConfig](#)

Creates a XGBoostConfig from the java object.

Parameters:

Name	Type	Description	Default
config ¶	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
XGBoostConfig	Configuration object according to its python representation.

connection

Class to represent the connection between Plan Operators.

Connection

```
Connection(operator_in: BaseOperator, operator_out: BaseOperator)
```

Represents a connection between two plan operators.

Parameters:

Name	Type	Description	Default
<code>operator_in</code> 	<code>BaseOperator</code>	Operator to connect from	<i>required</i>
<code>operator_out</code> 	<code>BaseOperator</code>	Operator to connect to	<i>required</i>

`operator_in` instance-attribute

```
operator_in = require_type(operator_in, BaseOperator)
```

`operator_out` instance-attribute

```
operator_out = require_type(operator_out, BaseOperator)
```

`get_names`

```
get_names() -> Tuple[str, str]
```

Gets the names of the connected operators.

Returns:

Type	Description
<code>tuple of str</code>	Tuple with the operator names

`validate_op_exists`

```
validate_op_exists(plan: Plan)
```

Validates the Connection operators existing in the plan.

Parameters:

Name	Type	Description	Default
<code>plan</code> 	<code>Plan</code>	Plan	<i>required</i>

Raises:

Type	Description
ValueError	If the plan does not contain the connection operators.

csv_headers

Module containing the headers used in the tabular version of features.

CSV_FILTER_OP_SCHEMA module-attribute

```
CSV_FILTER_OP_SCHEMA = Enum('CSV_OP_SCHEMA', FILTER\_OP\_HEADERS)
```

CSV_LIST_OP_SCHEMA module-attribute

```
CSV_LIST_OP_SCHEMA = Enum('CSV_OP_SCHEMA', LIST\_OP\_HEADERS)
```

CSV_MAP_OP_SCHEMA module-attribute

```
CSV_MAP_OP_SCHEMA = Enum('CSV_OP_SCHEMA', MAP\_OP\_HEADERS)
```

CSV_METRIC_OP_SCHEMA module-attribute

```
CSV_METRIC_OP_SCHEMA = Enum('CSV_OP_SCHEMA', METRIC\_OP\_HEADERS)
```

FILTER_OP_HEADERS module-attribute

```
FILTER_OP_HEADERS: list[str] = OP\_HEADERS + ['expression']
```

LIST_OP_HEADERS module-attribute

```
LIST_OP_HEADERS: list[str] = OP\_HEADERS + ['description', 'expression', 'list_operation', 'list_id', 'list_field', 'list_timestamp_field']
```

MAP_OP_HEADERS module-attribute

```
MAP_OP_HEADERS: list[str] = OP\_HEADERS + ['description', 'expression', 'map_field_type', 'map_field_config', 'map_remove_fields']
```

METRIC_OP_HEADERS module-attribute

```
METRIC_OP_HEADERS: list[str] = OP\_HEADERS + ['description', 'metric_groupby_fields', 'metric_aggregation', 'metric_window_size', 'metric_is_real_time', 'metric_scheduled_cron_expression', 'metric_filter', 'metric_delay', 'metric_is_tumbling', 'metric_is_infinite', 'metric_filter_incomplete_instances', 'metric_timestamp_field', 'metric_lookup_fields', 'metric_main_channel_contributes', 'metric_cascaded_scheduled_metrics', 'metric_coalesce', 'metric_secondary_channels']
```

OP_HEADERS module-attribute

```
OP_HEADERS: list[str] = ['operator_name', 'operator_type', 'feature_name']
```

PLAN_HEADERS module-attribute

```
PLAN_HEADERS: list[str] = list(fromkeys(OP_HEADERS + MAP_OP_HEADERS + METRIC_OP_HEADERS + FILTER_OP_HEADERS + LIST_OP_HEADERS))
```

diff

Module to enable Plans diff creation.

DIFF_NAME_PLACEHOLDER

```
DIFF_NAME_PLACEHOLDER = '{plan_a}_diff_{plan_b}_{datetime:%Y%m%d%H%M%S}'
```

LOGGER

```
LOGGER = getLogger(__name__)
```

PlanDiff

```
PlanDiff(plan_a: Plan, plan_b: Plan)
```

Encapsulates the logic to find the difference between two plans.

Note

The generated diff plan allows the difference between two plans to be executed. This means it not only has the diff but also all the relevant ancestors required to make the plan valid, as well as any downstream features that would be impacted by upstream changes.

Let's say feature X changed. We need to include:

- Feature X's ancestors to allow the computation of X.
- Feature X's descendants because they need to be updated.
- Feature X's descendants ancestors to make the plan valid.

Parameters:

Name	Type	Description	Default
plan_a	Plan	Base plan for the difference.	required
plan_b	Plan	Secondary/"new" plan.	required

features_to_keep

```
features_to_keep: FeatureContainer = _diff()
```

new_feature_dependencies

```
new_feature_dependencies: FeatureContainer
```

new_features

```
new_features: FeatureContainer
```

to_pulse [¶](#)

```
to_pulse(name: Optional[str] = None) -> Plan
```

Create the diff plan and persist it in Pulse.

If no name is given, it will create a plan named after the two plans used to create the diff: `{plan_a}_diff_{plan_b}_{timestamp}`.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the diff plan.	<code>None</code>

Returns:

Type	Description
<code>Plan</code>	The diff plan.

validate_plan_compatibility [¶](#)

```
validate_plan_compatibility(a: Plan, b: Plan)
```

Validate if two plans are compatible.

Two conditions must be met for two plans to be compatible:

1. Same schema
2. Same timestamp field.

Parameters:

Name	Type	Description	Default
<code>a</code> ¶	<code>Plan</code>	One plan.	<i>required</i>
<code>b</code> ¶	<code>Plan</code>	Another plan.	<i>required</i>

Raises:

Type	Description
<code>ValueError</code>	When plans are incompatible.

features

omnichannel

Module with specific Omnichannel tooling.

New feature

Omnichannel plan manipulation is a new feature of the ds-api.

LOGGER

module-attribute

```
LOGGER = getLogger(__name__)
```

PlanChannelConfig

module-attribute

```
PlanChannelConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.PlanChannelConfig')
```

SchemaModel

module-attribute

```
SchemaModel = autoclass(MODEL_CLASSPATH + 'SchemaModel')
```

Channel

dataclass

```
Channel(name: str, schema_model: JavaWrapper, timestamp_field: str, id: str | None = None)
```

Immutable representation of a Channel configuration for Omnichannel Plans.

Note that this can be instantiated with a `SchemaModel` directly or with a `ds_api.schema.Schema`.

Examples:

```
>>> from ds_api.plans.omnichannel import Channel
>>> app = ...
>>> schema = app.pullSchemaByDesc("Cards Schema")
>>> channel = Channel("cards", schema, "event_occurred_at")
```

Attributes:

Name	Type	Description
<code>name</code>	<code>str</code>	Name of the channel.
<code>schema_model</code>	<code>SchemaModel</code>	Simple schema representation.
<code>timestamp_field</code>	<code>str</code>	The name of the timestamp field as it appears in the UI.
<code>id</code>	<code>(str, optional)</code>	Unique identifier for the channel. If none is passed, the ID is automatically generated when interacting with Pulse.

id

class-attribute instance-attribute

```
id: str | None = None
```


name instance-attribute [¶](#)

```
name: str
```

schema_model instance-attribute [¶](#)

```
schema_model: JavaWrapper
```

timestamp_field instance-attribute [¶](#)

```
timestamp_field: str
```

from_config classmethod [¶](#)

```
from_config(config: JavaWrapper) -> Channel
```

Create a Channel from a Pulse `PlanChannelConfig` .

to_config [¶](#)

```
to_config() -> JavaWrapper
```

Return this Channel as a Pulse `PlanChannelConfig` .

operators

utils [↗](#)

Utilities for plan handling.

LOGGER module-attribute [↗](#)

```
LOGGER = getLogger(__name__)
```

FeatureUID [↗](#)

Bases: `NamedTuple`

Tuple containing a unique representation of a feature based on the feature and operator names.

name instance-attribute [↗](#)

```
name: str
```

op_name instance-attribute [↗](#)

```
op_name: Optional[str]
```

removed class-attribute instance-attribute [↗](#)

```
removed: bool = False
```

is_field_removed [↗](#)

```
is_field_removed() -> bool
```

Returns True if the FeatureUID is created from a field being removed in a MapOperator.

is_schema_field [↗](#)

```
is_schema_field() -> bool
```

True if the FeatureUID represents a schema field.

HtmlDiff [↗](#)

diff_dict staticmethod [↗](#)

```
diff_dict(d1: Dict, d2: Dict) -> Dict
```

Finds the dictionaries with items that contain different values.

diff_dicts_html [↗](#)

```
diff_dicts_html(dict1: List[Dict], dict2: List[Dict], join_keys: List[str], headers: List[str] | None = None, only_diff: bool = True) -> str
```

Finds the differences between two lists of dictionaries.

Differences can be either updates, additions or deletions and are formatted side by side in an HTML table. Updates take place if the values of 'join_keys' are equal.

Parameters:

Name	Type	Description	Default
<code>dicts_1</code> ¶	list of dict	List of dicts to be compared.	<i>required</i>
<code>dicts_2</code> ¶	list of dict	List of dicts to be compared.	<i>required</i>
<code>join_keys</code> ¶	list of str	Keys to pair dictionaries in both lists and evaluate equality.	<i>required</i>
<code>headers</code> ¶	list, optional, by default None	Headers to be displayed on the diff table, length 2.	None
<code>only_diff</code> ¶	bool, by default True.	Report only dictionaries with changes.	True

Returns:

Type	Description
str	Differences in HTML formatted code.

WindowInterval [¶](#)

```
WindowInterval(window_expression: str)
```

Manipulate window intervals/expressions. Mostly to generate metric names.

duration [class-attribute](#) [instance-attribute](#) [¶](#)

```
duration: Optional[int] = None
```

is_simple [property](#) [¶](#)

```
is_simple: bool
```

True when the window definition follows the simple format of ''.

unit [class-attribute](#) [instance-attribute](#) [¶](#)

```
unit: Optional[str] = None
```

window_expression [instance-attribute](#) [¶](#)

```
window_expression = require_type(window_expression, str, 'window_expression')
```

get_name [¶](#)

```
get_name() -> Optional[str]
```

Return the normalized name of the window expression.

Expressions that do not follow the '' pattern do not get a name.

Returns:

Type	Description
<code>str</code>	Normalized name of the window expression.

`get_normed_time_unit` staticmethod [¶](#)

```
get_normed_time_unit(time_unit) -> Optional[str]
```

Given a valid `time_unit` it returns the normalized string.

`has_simple_tokens` [¶](#)

```
has_simple_tokens() -> bool
```

True when there are two tokens, ie, the window expression follows the simple format ' '.

`has_valid_simple_duration` [¶](#)

```
has_valid_simple_duration() -> bool
```

True when the duration of a simple window expression is valid.

`has_valid_simple_unit` [¶](#)

```
has_valid_simple_unit() -> bool
```

True when the unit of a simple window expression is valid.

`tokenize` [¶](#)

```
tokenize() -> List[str]
```

Return a sequence of tokens from the window expression.

`deduplicate_names` [¶](#)

```
deduplicate_names(names: list[str]) -> list[str]
```

Deduplicate the names in the list by appending '_n+1'.

Parameters:

Name	Type	Description	Default
<code>names</code> ¶	<code>list</code>	List of names.	<i>required</i>

Returns:

Type	Description
<code>str</code>	Deduplicated list of names.

`in_jupyter` [¶](#)

```
in_jupyter() -> bool
```

Returns true if run from a jupyter notebook/lab.

parse_literal [↗](#)

```
parse_literal(val: Any | None) -> Any
```

Given a value either in a primitive type or as a literal it will return the correct type.

validate_plan_is_deployable [↗](#)

```
validate_plan_is_deployable(func)
```

Validates the Structure of a Plan is supported.

Raises:

Type	Description
<code>NotImplementedError</code>	If the PlanStructure type is not supported

base_sampler

If you want to implement a specific sampling algorithm, you should implement the abstract class :class: BaseSampler .

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

MARK_FIELD module-attribute

```
MARK_FIELD = 'fdz_sample_ismarked'
```

MS_PER_DAY module-attribute

```
MS_PER_DAY = 24 * 60 * 60 * 1000
```

S module-attribute

```
S = TypeVar('S')
```

T module-attribute

```
T = TypeVar('T')
```

TargetType module-attribute

```
TargetType = bool | int | str
```

BaseSampler

```
BaseSampler(seed: int | None = None)
```

Bases: ABC

Base class for sampling implementations.

All subclasses must implement the public :meth: sample method.

Any additional configurations that your custom sampler needs, should be passed during instantiation.

Parameters:

Name	Type	Description	Default
seed	int	Seed for Spark's random number generator.	None

Examples:

```
>>> class RandomSampler(BaseSampler):
...     '''Custom random sampler.'''
...     def __init__(self, fraction: float, seed: Optional[int] = None):
```

```

...     self._fraction = fraction
...     super().__init__(seed=seed)
...     def sample(self, df: DataFrame) -> DataFrame:
...         return df.sample(fraction=self._fraction)
>>> # Initialize the custom sampler
>>> random_sampler_lpc = RandomSampler(0.01)
>>> # Run it against some data
>>> df = ...
>>> random_sampler_lpc.sample(df)

```

TIMESTAMP_MS_FIELD class-attribute instance-attribute [⌕](#)

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

seed property [⌕](#)

```
seed: int
```

sample abstractmethod [⌕](#)

```
sample(df: _TDataFrame) -> _TDataFrame
```

Sample the input data.

Parameters:

Name	Type	Description	Default
df ⌕	<code>_TDataFrame</code>	Input data source.	<i>required</i>

full_recovery [⌕](#)

```
full_recovery(marked_df: _TDataFrame, entity_field: str, recover_column_name: str, mark_field: str = MARK_FIELD)
```

Performs full entity recovery over a sampled DataFrame.

That is, it recovers **all** original instances that belong to any of the sampled entities.

Parameters:

Name	Type	Description	Default
marked_df ⌕	<code>_TDataFrame</code>	Original pyspark DataFrame.	<i>required</i>
entity_field ⌕	<code>str</code>	Name of the entity field.	<i>required</i>
recover_column_name ⌕	<code>str</code>	Name of the column marking instances to be retrieved.	<i>required</i>
mark_field ⌕	<code>str</code>	Name of the field marking sampled instances.	<code>MARK_FIELD</code>

Returns:

Type	Description
DataFrame	A pyspark DataFrame containing all relevant instances.

generate_seed [↗](#)

```
generate_seed() -> int
```

Generate a random seed based on the timestamp in milliseconds.

optimal_recovery [↗](#)

```
optimal_recovery(marked_df: _TDataFrame, entity_field: str, timestamp_field: str, recovery_days: int, recover_column_name: str, mark_field: str = MARK_FIELD) -> _TDataFrame
```

Performs optimal entity recovery over a sampled DataFrame.

That is, for a given sampled instance, it recovers the instances belonging to the same entity and falling in a relevant time period. The relevant period of time is a window before the sampled instance, defined by the recovery_days parameter.

Parameters:

Name	Type	Description	Default
marked_df ↗	_TDataFrame	Original pyspark DataFrame.	<i>required</i>
entity_field ↗	str	Name of the entity field.	<i>required</i>
timestamp_field ↗	str	Name of timestamp field.	<i>required</i>
recovery_days ↗	int	Size of the recovery period. Typically this value should correspond to the maximum window size of your plan's metrics operator.	<i>required</i>
recover_column_name ↗	str	Name of the column marking instances to be retrieved.	<i>required</i>
mark_field ↗	str	Name of the field marking sampled instances.	MARK_FIELD

Returns:

Type	Description
DataFrame	A pyspark DataFrame containing all relevant instances from the original.

recover_entities [↗](#)

```
recover_entities(marked_df: _TDataFrame, entity_fields: dict[str, int | None], mark_field: str = MARK_FIELD, is_optimal_recovery: bool = True, timestamp_field: str | None = None) -> _TDataFrame
```

Performs entity recovery over a sampled DataFrame.

That is, it recovers the relevant original instances that belong to any of the sampled entities.

Parameters:

Name	Type	Description	Default
<code>marked_df</code> ¶	<code>_TDataFrame</code>	Original pyspark DataFrame. It should have a column marking which events were sampled.	<i>required</i>
<code>entity_fields</code> ¶	<code>dict[str, int None]</code>	Dictionary with names and respective recovery period (can be None) for each field to be recovered.	<i>required</i>
<code>mark_field</code> ¶	<code>str</code>	Name of the field marking sampled instances.	MARK_FIELD
<code>is_optimal_recovery</code> ¶	<code>bool</code>	Flags if optimal or full recovery should be performed. If 'True', then <code>timestamp_field</code> and <code>recovery_days</code> are mandatory. Default value is 'True'.	<code>True</code>
<code>timestamp_field</code> ¶	<code>str None</code>	Name of the timestamp field.	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	A pyspark DataFrame containing all relevant instances from the original.

`validate_entity_fields` [¶](#)

```
validate_entity_fields(entity_fields: list, recovery_days: int | None) -> tuple[dict[str, int | None], int | None]
```

Validate the types of parameter `entity_fields`.

Returns both a properly formatted dictionary for function `recover_entities`'s 'entity_fields' parameter input and the corresponding value for `recovery_days`

Parameters:

Name	Type	Description	Default
<code>entity_fields</code> ¶	<code>list</code>	List of field names to be recovered or list of tuples with field names and respective recovery days.	<i>required</i>
<code>recovery_days</code> ¶	<code>int None</code>	Days for recovery. When defined, this will set all entity fields to have the same recovery.	<i>required</i>

Returns:

Type	Description
<code>entity_fields_dict</code>	Dictionary of fields and respective recovery days.
<code>recovery_days</code>	Integer value or None

validate_target_parameters [¶](#)

```
validate_target_parameters(target_field: I, target_value: S) -> tuple[I, set]
```

Validate the types of parameters `target_field` and `target_value`.

Parameters:

Name	Type	Description	Default
<code>target_field</code> ¶	<code>I</code>	Expected to be of str type.	<i>required</i>
<code>target_value</code> ¶	<code>S</code>	Expected to be of a valid type, or a set with all elements belonging to the same valid type. Valid types are: bool, int, str.	<i>required</i>

Returns:

Type	Description
<code>target_field</code>	
<code>target_value</code>	<code>target_value</code> is turned into a set if it wasn't already.

validate_timestamp_field [¶](#)

```
validate_timestamp_field(timestamp_field: str | None, exclude_kickoff_window: bool,
recovery_days: int | None = None) -> None
```

Validate the timestamp field in the context of recovery and kickoff windows.

Parameters:

Name	Type	Description	Default
<code>timestamp_field</code> ¶	<code>str</code>	Name of the timestamp field.	<i>required</i>
<code>exclude_kickoff_window</code> ¶	<code>bool</code>	Whether to exclude the kickoff window.	<i>required</i>
<code>recovery_days</code> ¶	<code>int</code>	Days for recovery. When defined, this will set all entity fields to have the same recovery.	None

random_undersampler

Implementation of the history-preserving random undersampling strategy.

LOGGER

module-attribute

```
LOGGER = getLogger(__name__)
```

RandomUndersampler

```
RandomUndersampler(fraction: float | None = None, n: int | None = None, n_positives: int | None = None, target_field: str | None = None, target_value: TargetType | set[TargetType] | None = None, seed: int | None = None)
```

Bases: [BaseSampler](#)

History-preserving random undersampling strategy.

The random undersampling is performed according to one (and only one) of the following options to be specified: `fraction`, `n`, `n_positives`.

Note

It is not guaranteed to provide *exactly* the specified fraction/balance/n.

Parameters:

Name	Type	Description	Default
<code>fraction</code>	float	Specifies the fraction of instances to be sampled relative to the whole size of the input data.	None
<code>n</code>	int	Specifies the number of instances to sample.	None
<code>n_positives</code>	int	Specifies the desired number of instances for the positive class that the sampled data should contain, maintaining the original balance (random sample). If this option is chosen, the parameters <code>target_field</code> and <code>target_value</code> must also be given.	None
<code>target_field</code>	str	Name of the target field. This parameter must be specified together with <code>target_value</code> if the option <code>n_positives</code> was used, otherwise it will be ignored.	None
<code>target_value</code>	str	Name of class associated with the positive class in <code>target_field</code> . It can be a set of classes to represent that all those classes should be kept complete without sampling. This parameter must be specified together with <code>target_field</code> if the option <code>n_positives</code> was used, otherwise it will be ignored.	None
<code>seed</code>	int	Random seed for Spark's random number generator.	None

See Also

pyspark.sql.functions.rand

TIMESTAMP_MS_FIELD `class-attribute` `instance-attribute` [↗](#)

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

seed `property` [↗](#)

```
seed: int
```

sample [↗](#)

```
sample(df: _TDataFrame, entity_field: str | None = None, filter_expr: Column | None = None, repartition: bool = False, mark_field: str = MARK_FIELD, optimal_recovery: bool = True, exclude_kick_off_window: bool = True, timestamp_field: str | None = None, recovery_days: int | None = None, entity_fields: list[str] | None = None) -> _TDataFrame
```

Undersample the input data via a history-preserving strategy.

When specifying a filter it may happen that after the filter you get much less data than originally. In this case, your final sample might result in less rows than you expected.

For example, imagine you have 10M rows in your raw data, and you want to work with just 1M. In this case, you would instantiate your sampler with `fraction=0.1`. However, if only 1% of your rows are part of your filter (`filter_expr`) then your sample will only have those rows. In this case the final size will be 1% of the original, instead of 10%.

Parameters:

Name	Type	Description	Default
df ↗	<code>DataFrame</code>	Input data.	<i>required</i>
entity_field ↗	<code>str</code>	Name of the field for which recovery should occur. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_fields' is None and vice-versa.	None
filter_expr ↗	<code>Column</code>	PySpark expression to filter events from the random sampling election. A typical example is a filter for scorable events.	None
repartition ↗	<code>bool</code>	Indicate if the output DataFrame is to be repartitioned to match the number of instances per partition of the original dataset. Default is False, meaning default partitioning.	False
mark_field ↗	<code>str</code>	Name of the field that will be created to preserve the sampling history. Events that were randomly sampled will have a value 'True' for this field. Only applicable if <code>entity_field</code> is given. Default is "fdz_sample_ismarked".	MARK_FIELD
optimal_recovery ↗	<code>bool</code>	Indicates whether optimal recovery should be performed. If set to False, full recovery will be used instead. Only applicable if <code>entity_field</code> is given. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory.	True
	<code>bool</code>	Indicates whether first time window should be excluded from marking. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory. This is typically useful because profile metrics are not correctly computed in this window.	True

Name	Type	Description	Default
<code>exclude_kickoff_window</code> ¶			
<code>timestamp_field</code> ¶	<code>str</code> <code>None</code>	Name of the field that will be used to as timestamp. Required if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> .	<code>None</code>
<code>recovery_days</code> ¶	<code>int</code> <code>None</code>	Size in days of the window considered. Typically, this value should correspond to the maximum window size of your plan's metrics operator. Applicable if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> . In the case that recovery day values are provided through 'entity_fields', this argument is ignored.	<code>None</code>
<code>entity_fields</code> ¶	<code>list</code>	List of field names for which recovery should occur. Alternatively, can be a list of tuples (field, recovery_days) if different windows are to be applied to each of the fields. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_field' is <code>None</code> and vice versa. In the case that a list of tuples is provided, 'recovery_days' is ignored. If 'exclude_kickoff_window' is 'True' then the corresponding window will be the maximum value of the recovery_days provided in the 'entity_fields' tuples.	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	Sampled data.

Examples:

This examples demonstrates the behaviour of the option `exclude_kickoff_window`.

```
>>> first_timestamp = df.agg(F.min(timestamp_field)).first()[0]
>>> datetime.fromtimestamp(first_timestamp/1000)
2018-12-20 09:27:53
>>> window_size = 7 #days
>>> df_sampled = sampler.sample(
...     df,
...     mark_field='ismarked',
...     exclude_kickoff_window = True,
...     timestamp_field='timestamp',
...     recovery_days=window_size,
... )
>>> first_marked_timestamp = (
...     df.filter(col(is_marked)).
...     agg(F.min(timestamp_field)).first()[0]
... )
>>> datetime.fromtimestamp(first_marked_timestamp/1000)
2018-12-27 10:15:48
>>> MS_PER_DAY = 24*60*60*1000
>>> first_marked_timestamp >= first_timestamp + max_window*MS_PER_DAY
True
```

stratified_undersampler

Implementation of the history-preserving stratified random undersampling strategy.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

StratifiedRandomUndersampler

```
StratifiedRandomUndersampler(target_field: str, target_value: TargetType | set[TargetType], fraction: float | None = None, balance: float | None = None, n: int | None = None, seed: int | None = None)
```

Bases: [BaseSampler](#)

History-preserving stratified random undersampling strategy.

Constructor for StratifiedRandomUndersampler.

The random undersampling is performed according to one (and only one) of the following options to be specified: `fraction`, `balance`, `n`.

Note

It is not guaranteed to provide *exactly* the specified fraction/balance/n.

Parameters:

Name	Type	Description	Default
<code>target_field</code>	<code>str</code>	Name of the field used for stratification.	<i>required</i>
<code>target_value</code>	<code>(bool, int, str or set)</code>	Name of class associated with the positive class in <code>target_field</code> . This class will be maintained complete. It can be a set of classes to represent that all those classes should be kept complete without sampling.	<i>required</i>
<code>fraction</code>	<code>float</code>	Specifies the fraction of the negative class to be sampled. All instances from the positive class will be kept.	<code>None</code>
<code>balance</code>	<code>float</code>	Specifies the desired target balance. E.g. 0.9 means that the positive:negative ratio will be 1:9. All instances from the positive class will be kept, while the negative class will be sampled in order to result in the specified balance ratio.	<code>None</code>
<code>n</code>	<code>int</code>	Specifies the number of instances to sample from the negative class. All instances from the positive class will be kept.	<code>None</code>

Name	Type	Description	Default
<code>seed</code> ↗	<code>int</code>	Random seed for Spark's random number generator.	<code>None</code>

See Also

`pyspark.sql.functions.rand`

`TIMESTAMP_MS_FIELD` `class-attribute` `instance-attribute` [↗](#)

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

`seed` `property` [↗](#)

```
seed: int
```

`sample` [↗](#)

```
sample(df: _TDataFrame, entity_field: str | None = None, filter_expr: Column | None = None, repartition: bool = False, mark_field: str = MARK_FIELD, optimal_recovery: bool = True, exclude_kick_off_window: bool = True, timestamp_field: str | None = None, recovery_days: int | None = None, entity_fields: list[str] | None = None) -> _TDataFrame
```

Undersample the input data via a history-preserving stratified strategy.

Parameters:

Name	Type	Description	Default
<code>df</code> ↗	<code>DataFrame</code>	Input data, as a pyspark DataFrame.	<i>required</i>
<code>entity_field</code> ↗	<code>str</code>	Name of the field for which recovery should occur. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_fields' is None and vice-versa.	<code>None</code>
<code>filter_expr</code> ↗	<code>Column</code>	PySpark expression to filter events from the random sampling selection. A typical example is a filter for scorable events.	<code>None</code>
<code>repartition</code> ↗	<code>bool</code>	Indicate if the output DataFrame is to be repartitioned to match the number of instances per partition of the original dataset. Default is False, meaning default partitioning.	<code>False</code>
<code>mark_field</code> ↗	<code>str</code>	Name of the field that will be created to preserve the sampling history. Events that were randomly sampled will have a value 'True' for this field. Only applicable if <code>entity_field</code> is given. Default is "fdz_sample_ismarked".	<code>MARK_FIELD</code>
<code>optimal_recovery</code> ↗	<code>bool</code>	Indicates whether optimal recovery should be performed. If set to False, full recovery will be used instead. Only applicable if <code>entity_field</code> is given. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory.	<code>True</code>
<code>exclude_kick_off_window</code> ↗	<code>bool</code>	Indicates whether first time window should be excluded from marking. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory. This is typically useful because profile metrics are not correctly computed in this window.	<code>True</code>

Name	Type	Description	Default
<code>timestamp_field</code> ¶	<code>str</code> <code>None</code>	Name of the field that will be used to as timestamp. Required if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> .	<code>None</code>
<code>recovery_days</code> ¶	<code>int</code> <code>None</code>	Size in days of the window considered. Typically, this value should correspond to the maximum window size of your plan's metrics operator. Applicable if <code>optimal_recovery</code> is <code>True</code> or <code>exclude_kickoff_window</code> is <code>True</code> . In the case that recovery day values are provided through 'entity_fields', this argument is ignored.	<code>None</code>
<code>entity_fields</code> ¶	<code>list</code>	List of field names for which recovery should occur. Alternatively, can be a list of tuples (field, recovery_days) if different windows are to be applied to each of the fields. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can only be provided if 'entity_field' is <code>None</code> and vice-versa. In the case that a list of tuples is provided, 'recovery_days' is ignored. If 'exclude_kickoff_window' is 'True' then the corresponding window will be the maximum value of the recovery_days provided in the 'entity_fields' tuples.	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	Sampled data.

Examples:

This examples demonstrates the behaviour of the option `exclude_kickoff_window`.

```
>>> first_timestamp = df.agg(F.min(timestamp_field)).first()[0]
>>> datetime.fromtimestamp(first_timestamp/1000)
2018-12-20 09:27:53
>>> window_size = 7 #days
>>> df_sampled = sampler.sample(
...     df,
...     mark_field='ismarked',
...     exclude_kickoff_window = True,
...     timestamp_field='timestamp',
...     recovery_days=window_size,
... )
>>> first_marked_timestamp = (
...     df.filter(col(is_marked)).
...     agg(F.min(timestamp_field)).first()[0]
... )
>>> datetime.fromtimestamp(first_marked_timestamp/1000)
2018-12-27 10:15:48
>>> MS_PER_DAY = 24*60*60*1000
>>> first_marked_timestamp >= first_timestamp + max_window*MS_PER_DAY
True
```

fit_indexers

This module implements the API to allow users to fit new indexers by wrapping the needed functionality from the string-indexing library.

DEFAULT_MAX_CATEGORIES module-attribute

```
DEFAULT_MAX_CATEGORIES = 10000
```

DEFAULT_MIN_CATEGORY_SAMPLES module-attribute

```
DEFAULT_MIN_CATEGORY_SAMPLES = 100
```

logger module-attribute

```
logger = getLogger(__name__)
```

bulk_fit

```
bulk_fit(app: App, indexer_name_pattern: str, data: DataFrame, features: List[str], max_categories: int = DEFAULT_MAX_CATEGORIES, min_category_samples: int = DEFAULT_MIN_CATEGORY_SAMPLES) -> List[PulseIndexer]
```

Create several PulseIndexers from a given PySpark DataFrame. This is more performant than calling `fit` directly as operations are submitted as parallel Spark jobs. However, make sure there is enough memory in the executors to hold all indexers in memory.

Indexers with no items are not created on purpose as they would have no use. A warning will be issued for each indexer that was empty.

Parameters:

Name	Type	Description	Default
<code>app</code> 	<code>App</code>	Pulse Application where the indexer will be created.	<i>required</i>
<code>indexer_name_pattern</code> 	<code>str</code>	A format pattern to generate the indexer name for each indexed feature. For example: "v18_{feature}max_cats". The possible pattern placeholders are: - {feature} : The name of the column being fit - {max_categories} : The actual <code>max_categories</code> parameter value. - {min_category_samples} : The actual <code>min_category_samples</code> parameter value. It will show up in the Pulse Lists' pane with that name and the indexer resource prefix (<code>ds_api.string_indexing.pulse_indexers.PULSE_INDEXER_DESC_PREFIX</code>).	<i>required</i>
<code>data</code> 	<code>DataFrame</code>	PySpark DataFrame with data to index. Must have the columns listed in <code>features</code> .	<i>required</i>
<code>features</code> 	<code>list of str</code>	Names of the columns to index.	<i>required</i>

Name	Type	Description	Default
<code>max_categories</code> ¶	(int,)	default = ds_api.string_indexing.fit_indexers.DEFAULT_MAX_CATEGORIES Maximum number of categories from the target column to index. Categories with the least number of observations will be filtered out from the indexer mapping until only <code>max_categories</code> are left. Any categories left outside this will be left to the "others" bucket. If set to 0 no such clipping is applied.	DEFAULT_MAX_CATEGORIES
<code>min_category_samples</code> ¶	(int,)	default = ds_api.string_indexing.fit_indexers.DEFAULT_MAX_CATEGORIES Minimum number of observations that a category must have during fit to be indexed. If set to 0 no such clipping is applied.	DEFAULT_MIN_CATEGORY_SAMPLES

Returns:

Type	Description
List of <code>PulseIndexer</code>	The created <code>PulseIndexer</code> Lists.

fit [¶](#)

```
fit(app: App, indexer_name: str, data: DataFrame, feature: str, max_categories: int = DEFAULT_MAX_CATEGORIES, min_category_samples: int = DEFAULT_MIN_CATEGORY_SAMPLES) -> PulseIndexer
```

Create a new `PulseIndexer` from a given PySpark `DataFrame`. The Spark `DataFrame` must have a column with that name.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Pulse Application where the indexer will be created.	<i>required</i>
<code>indexer_name</code> ¶	str	Name to give to the generated <code>PulseIndexer</code> . It will show up in the Pulse Lists' pane with that name and the indexer resource prefix (<code>ds_api.string_indexing.pulse_indexers.PULSE_INDEXER_DESC_PREFIX</code>).	<i>required</i>
<code>data</code> ¶	DataFrame	PySpark <code>DataFrame</code> with data to index. Must have the column <code>colname</code> .	<i>required</i>
<code>feature</code> ¶	str	Name of the column to index.	<i>required</i>
<code>max_categories</code> ¶	(int,)	default = ds_api.string_indexing.fit_indexers.DEFAULT_MAX_CATEGORIES Maximum number of categories from the target column to index. Categories with the least number of observations will be filtered out from the indexer mapping until only <code>max_categories</code> are left. Any categories left outside this will be left to the "others" bucket. If set to 0 no such clipping is applied.	DEFAULT_MAX_CATEGORIES
	(int,)		

Name	Type	Description	Default
<code>min_category_samples</code>		<code>default =</code> <code>ds_api.string_indexing.fit_indexers.DEFAULT_MAX_CATEGORIES</code> Minimum number of observations that a category must have during fit to be indexed. If set to 0 no such clipping is applied.	<code>DEFAULT_MIN_CATEGORY_SAMPLES</code>

Returns:

Type	Description
<code>PulseIndexer</code>	The created PulseIndexer List.

Raises:

Type	Description
<code>ValueError</code>	If during the indexer fit there are not enough samples to generate any categories at all and the indexer would be empty.

pulse_indexers [↗](#)

Manage Pulse Indexer resources.

CARDINALITY_TAG_PREFIX module-attribute [↗](#)

```
CARDINALITY_TAG_PREFIX = 'defaultValue'
```

DEFAULT_CATEGORY_COLNAME module-attribute [↗](#)

```
DEFAULT_CATEGORY_COLNAME = 'category'
```

INDEXER_TAG_DESC module-attribute [↗](#)

```
INDEXER_TAG_DESC = 'Indexer'
```

INDEX_COL module-attribute [↗](#)

```
INDEX_COL = 'index'
```

MAX_CATEGORIES_TAG_PREFIX module-attribute [↗](#)

```
MAX_CATEGORIES_TAG_PREFIX = 'max_categories'
```

MIN_CATEGORY_SAMPLES_TAG_PREFIX module-attribute [↗](#)

```
MIN_CATEGORY_SAMPLES_TAG_PREFIX = 'min_category_samples'
```

PULSE_INDEXER_DESC_PREFIX module-attribute [↗](#)

```
PULSE_INDEXER_DESC_PREFIX = '{StringIndexer} '
```

PulseIndexer module-attribute [↗](#)

```
PulseIndexer: ComplexManagedListModel = ComplexManagedListModel
```

PulseIndexerItem module-attribute [↗](#)

```
PulseIndexerItem: ComplexManagedListItemModel = ComplexManagedListItemModel
```

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

assert_can_create_indexer [↗](#)

```
assert_can_create_indexer(app: App, indexer_name: str) -> None
```

Asserts that a PulseIndexer with such name can still be created. Raises a PulseIndexerExistsError if another PulseIndexer with such name already exists.

assert_can_create_indexers [¶](#)

```
assert_can_create_indexers(app: App, indexer_names: List[str]) -> None
```

Asserts that PulseIndexers with such names can still be created. Raises a PulseIndexerExistsError if other indexers already exist.

can_create_indexer [¶](#)

```
can_create_indexer(app: App, indexer_name: str) -> bool
```

Checks that a PulseIndexer with such name can still be created.

create_from_pandas_indexer_mapping [¶](#)

```
create_from_pandas_indexer_mapping(app: App, indexer_mapping: DataFrame, indexer_name: str, category_colname: str = DEFAULT_CATEGORY_COLNAME, index_colname: str = INDEX_COL, max_categories: int = 0, min_category_samples: int = 0) -> Optional[PulseIndexer]
```

Takes a dataframe with the indexer mapping and creates a PulseIndexer.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Pulse Application.	<i>required</i>
<code>indexer_mapping</code> ¶	<code>DataFrame</code>	Indexer mapping data. Must have a "category" and an "index" column.	<i>required</i>
<code>indexer_name</code> ¶	<code>str</code>	Name to give to the generated PulseIndexer. It will show up in the Pulse Lists' pane with that name and the indexer resource prefix (<code>PULSE_INDEXER_DESC_PREFIX</code>).	<i>required</i>
<code>category_colname</code> ¶	<code>str</code>	Name of the category column name.	<code>DEFAULT_CATEGORY_COLNAME</code>
<code>index_colname</code> ¶	<code>str</code>	Name of the index column name.	<code>= INDEX_COL</code>
<code>max_categories</code> ¶	<code>int</code>	Optional Metadata for end-users. Will be encoded in a tag. Might not make sense for libraries that do not support this feature. Maximum number of categories allowed when fitting the original indexer.	<code>0</code>
<code>min_category_samples</code> ¶	<code>int</code>	Optional Metadata for end-users. Will be encoded in a tag. Might not make sense for libraries that do not support this feature. Minimum required number of samples allowed to form a category when fitting the indexer.	<code>0</code>

Returns:

Type	Description
Optional[PulseIndexer]	The created PulseIndexer List. If it is empty it is not created and None is returned instead.

create_from_string_indexer

```
create_from_string_indexer(app: App, indexer: FrequencyStringIndexer, indexer_name: str) -> Optional[PulseIndexer]
```

Turns a FrequencyStringIndexer into a PulseIndexer.

Only FrequencyStringIndexer is supported given the output value produced for unknown/"other" values which matches the number of categories in the indexer. Usually that only makes sense for Frequency Indexers.

Parameters:

Name	Type	Description	Default
app 	App	Pulse Application where the indexer will be created.	<i>required</i>
indexer 	FrequencyStringIndexer	Frequency Indexer. Only FrequencyStringIndexer is supported given the output value for "other" generally doesn't make sense for other indexer types.	<i>required</i>
indexer_name 	str	Name of the generated indexer in Pulse.	<i>required</i>

Returns:

Type	Description
Optional[PulseIndexer]	Created Pulse Indexer List. If it is empty it is not created and None is returned instead.

generate_indexer_desc

```
generate_indexer_desc(indexer_name: str) -> str
```

From an indexer name it computes the desc to use. It adds a prefix signaling this is an indexer. That prefix ensures indexers show up after all other resources with alphanumeric names if resources are sorted according to ASCII encoding.

Parameters:

Name	Type	Description	Default
indexer_name 	str	Name of the indexer.	<i>required</i>

Returns:

Type	Description
str	Indexer user-facing description (desc), with indexer prefix included.

get

```
get(app: App, *, name: str = '', desc: str = '', include_items: bool = False) -> Optional[PulseIndexer]
```

Returns the indexer list if it exists in the app.

Parameters:

Name	Type	Description	Default
app 	<code>App</code>	Pulse Application.	<i>required</i>
name 	<code>str</code>	Pass the indexer name (not the internal name!) (without indexer prefix) Either pass this or <code>indexer_desc</code> .	<code>''</code>
desc 	<code>str</code>	Pass the indexer desc (with indexer prefix). Either pass this or <code>indexer_name</code> .	<code>''</code>
include_items 	<code>bool</code>	Include the indexer items.	<code>False</code>

Returns:

Type	Description
<code>PulseIndexer</code> or <code>None</code>	Pulse Indexer resource if found, or <code>None</code> otherwise.

Raises:

Type	Description
<code>ValueError</code>	Raised if either: - <code>name</code> and <code>desc</code> are empty. - <code>desc</code> is invalid (misses the string indexer prefix).

feature

Base class for all Plan Features.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

BaseFeature

```
BaseFeature(name: str, description: Optional[str] = None, operator: Optional[BaseFeaturesOperator] = None)
```

Bases: `ABC`

Abstract base class for all Features in a Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> 	<code>str</code>	Name of the feature	<i>required</i>
<code>description</code> 	<code>str</code>	description of the feature.	<code>None</code>
<code>operator</code> 	<code>BaseFeaturesOperator</code>	Operator where the feature is in.	<code>None</code>

`description` property writable

```
description
```

`name` property writable

```
name
```

`operator` property

```
operator
```

`from_dict` abstractmethod classmethod

```
from_dict(feat_dict: Dict[str, str]) -> BaseFeature
```

Creates a Feature from a dict type to facilitate importing features from a tabular format.

Parameters:

Name	Type	Description	Default
<code>feat_dict</code> 	<code>dict</code>	Dictionary containing the feature in a tabular format.	<i>required</i>

Returns:

Type	Description
<code>BaseFeature</code>	Feature initiated with the attributes from the dict.

from_java_feature `abstractmethod` `classmethod` [¶](#)

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> BaseFeature
```

Converts a Java DS-API Feature to a Python feature class.

get_dependencies `abstractmethod` [¶](#)

```
get_dependencies() -> Set[str]
```

Gets the list of other features names that are used to create the feature.

Dependencies are not recursive.

Returns:

Type	Description
<code>set</code>	Set of feature names.

get_dependencies_by_type `abstractmethod` [¶](#)

```
get_dependencies_by_type() -> Dict[str, List[str]]
```

Gets the lists (by type of dependency) of other features names that are used to create the feature.

Dependencies are not recursive.

Returns:

Type	Description
<code>Dict[str, List[str]]</code>	Dictionary of dependencies, the keys are the dependency types and the values are the list of feature names.

rename_dependencies `abstractmethod` [¶](#)

```
rename_dependencies(old_name: str, new_name: str)
```

Renames the feature dependencies matching the old name.

Parameters:

Name	Type	Description	Default
<code>old_name</code> ¶	<code>str</code>	Old name to change from.	<i>required</i>
<code>new_name</code> ¶	<code>str</code>	New name to change to.	<i>required</i>

to_dict `abstractmethod` [¶](#)

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

Converts the feature to a dict type to facilitate exporting features into a tabular format.

Parameters:

Name	Type	Description	Default
<code>with_operator</code> ¶	<code>bool</code>	When 'false', returns the object without its operator attributes.	<code>True</code>

Returns:

Type	Description
<code>dict</code>	Dictionary containing all the editable fields from the plan to create this feature.

Raises:

Type	Description
<code>AttributeError</code>	When <code>with_operator</code> is true and the feature has not been added to an operator.

`to_java_feature_builder` [abstractmethod](#) [¶](#)

```
to_java_feature_builder() -> JavaClass
```

Converts a Python feature class to a Java DS-API Feature Builder.

feature_container

FeatureContainer

```
FeatureContainer(features: Iterable[BaseFeature] | None = None)
```

Holds a collection of features while retaining both their "individual" definition, and their definition within the context of a plan.

This is similar to a dictionary that maps the individual feature definition (ie, without the operator context), with the "full feature" (ie with the operator).

values property

```
values: List[BaseFeature]
```

add

```
add(feature: BaseFeature) -> FeatureContainer
```

remove

```
remove(feature: BaseFeature) -> FeatureContainer
```

HashableFeature `dataclass`

```
HashableFeature(name: str, feature: str)
```

Feature representation without its operator. This is useful to define the 'uniqueness' of a feature. A feature is considered unique based on its contents excluding the operator to which it is attached.

feature instance-attribute

```
feature: str
```

name instance-attribute

```
name: str
```

from_feature classmethod

```
from_feature(feature: BaseFeature) -> HashableFeature
```

Create a HashableFeature from a BaseFeature (Map, Metric, ...).

For this purpose we only want to keep the feature name ("desc") and its definition without the link to the Operator.

list_contains_feature [🔗](#)

LOGGER module-attribute [🔗](#)

```
LOGGER = getLogger(__name__)
```

ListContainsFeatureBuilder module-attribute [🔗](#)

```
ListContainsFeatureBuilder = autoclass(DS\_API\_CLASSPATH + 'plan.features.ListContainsFeature$Builder')
```

ListContainsFeature [🔗](#)

```
ListContainsFeature(name: str, list_id: str, expression: str, *, timestamp_field: str | None = None, description: str | None = None)
```

Bases: [ListFeature](#)

Represents a ListContains feature in Pulse.

description property writable [🔗](#)

```
description
```

expression property writable [🔗](#)

```
expression: str
```

list_id property writable [🔗](#)

```
list_id: str
```

name property writable [🔗](#)

```
name
```

operator property [🔗](#)

```
operator
```

timestamp_field property writable [🔗](#)

```
timestamp_field: str | None
```

from_dict classmethod [🔗](#)

```
from_dict(feats_dict: Dict[str, str]) -> ListContainsFeature
```

from_java_feature classmethod [🔗](#)

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> ListContainsFeature
```

Creates a ListContainsFeature from a Java DS-API ListContainsFeature.

Parameters:

Name	Type	Description	Default
<code>feature</code> ¶	Java DS-API ListContainsFeature		<i>required</i>
<code>operator</code> ¶	ListOperator		<i>required</i>

`get_dependencies` [¶](#)

```
get_dependencies() -> set[str]
```

`get_dependencies_by_type` [¶](#)

```
get_dependencies_by_type() -> dict[str, list[str]]
```

`rename_dependencies` [¶](#)

```
rename_dependencies(old_name: str, new_name: str)
```

`to_dict` [¶](#)

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

`to_java_feature_builder` [¶](#)

```
to_java_feature_builder() -> JavaClass
```

From ListContainsFeature to a Java DS-API ListContainsFeature.Builder.

Returns:

Type	Description
<code>Builder</code>	ListContainsFeature.Builder

list_feature

LIST_DEP_TYPE_NAME module-attribute

LIST_DEP_TYPE_NAME = 'list'

LOGGER module-attribute

LOGGER = getLogger(__name__)

ListFeature

ListFeature(name: str, list_id: str, expression: str, *, timestamp_field: str | None = None, description: str | None = None)

Bases: BaseFeature, ABC

Base class for ListGetFeature and ListContainsFeature.

Parameters:

Name	Type	Description	Default
name	str	Name of the feature.	required
list_id	str	id of the List in Pulse.	required
expression	str	Feature expression. Fields must be enclosed with the pattern <code>\${field}</code> eg. <code>"com.feedzai.cloud.udf.Strings.normalize(\${MerchantName})" .</code>	required
description	str	Feature description, by default None.	None

description property writable

description

expression property writable

expression: str

list_id property writable

list_id: str

name property writable

name

operator **property** [¶](#)

operator

timestamp_field **property** **writable** [¶](#)

timestamp_field: str | None

from_dict **abstractmethod** **classmethod** [¶](#)

from_dict(feats_dict: Dict[str, str]) -> BaseFeature

Creates a Feature from a dict type to facilitate importing features from a tabular format.

Parameters:

Name	Type	Description	Default
feats_dict ¶	dict	Dictionary containing the feature in a tabular format.	<i>required</i>

Returns:

Type	Description
BaseFeature	Feature initiated with the attributes from the dict.

from_java_feature **abstractmethod** **classmethod** [¶](#)

from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> BaseFeature

Converts a Java DS-API Feature to a Python feature class.

get_dependencies [¶](#)

get_dependencies() -> set[str]

get_dependencies_by_type [¶](#)

get_dependencies_by_type() -> dict[str, list[str]]

rename_dependencies [¶](#)

rename_dependencies(old_name: str, new_name: str)

to_dict [¶](#)

to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]

to_java_feature_builder **abstractmethod** [¶](#)

to_java_feature_builder() -> JavaClass

Converts a Python feature class to a Java DS-API Feature Builder.

list_get_feature

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

ListGetFeatureBuilder module-attribute

```
ListGetFeatureBuilder = autoclass(DS_API_CLASSPATH + 'plan.features.ListGetFeature$Builder')
```

ListGetFeature

```
ListGetFeature(name: str, list_id: str, expression: str, list_field: str, *, timestamp_field: str | None = None, description: str | None = None)
```

Bases: [ListFeature](#)

Represents a ListGet feature in Pulse.

Parameters:

Name	Type	Description	Default
<code>name</code>	<code>str</code>	Name of the feature.	<i>required</i>
<code>list_id</code>	<code>str</code>	id of the List in Pulse.	<i>required</i>
<code>expression</code>	<code>str</code>	Feature expression. Fields must be enclosed with the pattern <code>\${field}</code> and the field type of the returned expression must match the type of the List key. eg. <code>"com.feedzai.cloud.udf.Strings.normalize(\${MerchantName})" .</code>	<i>required</i>
<code>list_field</code>	<code>str</code>	Field to be accessed from the complex list.	<i>required</i>
<code>description</code>	<code>str</code>	Feature description, by default None.	None

description property writable

```
description
```

expression property writable

```
expression: str
```

list_field property writable [¶](#)

```
list_field
```

list_id property writable [¶](#)

```
list_id: str
```

name property writable [¶](#)

```
name
```

operator property [¶](#)

```
operator
```

timestamp_field property writable [¶](#)

```
timestamp_field: str | None
```

from_dict classmethod [¶](#)

```
from_dict(feats_dict: Dict[str, str]) -> ListGetFeature
```

from_java_feature classmethod [¶](#)

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> ListGetFeature
```

Creates a ListGetFeature from a Java DS-API ListGetFeature.

Parameters:

Name	Type	Description	Default
feature ¶	Java DS-API ListGetFeature		<i>required</i>
operator ¶	ListOperator		<i>required</i>

Returns:

Type	Description
ListGetFeature	ListGetFeature

get_dependencies [¶](#)

```
get_dependencies() -> set[str]
```

get_dependencies_by_type [¶](#)

```
get_dependencies_by_type() -> dict[str, list[str]]
```

rename_dependencies [¶](#)

```
rename_dependencies(old_name: str, new_name: str)
```

to_dict [¶](#)

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

to_java_feature_builder [¶](#)

```
to_java_feature_builder() -> JavaClass
```

From ListGetFeature to a Java DS-API ListGetFeature.Builder.

Returns:

Type	Description
<code>Builder</code>	ListGetFeature.Builder

map_feature

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

MAP_DEP_TYPE_NAME module-attribute

```
MAP_DEP_TYPE_NAME = 'map'
```

MapFeatureBuilder module-attribute

```
MapFeatureBuilder = autoclass(DS_API_CLASSPATH + 'plan.features.MapFeature$Builder')
```

MapFeature

```
MapFeature(name: str, expression: str, field_type: DataScienceField, description: Optional[str] = None)
```

Bases: [BaseFeature](#)

Represents a Map feature in Pulse.

Parameters:

Name	Type	Description	Default
name	str	Name of the map feature	required
expression	str	Feature expression. Fields must be enclosed with the pattern <code>\${field}</code> , eg, <code>"\${TransactionAmount} > 100 and \${Fraud} != false"</code> .	required
field_type	DataScienceField	For example <code>CategoricalField(["true", "false"])</code> .	required
description	str	Feature description, by default None.	None

description property writable

```
description
```

expression property writable

```
expression
```

field_type property writable [¶](#)

field_type

name property writable [¶](#)

name

operator property [¶](#)

operator

from_dict classmethod [¶](#)

from_dict(feats_dict: Dict[str, str]) -> [MapFeature](#)

from_java_feature classmethod [¶](#)

from_java_feature(feature: JavaClass, operator: [BaseFeaturesOperator](#)) -> [MapFeature](#)

Creates a MapFeature from a Java DS-API MapFeature.

Parameters:

Name	Type	Description	Default
feature ¶	Java DS-API MapFeature		<i>required</i>
operator ¶	MapOperator		<i>required</i>

Returns:

Type	Description
MapFeature	MapFeature

get_dependencies [¶](#)

get_dependencies() -> Set[str]

get_dependencies_by_type [¶](#)

get_dependencies_by_type() -> Dict[str, List[str]]

rename_dependencies [¶](#)

rename_dependencies(old_name: str, new_name: str)

to_dict [¶](#)

to_dict(with_operator=True) -> dict[str, str | float | int | bool | None]

to_java_feature_builder [¶](#)

to_java_feature_builder() -> JavaClass

From MapFeature to a Java DS-API MapFeature.Builder.

Returns:

Type	Description
Builder	MapFeature.Builder

metric_feature

This module implements the concrete `MetricFeature` that allows the manipulation of metrics in plans.

From an API perspective there are two properties that even though are persisted at the "metric level", they have to be defined and managed at the operator/plan level":

1. `filter_incomplete_instances` : When true, the output of the Metrics **operator** will exclude instances whose timestamp is older than the period defined by the start of the dataset and the largest window.
2. `timestamp_field` : Defined at the **plan** level in deployable plans.

AGG_DEP_TYPE_NAME module-attribute

```
AGG_DEP_TYPE_NAME = 'aggregation'
```

DEFAULT_FILTER_INCOMPLETE_INSTANCES module-attribute

```
DEFAULT_FILTER_INCOMPLETE_INSTANCES = False
```

DEFAULT_TIMESTAMP_FIELD module-attribute

```
DEFAULT_TIMESTAMP_FIELD = ''
```

FILTER_DEP_TYPE_NAME module-attribute

```
FILTER_DEP_TYPE_NAME = 'filter'
```

GROUP_BY_DEP_TYPE_NAME module-attribute

```
GROUP_BY_DEP_TYPE_NAME = 'group_by'
```

Optional module-attribute

```
JOptional = autoclass('java.util.Optional')
```

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

LOOKUP_DEP_TYPE_NAME module-attribute

```
LOOKUP_DEP_TYPE_NAME = 'lookup'
```

Long module-attribute

```
Long = autoclass('java.lang.Long')
```

METRIC_TS_DEP_TYPE_NAME module-attribute [Â¶](#)

```
METRIC_TS_DEP_TYPE_NAME = 'metric_ts'
```

MetricChannelConfig module-attribute [Â¶](#)

```
MetricChannelConfig = autoclass(MODEL_CLASSPATH +  
'datascience.plan.operator.MetricChannelConfig')
```

MetricCoalesce module-attribute [Â¶](#)

```
MetricCoalesce = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.coalesce.MetricCoalesce')
```

MetricFeatureJava module-attribute [Â¶](#)

```
MetricFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.MetricFeature')
```

UpdatePeriod module-attribute [Â¶](#)

```
UpdatePeriod = autoclass(MODEL_CLASSPATH + 'fraudclassifierdefinition.UpdatePeriod')
```

UpdatePeriodType module-attribute [Â¶](#)

```
UpdatePeriodType = autoclass(PULSE_API_CLASSPATH + 'apps.enums.UpdatePeriodType')
```

WindowConfig module-attribute [Â¶](#)

```
WindowConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.type.WindowConfig')
```

MetricFeature [Â¶](#)

```
MetricFeature(name: str, groupby_fields: list[str], aggregation: str, window_size: str, is_real_  
time: bool = True, *, lookup_fields: list[str] | None = None, scheduled_cron_expression: str | N  
one = None, filter_metric: str | None = None, delay: int | None = None, is_tumbling: bool =  
False, is_infinite: bool = False, main_channel_contributes: bool = True, secondary_channels:  
dict[str, MetricSecondaryChannelConfig] | None = None, cascaded_scheduled_metrics: bool = False,  
coalesce: float | int | str | None = None, description: str | None = None)
```

Bases: [BaseFeature](#)

Represents a Metric feature in Pulse.

Parameters:

Name	Type	Description	Default
name Â¶	str	Name of the metric feature.	<i>required</i>
groupby_fields Â¶	list of str	Fields to group by.	<i>required</i>
	str		<i>required</i>

Name	Type	Description	Default
aggregation ↗		Aggregation expression. Aggregations must be written in PQL: "avg","min","max","count","countDistinct","sum","stddev", "last","first","prev",... Fields must be enclosed with the pattern <code>\${field}</code> eg. <code>'avg(\${amount})'</code> , <code>'countDistinct(\${MerchantCity})'</code> , <code>'count()'</code> . Note that "advanced" aggregations are not compatible with Railgun.	
window_size ↗	str	Window expression. Typically, "1 day", "4 weeks", etc.	<i>required</i>
is_real_time ↗	bool	True = Real time window, False = Scheduled window. True by default.	True
scheduled_cron_expression ↗	str	If <code>is_real_time</code> = False, a cron expression must be specified to define the periodicity of the window, by default None e.g. <code>'0 0 0 * * ?'</code> for a scheduled window running every day at midnight.	None
filter_metric ↗	str	Filter expression, by default None. Fields must be enclosed with the pattern <code>\${field}</code> eg. <code>"\${TransactionAmount} > 100 and \${Fraud} != false"</code> .	None
delay ↗	int	Defines delay in milliseconds, must be greater than 1000, by default None.	None
is_tumbling ↗	bool	Tumbling window, by default False.	False
is_infinite ↗	bool	When True, this metric will be considered a "non-windowed" metric. <i>Requires Pulse 25.0.89. Experimental Feature.</i>	False
description ↗	str	Metric description, by default None.	None
lookup_fields ↗	list of str	Lookup fields, ie the fields used to query the aggregation. By default these are set to be the same as the <code>groupby_fields</code> . When different, it means that one can execute a query like: <code>group by sender -> lookup by receiver</code> . <i>Experimental feature.</i>	None
	bool	When false, the main channel does not contribute to the aggregation. True by default. <i>Omnichannel feature.</i>	True

Name	Type	Description	Default
<code>main_channel_contributes</code> ¶			
<code>secondary_channels</code> ¶	<code>dict[str, MetricSecondaryChannelConfig]</code>	Mapping of channel ID to a <code>MetricSecondaryChannelConfig</code> . Empty by default, ie no secondary channels configured. <i>Omnichannel</i> feature.	None
<code>cascaded_scheduled_metrics</code> ¶	<code>bool</code>	When True, it indicates this feature is in a plan with cascading scheduled metrics that depend on each other. False by default.	False
<code>coalesce</code> ¶	<code>float</code> <code>or int</code> <code>or str</code>	Coalesce value for when the metric aggregation is empty.	None

Raises:

Type	Description
<code>ValueError</code>	If <code>is_real_time = False</code> and <code>scheduled_cron_expression</code> is <code>None</code> . If the timestamp field from the schema is not specified in the Plan. If delay is less than 1000 ms.

`aggregation` `property` `writable` [¶](#)

`aggregation`

`cascaded_scheduled_metrics` `property` `writable` [¶](#)

`cascaded_scheduled_metrics`: `bool`

`coalesce` `property` `writable` [¶](#)

`coalesce`: `float` | `int` | `str` | `None`

`delay` `property` `writable` [¶](#)

`delay`

`description` `property` `writable` [¶](#)

`description`

`filter_incomplete_instances` `property` `writable` [¶](#)

`filter_incomplete_instances`: `bool`

filter_metric property writable [🔗](#)

```
filter_metric
```

groupby_fields property writable [🔗](#)

```
groupby_fields
```

is_infinite property writable [🔗](#)

```
is_infinite: bool
```

is_real_time property writable [🔗](#)

```
is_real_time
```

is_tumbling property writable [🔗](#)

```
is_tumbling: bool
```

lookup_fields property writable [🔗](#)

```
lookup_fields
```

main_channel_contributes property writable [🔗](#)

```
main_channel_contributes: bool
```

When False, the main channel will not contribute to the metric.

Omnichannel

This property is part of the Omnichannel suite of properties.

name property writable [🔗](#)

```
name
```

operator property [🔗](#)

```
operator
```

scheduled_cron_expression property writable [🔗](#)

```
scheduled_cron_expression
```

secondary_channels property writable [🔗](#)

```
secondary_channels: dict[str, MetricSecondaryChannelConfig]
```

Mapping between secondary channels and their configuration.

Omnichannel

This property is part of the Omnichannel suite of properties.

timestamp_field property writable [🔗](#)

```
timestamp_field: str
```

window_size `property` `writable` [¶](#)

```
window_size
```

auto_rename [¶](#)

```
auto_rename()
```

Automatically renames the metric based on its configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation_fields}_{window_size}_{is_real_time})` eg. `accountid_avg_amount_1h_rt`. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_{window_size}_{is_real_time})`.

from_dict `classmethod` [¶](#)

```
from_dict(feats_dict: dict[str, str]) -> MetricFeature
```

from_java_feature `classmethod` [¶](#)

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> MetricFeature
```

Creates a MetricFeature from a Java DS-API MetricFeature.

Parameters:

Name	Type	Description	Default
feature ¶	Java DS-API MetricFeature		<i>required</i>
operator ¶	MetricOperator		<i>required</i>

Returns:

Type	Description
MetricFeature	MetricFeature

get_default_naming [¶](#)

```
get_default_naming() -> str
```

Returns a default name based on its configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation_fields}_{window_size}_{is_real_time})` eg. `accountid_avg_amount_1h_rt`. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_{window_size}_{is_real_time})`.

In the case of "advanced" window expressions, the window expression is omitted from the name.

Railgun compatibility

Note that "advanced" window expressions are not compatible with Railgun. Please use only the appropriate set of window expressions.

Returns:

Type	Description
<code>str</code>	Default name

`get_dependencies` [¶](#)

```
get_dependencies() -> set[str]
```

`get_dependencies_by_type` [¶](#)

```
get_dependencies_by_type() -> dict[str, list[str]]
```

`rename_dependencies` [¶](#)

```
rename_dependencies(old_name: str, new_name: str)
```

`to_dict` [¶](#)

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

`to_java_feature_builder` [¶](#)

```
to_java_feature_builder() -> JavaClass
```

From MetricFeature to a Pulse API MetricOperatorConfig.

Returns:

Type	Description
<code>Builder</code>	MetricOperatorConfig.Builder

`MetricSecondaryChannelConfig` `dataclass` [¶](#)

```
MetricSecondaryChannelConfig(groupby_fields: list[str], aggregation_fields: list[str], filter_expr: str | None = None)
```

Pythonic representation of a MetricChannelConfig.

Info

Using this dataclass is required to interact with Omnichannel plans.

`aggregation_fields` `instance-attribute` [¶](#)

```
aggregation_fields: list[str]
```

`filter_expr` `class-attribute` `instance-attribute` [¶](#)

```
filter_expr: str | None = None
```

`groupby_fields` `instance-attribute` [¶](#)

```
groupby_fields: list[str]
```

`from_java` `classmethod` [¶](#)

```
from_java(cfg: JavaClass) -> MetricSecondaryChannelConfig
```

Return the Python representation of this channel from its Java counterpart.

`to_java` 

```
to_java() -> JavaClass
```

Return the Java representation of this channel configuration.

filter_operator

FilterOperatorConfigBuilder

FilterOperatorConfigBuilder = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.FilterOperatorConfig\$Builder')

LOGGER

LOGGER = getLogger(__name__)

FilterOperator

FilterOperator(pyjnius_java)

Bases: BaseOperator

Represents a Filter Operator in Pulse.

Attributes:

Name	Type	Description
expression	str	Represents the filter expression of the operator. Fields must be enclosed with the pattern <code>\${field}</code> , eg. <code>"\${TransactionAmount} > 100 and \${Fraud} != false"</code> .

expression

expression

subclasses

subclasses: List[JavaWrapper] = []

get_dependencies

get_dependencies() -> Dict[FeatureUID, Set[str]]

get_dependencies_by_type

get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, List[str]]]

get_type

get_type() -> str

rename_feature

rename_feature(old_name: str, new_name: str)

Rename a field used in the filter expression.

to_dict [↗](#)

```
to_dict() -> List[Dict]
```

update_from_dict [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```


list_operator

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

ListContainsFeatureJava module-attribute

```
ListContainsFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.ListContainsFeature')
```

ListGetFeatureJava module-attribute

```
ListGetFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.ListGetFeature')
```

ListOperator

```
ListOperator(pyjnius_java)
```

Bases: [BaseFeaturesOperator](#)

Represents a List Operator in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_feature

```
add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature
```

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature	BaseFeature	New Feature.	<i>required</i>
logger	bool	Whether to log info message or not.	True

Returns:

Type	Description
BaseFeature	New Feature.

diff

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
<code>other</code> ¶	BaseFeaturesOperator	BaseFeaturesOperator to compare against.	<i>required</i>
<code>only_diff</code> ¶	<code>bool</code>	Show features with differences only. Defaults to 'true'.	<code>True</code>

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

`get_dependencies` [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

`get_dependencies_by_type` [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, list[str]]]
```

`get_feature` [¶](#)

```
get_feature(name: str) -> ListFeature
```

Gets a Feature by a given name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the feature	<i>required</i>

Returns:

Type	Description
ListFeature	ListFeature

`get_features` [¶](#)

```
get_features() -> List[ListFeature]
```

Gets all the features in this Operator instance.

Returns:

Type	Description
<code>list of ListFeature</code>	List of ListFeature.

get_type staticmethod [¶](#)

```
get_type() -> str
```

remove_feature [¶](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
<code>bool</code>	If the feature is removed.

rename_feature [¶](#)

```
rename_feature(old_name: str, new_name: str)
```

to_csv [¶](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	<code>Path</code> or <code>str</code>	Path to where the csv will be written.	<i>required</i>

to_dict [¶](#)

```
to_dict() -> List[Dict]
```

to_pandas [¶](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
<code>DataFrame</code>	Pandas dataframe containing features and all their editable attributes.

update_from_csv [¶](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

`update_from_dict` [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

`update_from_pandas` [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ↗	DataFrame	Pandas dataframe with the operator features.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

map_operator

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

ProjectionFeature module-attribute

```
ProjectionFeature = autoclass(DS_API_CLASSPATH + 'plan.features.ProjectionFeature')
```

MapOperator

```
MapOperator(pyjnius_java)
```

Bases: [BaseFeaturesOperator](#)

Represents a Map Operator in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_feature

```
add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature
```

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature 	BaseFeature	New Feature.	<i>required</i>
logger 	bool	Whether to log info message or not.	True

Returns:

Type	Description
BaseFeature	New Feature.

add_fields_to_remove

```
add_fields_to_remove(fields=Set[str])
```

Sets the fields to be removed in the Operator.

Parameters:

Name	Type	Description	Default
fields ¶	set of str	Names of the fields, by default Set[str].	Set[str]

diff [¶](#)

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
other ¶	BaseFeaturesOperator	BaseFeaturesOperator to compare against.	<i>required</i>
only_diff ¶	bool	Show features with differences only. Defaults to 'true'.	True

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

get_dependencies [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

get_dependencies_by_type [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, List[str]]]
```

get_feature [¶](#)

```
get_feature(name: str) -> MapFeature
```

Gets a Feature by a given name.

Parameters:

Name	Type	Description	Default
name ¶	str	Name of the feature.	<i>required</i>

Returns:

Type	Description
MapFeature	MapFeature

get_features [¶](#)

```
get_features() -> List[MapFeature]
```

Gets all the features in this Operator instance.

Returns:

Type	Description
list of MapFeature	List of MapFeature

`get_fields_to_remove` [¶](#)

```
get_fields_to_remove() -> Set[str]
```

Gets the fields to be removed on this Operator.

Returns:

Type	Description
set of optional str	Names of the fields to be removed.

`get_type` `staticmethod` [¶](#)

```
get_type() -> str
```

`remove_feature` [¶](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	str	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
bool	If the feature is removed.

`rename_feature` [¶](#)

```
rename_feature(old_name: str, new_name: str)
```

`to_csv` [¶](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	Path or str	Path to where the csv will be written.	<i>required</i>

to_dict [↗](#)

```
to_dict() -> List[Dict]
```

to_pandas [↗](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
DataFrame	Pandas dataframe containing features and all their editable attributes.

update_from_csv [↗](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
path ↗	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

update_from_dict [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

update_from_pandas [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
pd_df ↗	DataFrame	Pandas dataframe with the operator features.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

metric_operator

LOGGER module-attribute

LOGGER = getLogger(__name__)

MetricOperator

MetricOperator(pyjnius_java, filter_incomplete_instances: bool = DEFAULT_FILTER_INCOMPLETE_INSTANCES)

Bases: BaseFeaturesOperator

Represents a Metric Operator in Pulse.

Parameters:

Name	Type	Description	Default
filter_incomplete_instances	bool	If True, the instances from the input datasource that do not have a complete window, based on the largest window from all the MetricFeatures, will be filtered out, by default False.	DEFAULT_FILTER_INCOMPLETE_INSTANCES

filter_incomplete_instances property writable

filter_incomplete_instances

Warning

Defining the value of this property triggers an update in all 'MetricFeatures' that are part of this operator.

subclasses class-attribute instance-attribute

subclasses: List[JavaWrapper] = []

timestamp_field property

timestamp_field: str

Warning

Only works with deployable plans.

add_feature

add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature	BaseFeature	New Feature.	required

Name	Type	Description	Default
<code>logger</code> ¶	<code>bool</code>	Whether to log info message or not.	<code>True</code>

Returns:

Type	Description
<code>BaseFeature</code>	New Feature.

`diff` [¶](#)

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
<code>other</code> ¶	<code>BaseFeaturesOperator</code>	BaseFeaturesOperator to compare against.	<i>required</i>
<code>only_diff</code> ¶	<code>bool</code>	Show features with differences only. Defaults to 'true'.	<code>True</code>

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

`get_dependencies` [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

`get_dependencies_by_type` [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, list[str]]]
```

`get_feature` [¶](#)

```
get_feature(name: str) -> MetricFeature | None
```

Gets a Feature by a given name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the feature.	<i>required</i>

Returns:

Type	Description
MetricFeature	MetricFeature

`get_features` [¶](#)

```
get_features() -> List[MetricFeature]
```

Gets all the features in this Operator instance.

Returns:

Type	Description
list of MetricFeature	List of MetricFeature.

`get_type` [¶](#) staticmethod

```
get_type() -> str
```

`remove_feature` [¶](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	str	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
bool	If the feature is removed.

`rename_feature` [¶](#)

```
rename_feature(old_name: str, new_name: str)
```

`to_csv` [¶](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	Path or str	Path to where the csv will be written.	<i>required</i>

`to_dict` [¶](#)

```
to_dict() -> List[Dict]
```

to_pandas [¶](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
DataFrame	Pandas dataframe containing features and all their editable attributes.

update_from_csv [¶](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
path ¶	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

update_from_dict [¶](#)

```
update_from_dict(dict_feats: list[dict])
```

update_from_pandas [¶](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
pd_df ¶	DataFrame	Pandas dataframe with the operator features.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

operator

Base class for all Plan Operators.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

BaseFeaturesOperator

```
BaseFeaturesOperator(pyjnius_java)
```

Bases: [BaseOperator](#) , ABC

Abstract base class to provide additional Python functionality to Plan Operators containing features.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_feature

```
add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature
```

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature 	BaseFeature	New Feature.	<i>required</i>
logger 	bool	Whether to log info message or not.	True

Returns:

Type	Description
BaseFeature	New Feature.

diff

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
other ¶	<code>BaseFeaturesOperator</code>	BaseFeaturesOperator to compare against.	<i>required</i>
only_diff ¶	<code>bool</code>	Show features with differences only. Defaults to 'true'.	<code>True</code>

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

get_dependencies [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

get_dependencies_by_type [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, list[str]]]
```

get_feature `abstractmethod` [¶](#)

```
get_feature(name: str)
```

get_features `abstractmethod` [¶](#)

```
get_features()
```

get_type `abstractmethod` `staticmethod` [¶](#)

```
get_type() -> str
```

Gets the operator type.

Returns:

Type	Description
<code>str</code>	Operator type.

remove_feature [¶](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
name ¶	<code>str</code>	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
<code>bool</code>	

Type	Description
	If the feature is removed.

rename_feature [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

to_csv [↗](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path to where the csv will be written.	<i>required</i>

to_dict [↗](#)

```
to_dict() -> List[Dict]
```

to_pandas [↗](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
DataFrame	Pandas dataframe containing features and all their editable attributes.

update_from_csv [↗](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

update_from_dict [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

update_from_pandas [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ¶	<code>DataFrame</code>	Pandas dataframe with the operator features.	<i>required</i>

See Also

`Plan.from_csv` : For guidelines to input csv data in the right format.

BaseOperator [¶](#)

```
BaseOperator(pyjnius_java)
```

Bases: `JavaWrapper` , `ABC`

Abstract class to provide additional Python functionality to all Plan Operators.

Notes

When comparing operators we consider two operators to be the same if they are of the same type and have the same name. This allows a more intuitive comparison between plans where we want to analyze differences of "the same" operator. In such cases we need to first identify the "same" operator between plans so that then we can check how their contents are different.

For reference, `BaseOperator.getName` returns the name of the operator as it appears in the UI.

subclasses **class-attribute** **instance-attribute** [¶](#)

```
subclasses: List[JavaWrapper] = []
```

get_dependencies **abstractmethod** [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

Gets the features and their direct dependencies within the operator.

Dependencies are not recursive.

Returns:

Type	Description
<code>dict</code>	Dictionary of FeatureUID and the name of their direct dependencies.

get_dependencies_by_type **abstractmethod** [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, List[str]]]
```

Gets the features and their direct dependencies (by type) within the operator.

Dependencies are not recursive.

Returns:

Type	Description
<code>dict</code>	Dictionary of FeatureUID and the dict mapping the types and names of their direct dependencies.

`get_type` abstractmethod staticmethod [↗](#)

```
get_type() -> str
```

Gets the operator type.

Returns:

Type	Description
<code>str</code>	Operator type.

`rename_feature` abstractmethod [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

Renames the dependencies or features matching the old name.

Parameters:

Name	Type	Description	Default
<code>old_name</code> ↗	<code>str</code>	Old name to change from.	<i>required</i>
<code>new_name</code> ↗	<code>str</code>	New name to change to.	<i>required</i>

`to_dict` abstractmethod [↗](#)

```
to_dict() -> List[Dict]
```

Exports all the features in the operator as a list of dictionaries.

Returns:

Type	Description
<code>list of dict</code>	List containing features and all their editable attributes.

`update_from_dict` abstractmethod [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

Replaces the operator features with the ones on the list.

Parameters:

Name	Type	Description	Default
<code>dict_feats</code> ↗	<code>list of dict</code>	List containing the features in a dict format.	<i>required</i>